

# **Assembly Arithmetic Algorithms**

32-bit Linux Edition

# Preface

This book is the Linux edition of Assembly Arithmetic Algorithms. The first book was for 16-bit DOS programming using Assembly. This book is for 32-bit Linux programming using the same assembly language for Intel machines.

Although the DOS book was useful, reading it first is not required to understand this book because programming for Linux is very different from what it was for DOS. Using interrupt 0x21 no longer works because the operating system is different. Instead, everything will use interrupt 0x80 because this is how the Linux kernel is called in assembly language when running on a 32-bit Linux operating system.

Keep in mind that most Linux distributions today are capable of running 32-bit or 64-bit code. The reason I am teaching 32-bit is that I know it better, and because programs using 32-bit instructions are usually smaller than the same thing written in 64-bit.

The Linux distribution I am using to write and test my examples is Debian version 12 (bookworm). It works well enough for what I do on a daily basis. These examples should Assemble on almost any distribution of Linux because the system call numbers are the same on any distro running x86 Intel because they are hardcoded into the Linux kernel.

This standard means that the information here is useful to anyone who is running Linux unless their Central Processing Unit is an ARM, RISC-V, or something else. Intel is still the most popular at this time and is traditionally the same type of machine that usually runs Microsoft Windows.

So if you happen to have an old computer around with Windows that runs too slow, you might decide to install Linux on it and get started with the world of Free Software and writing your own programs. Linux is a better environment for programming in ANY language (I will explain more about that later).

# Introduction

First, let me introduce this book by telling you what I will teach you. By the end of this book, you will have enough information to write any text-based console program in the form of a 32-bit Linux “ELF” file.

The “ELF” format is an acronym for “Executable and Linkable Format”. This format is used on all modern Linux operating systems and some Unix systems like FreeBSD. The header for this format is slightly complicated, but FASM is capable of generating one for you so that you don’t generally have to worry about creating it yourself.

If you are a user who prefers NASM, there are also ways to have one generated for you by the GNU linker, which is already installed on any system that has the GCC compiler. The linker does not require you to write C code because it can also link standard ELF objects created by NASM.

## Required Knowledge

To get the most out of this book, some background on the Binary and Hexadecimal numeral systems is going to be helpful, but this is not strictly required because I will be providing functions you can use in your code that will convert between decimal (base ten), binary (base two), and hexadecimal (base 16).

However, I would say that experience in at least one programming language is necessary for an understanding of terminology like “arrays”, “pointers”, “addresses”, “integers”, “floating point”, etc. I recommend the C Programming Language as a start. C++ is also a good starting language, but it tends to abstract details away that directly apply to Assembly Language, which is the lowest level a human can go for understanding a computer.

But specifically in this book about Linux assembly, I will make comparisons between C programs and Assembly because C is usually considered the native language of the Linux kernel. The manual pages of the system calls also show the C prototypes of the functions. Translating these into Assembly has not been easy because information on Assembly doesn’t come by as easy as C or C++. I hope my book will save you the time and trouble I experienced when I first starting writing Assembly programs for Linux.

## Low Level

Low level is a term that confuses people. People think something high-level is better than low-level. In simple terms, humans consider themselves superior to machines and therefore think themselves higher or more important because of their abstract thought.

A computer thinks only in terms of numbers. A computer may not understand “high-level” abstractions such as love, religion, philosophy, etc, but that is not its job. A

computer must add, subtract, multiply, and divide. These are the four arithmetic functions that many humans struggle with.

Therefore, I ask you, between a human and a computer, who is really low level or high level? In the age of Artificial Intelligence taking over human jobs and beating humans at Chess, we would all do well to take this question seriously.

I wrote this book because I think like a machine, and I hope to help others think this way because it is the best way to learn programming and control your computer by writing Assembly Language programs, or to go back to your favorite programming language with a greater understanding of why things work as they do.

## Why Linux?

Linux is a controversial operating system because most people who use a computer just use the operating system that was preinstalled on their computer when they bought it. Usually, this is Windows by Microsoft, MacOS by Apple, or ChromeOS by Google.

Therefore, most of these people don't know that other operating systems exist, unless they are programmers. I am a programmer, and I switched to Linux many years ago on a computer that formerly ran Windows XP and only had 256 megabytes of RAM. It booted up faster than Windows did, and I learned to use the terminal quickly to compile and run my C programs. My DOS background certainly helped me adjust.

The average person is not interested in learning about operating systems. They are usually also not interested in learning about number bases other than the default decimal system humans use.

This book is not for ordinary people. It is for smart people who love to learn how computers work and also to customize their computing experience in a way that only Linux allows. The source code of everything on a GNU/Linux system is available for people to modify anything they like, either by recompiling the source code or by changing a few configuration files, which are usually in a plain text format.

But to explain to the average person why I chose Linux as my primary operating system, I have two reasons.

0. My former favorite OS, DOS, is really old and not suited for the modern world of gaming and programming with better text editors and compilers. Linux is as close as I can get to DOS because I can run commands the way I like, but also enjoy the benefits of a GUI environment when I want to.
1. Microsoft Windows actively sabotages programming by its forced system updates, reboots, and AI advertising forced into every part of the OS. Also, it is REALLY slow because it spends most of the CPU power spying on you and sending the data to Microsoft and the government so they can kill you as soon as you do something they don't like.

# Chapter 1: The First Program

For this chapter, I will explain give the source code of an example program that works in Linux, how to assemble it using the tools FASM or NASM, and finally, how the program works line by line.

First, here is the source code of a program that looks like nonsense but does something really cool.

```
format ELF executable
```

```
mov eax,4    ; invoke SYS_WRITE (kernel opcode 4 on 32 bit systems)
mov ebx,1    ; write to the STDOUT file
mov ecx,msg  ; pointer/address of string to write
mov edx,13   ; number of bytes to write
int 80h
```

```
mov eax,1    ; function SYS_EXIT (kernel opcode 1 on 32 bit systems)
mov ebx,0    ; return 0 status on exit - 'No Errors'
int 80h
```

```
msg db 'Hello World!',0Ah,0
```

You will need an assembler. My first recommendation is FASM, the Flat Assembler.

<https://flatassembler.net/>

You can download FASM and install it by putting it in your path. The instructions for doing this depend on your operating system but it can be done on Windows, Linux, or even within a DOS operating system. You can assemble your source from any operating system but you will still need a Linux system (either real or emulated) to run the programs made from the source code in this book.

## Assemble with FASM

To assemble this program with FASM, place the source in a file named “main.asm” and run this command from a terminal opened into the same folder as the file.

```
fasm main.asm
```

FASM will automatically create a “main” in the ELF format that is executable. FASM understands by the command of “format ELF executable” that we intend this file to contain code that will be executed. But we are not read to run it yet. We also have to give the operating system permission to run it! The following command will do the trick.

```
chmod +x main
```

Now the file will have the Linux permissions to be executed. To execute the program we simple do this command:

```
./main
```

The dot and backslash is how you run a program that exists in the current directory. The process is the same as any other Linux command except that the “./” is not needed since they are usually in the path already when you install them with whatever package manager your Linux distribution uses.

Anyway, when you run the program, it should output  
Hello World!

If you have Gnu Make installed on your Linux system (which I highly recommend), you can copy paste the following into a “makefile” to easily build and run your code just by typing “make”.

## makefile for FASM users

```
main-fasm:
    fasm main.asm
    chmod +x main
    ./main
```

A makefile is basically like a shell script except it has more options. For the most part, you don’t need to worry about the details of how it works because I will be providing any makefiles you will need for the examples in this book. However, I have read the documentation of Gnu Make and I frequently use it to manage converting my books with Pandoc and managing updates to my Git repositories. It is a fabulous tool for any programmer because it allows setting up “rules” for commands you need to run.

## Assemble with NASM

You can assemble the example program with NASM instead of FASM if you wish. However, you will need to make a few small changes. The following is a form that will be acceptable to NASM and the GNU linker.

```
global _start

_start:

section .text

mov eax,4    ; invoke SYS_WRITE (kernel opcode 4 on 32 bit systems)
mov ebx,1    ; write to the STDOUT file
mov ecx,msg  ; pointer/address of string to write
mov edx,13   ; number of bytes to write
int 80h

mov eax,1    ; function SYS_EXIT (kernel opcode 1 on 32 bit systems)
mov ebx,0    ; return 0 status on exit - 'No Errors'
int 80h

section .data

msg db 'Hello World!',0Ah,0
```

If you take a look at it, you will see it is the exact same program but that things are separated into “.text” and “.data” sections. This is a rule that most programs follow and it is required by the GNU linker which is very strict about which is code(.text) and which is data that will be read from or written to. Also you will see the global symbol \_start which is expected by the linker.

The following command will make an ELF object.

```
nasm -f elf32 main.asm
```

The object created is not executable but you can make one that is by linking it with the GNU Linker.

```
ld -m elf_i386 main.o -o main
```

And finally, you can run the program in the traditional way.

```
./main
```

## Interrupt FAQ

I know you are probably a little bit confused at this point and have many questions such as

- What is an interrupt?
- What is a system call?
- Why do you write your numbers in hexadecimal?
- What is a register?

An interrupt is where your code passes control to the operating system which has loaded code to run various functions. Only the Linux kernel has authority to access devices like the console and file system. By placing the right numbers in the right registers and calling interrupt 0x80, we let the kernel “interrupt” our program and then display the data using the information we have given it. Obviously the writers of the kernel know more than I do and chose which registers are used in which functions.

A helpful reference I use to remind myself which call numbers to use for interrupts/syscalls is here:

<https://www.chromium.org/chromium-os/developer-library/reference/linux-constants/syscalls/#x86-32-bit>

Although that reference is for Chromium-OS, it is Linux based and so the information is valid. It is only a brief reference and I will be explaining more as this book continues.

The reason the interrupts and other numbers in my code are in hexadecimal is because most assembly language books and tutorials use them. Hexadecimal is objectively more convenient because it can be more easily converted to and from binary. For now just remember that interrupt 80h is actually 128 decimal and for some reason was chosen as the interrupt number for Linux on Intel systems. I have chosen to stick mostly with

hexadecimal for this book because it will be relevant later on when I show you other tools which can be used if you understand hexadecimal!

A register is a special variable that exists on a specific CPU type. DOS, Windows, and most Linux operating systems run on an Intel 8086 compatible CPU. I will explain the registers and their functions.

## The General Purpose Registers

There are 8 general purpose registers.

- EAX: The Accumulator Register
- EBX: The Base Register
- ECX: The Count Register
- EDX: The Data Register
- ESI: The Source Index
- EDI: the Destination index
- EBP: The Base Pointer
- ESP: The Stack Pointer

If you read the DOS version of this book, you will notice these are the same names as the 8 general 16-bit registers except prefixed with an E. The E stands for Extended. You can still access the 8 and 16 bit registers, for example AX is still the lower half of EAX and is divided into the 8 bit registers AH and AL. I still frequently use these in 32 bit code.

These registers are “general” in that they can do many things, but they each have more “specific” uses also.

In my source code, I use lowercase for the names of instructions and registers, but for this section, I listed them in capital letters because they are actually acronyms named for their purpose according to what Intel had in mind when making the 8086 and above Central Processing Units.

Most of the time, I stick with only EAX,EBX,ECX,EDX when writing my programs. If I need an extra registers, I will use EBP,ESI,EDI. There are special instructions for them but these are outside the scope of what I am trying to teach with this book.

You might wonder, why isn’t there EX,FX,...YX,ZX? Perhaps in a perfect world there should have been, but they probably didn’t want to spend the extra money on having extra registers for all 26 letters of the alphabet.

In the next chapter I will show you a few other ways to assemble the program from this chapter using NASM and GAS. I will also be explaining the pros and cons of executable files compared to linkable ELF objects.

## Chapter 2: Assembler Choices

Most of the time I used the FASM assembler. It is the Flat Assembler which means it generates flat or raw binary executable files that are already to run. The information in the file includes the ELF header at the beginning and then the code and data defined by you in your program.

In chapter 1, I showed the same program using both FASM and NASM. The code is identical but the assembler directives are different. If you are using FASM, the “format ELF executable” at the start of the source is enough for FASM to auto generate what you need. This does not exist in NASM, so I have created a header file that achieves the same thing!

The contents of my custom built 32 bit ELF header is below. It can be included in any NASM program so you only need to copy paste this file once or download it from my repository.

### chaste-elf-32.nasm

```
;Chastity's Source for ELF 32-bit executable creation
;
;All data as defined in this file is based off of the specification of
the ELF file format.
;I first looked at the type of file created by FASM's "format ELF
executable" directive.
;It is great that FASM can create an executable file automatically.
(Thanks Tomasz Grysztar, you are a true warrior!)

;However, I wanted to understand the format for theoretical use in other
assemblers like NASM.
;Therefore, what you see here is a complete Hello World program that
should work within NASM
;to create an executable file without using a linker. It worked
perfectly on my machine running Debian Linux and NASM version 2.16.01.

;The Github repository with the spec I used is here.
;<https://github.com/xinuos/gabi>
;And this is the wikipedia article which linked me to the specification
document
;<https://en.wikipedia.org/wiki/Executable\_and\_Linkable\_Format>

;This file contains a raw binary ELF32 header created using db,dw,dd
commands.
;After that, it proceeds to assemble a real "Hello World!" program

;Header for 32 bit ELF executable (with comments based on specification)

db 0x7F,"ELF" ;ELFMAGIC: 4 bytes that identify this as an ELF file. The
magic numbers you could say.
db 1           ;EI_CLASS: 1=32-bit 2=64-bit
```

```

db 1          ;EI_DATA: The endianness of the data. 1=ELFDATA2LSB
2=ELFDATA2MSB For Intel x86 this is always 1 as far as I know.
db 1          ;EI_VERSION: 1=EV_CURRENT (ELF identity version 1) (which
is current at time of specification Version 4.2 I was using)
db 9 dup 0    ;padding zeros to bring us to address 0x10
dw 2          ;e_type: 2=ET_EXEC (executable instead of object file)
dw 3          ;e_machine : 3=EM_386 (Intel 80386) 0x3E (AMD x86-64
architecture)
dd 1          ;e_version: 1=EV_CURRENT (ELF object file version.)

```

```

p_vaddr equ 0x8048000 ;the absolute base address where the file is
loaded into memory
e_entry equ 0x8048054 ;program starts running at this address (right
after header)

```

```

dd e_entry    ;e_entry: the address at which the program starts running
dd 0x34       ;e_phoff: where in the file the program header offset is
db 8 dup 0    ;e_shoff and e_flags are unused in this example, therefore
all zeros
dw 0x34       ;e_ehsize: size of the ELF header
dw 0x20       ;e_phentsize: size of program header which happens after
ELF header
dw 1          ;e_phnum: How many program headers. Only 1 in this case
dw 0x28       ;e_shentsize: Size of a section header
dw 0          ;e_shnum number of section headers
dw 0          ;e_shstrndx: section header string index (not used here)

```

;That is the end of the 0x34 byte (52 bytes decimal) ELF header. Sadly, this is not the end and a program header is also required (what drunk person made this format?)

```

dd 1          ;p_type: 1=PT_LOAD
dd 0          ;p_offset: Base address from file (zero)
dd p_vaddr    ;p_vaddr: Virtual address in memory where the file will
be.
dd p_vaddr    ;p_paddr: Physical address. Same as previous

```

;The file\_size variable I have defined uses some trickery to get the size of the file.  
;An EOF constant (End Of File) is defined at the end of the program code  
;By subtracting the program virtual address from that address,  
;I get the actual number of bytes of this entire program

```

file_size equ EOF-p_vaddr ;Place the actual size of the file using NASM
address constants

```

```

dd file_size  ;p_filesz: Size of file image of the segment. Must be
equal to the file size or greater
dd file_size  ;p_memsz: Size of memory image of the segment, which may
be equal to or greater than file image.

```

```

dd 7          ;p_flags: permission flags: 7=4(Read)+2(Write)+1(Execute)
dd 0x1000     ;p_align; Alignment (same page alignment that FASM uses
of 4096 bytes)

```

;important NASM directives

```
use32          ;tell assembler that 32 bit code is being used
org p_vaddr    ;origin of new code begins here
```

Next, here is a short program which properly includes the header above.

## main.asm

```
%include 'chaste-elf-32.nasm'
```

```
mov eax,4      ;invoke SYS_WRITE (kernel opcode 4 on 32 bit systems)
mov ebx,1      ;write to the STDOUT file
mov ecx,msg    ;pointer/address of string to write
mov edx,13     ;number of bytes to write
int 80h
```

```
mov eax,1      ;function SYS_EXIT (kernel opcode 1 on 32 bit systems)
mov ebx,0      ;return 0 status on exit - 'No Errors'
int 80h
```

```
msg db 'Hello World! ',0Ah,0
```

```
EOF equ $ ; End Of File label
```

This program uses the “%include” directive to include the header file. This file will automatically create the header file needed to run the program by the operating system.

The “EOF equ \$” is required at the end because this makes an “EOF” label equal to the current address. The header uses the EOF label to tell it how big the file is. The size of the file must be hard coded into the header.

Linux is very strict and picky about what it will load. It doesn’t just load any random file like DOS does. But if you actually assemble and run this program, it will work flawlessly.

The following makefile was written for assembling and running in one “make” command. I highly recommend GNU make so you can make use of it and save yourself typing each command.

## NASM makefile

```
main-nasm:
    nasm main.asm
    chmod +x main
    ./main
ndisasm:
    ndisasm -b 32 -o 0x8048054 -e 0x54 main
```

Although this works, you might wonder why I went to the trouble of writing this header file to be used with NASM. The following are my reasons:

1. FASM is better, but not always available on every operating system.

2. NASM is written in C and ported to more platforms
3. Writing the header manually allows me full control and understanding of the process.

I believe item 3 is most important because if I was to just allow FASM to generate the header and not know what it was doing, I would not be the KIND OF PERSON who writes assembly language. It is much better to program in C or C++ if you just want to get programs working and don't care about what is happening behind the scenes.

In fact, I will prove this with one more example. This next program does the exact same thing except that it uses the GNU assembler (also called GAS) and linker.

## GNU Assembler

The GNU Assembler is automatically installed on systems that have the GCC compiler, which uses it as part of the process of transforming C programs into executable code. The compiler normally creates an assembly file, assembles it, and then deletes the source.

However, you can supply your own assembly code. The following is one I wrote that behaves the same as the NASM program above.

I must warn you though, GAS uses the AT&T syntax which uses a different order of operands. For example, the destination register is on the right and the source address or number is on the left. This is opposite of FASM and NASM source code which uses the Intel Style.

History Note:

Intel made the 8086 Central Processing Unit and its descendants, but AT&T (yes the phone company) was responsible for the C programming language and UNIX. Because of this, they made their own style of writing assembly that is used by some C compiler systems to this day.

### main.s

```
# Using Linux System calls for 32-bit
# Tested with GNU Assembler on Debian 12 (bookworm)
# Writes a message to standard output with Linux system calls

.global _start

.text

_start:

mov     $0xD,%edx # copy number of bytes to edx
mov     $msg,%ecx # address of string to output
mov     $0x1,%ebx # file handler 1 is stdout
mov     $0x4,%eax # system call 4 is write
int     $0x80
```

```

mov    $0x1,%eax    # system call 1 is exit
mov    $0x0,%ebx    # we want to return code 0
int    $0x80        # end program with system call

.data    # must declare data section for mutable memory

msg:
.ascii  "Hello World!\n"

```

I have created a makefile for this program as well. I use the gcc command because it automatically calls “as” and “ld” for us. The arguments I used are for tricking it into not including the C standard library. Remember, GAS was designed to process the output of assembly created from a C compiler and they didn’t plan on us writing our own, partly because the syntax is a lot harder to understand than Intel style used by FASM and NASM.

## GAS makefile

```

main-gas:
    gcc -nostdlib -nostartfiles -nodefaultlibs -static main.s -o main -
m32
    ./main

```

To sum up this chapter, you have the choice of at least 3 assemblers to use when getting started with Intel Assembly language for Linux Operating Systems. Here is a breakdown

- FASM: Easy to use and does a lot of work for you.
- NASM: Portable and reliable but does not make headers for you.
- GAS: Exists almost everywhere a C compiler is but uses a hard to read syntax!

I will be using FASM for the rest of this book because I have the most experience with it and it is available for Linux. It was also written in assembly and is much faster than the other two assemblers I have mentioned.

There are many more assemblers for Intel processors but I will not be covering them unless requested by readers in the future.

Congratulations! If you have followed the book this far, you now know enough to get “Hello World!” on the screen in Assembly language for 3 different assemblers. The following chapters will cover more advanced ways of printing strings and integers using the FASM assembler, but can be translated into other assemblers if you learn the subtle differences between them.

## Chapter 3: Linux System Calls

Any book about Assembly programming for Linux is going to have to include information about system calls. This book will use what I consider the most useful of the system calls to achieve everything that the C standard library can do.

For example, we need to be able to open files, read from them, write to them, and close them. These are the 4 fundamental operations and without them, nothing would work for building text based programs and getting results. Otherwise we will never know if a program was written correctly.

The Linux kernel has hundreds of system calls. However only about 8 of them have I ever used. Each call has a number which may be different depending on which CPU type you have and what your operating system supports. Here is a command I can use to find the information on which numbers go with which function names. This will be extremely relevant.

```
find /usr/include -name "unistd_*.h"
```

On my system, that command shows 4 files.

```
/usr/include/x86_64-linux-gnu/asm/unistd_64.h
/usr/include/x86_64-linux-gnu/asm/unistd_x32.h
/usr/include/x86_64-linux-gnu/asm/unistd_32.h
/usr/include/x86_64-linux-gnu/bits/unistd_ext.h
```

Specifically, it is the file:

```
/usr/include/x86_64-linux-gnu/asm/unistd_32.h
```

which shows us the numbers for all the system calls in a 32 bit Linux environment. Here are a few of them.

```
#define __NR_exit 1
#define __NR_fork 2
#define __NR_read 3
#define __NR_write 4
#define __NR_open 5
#define __NR_close 6
#define __NR_waitpid 7
#define __NR_creat 8
#define __NR_link 9
#define __NR_unlink 10
#define __NR_execve 11
#define __NR_chdir 12
#define __NR_time 13
#define __NR_mknod 14
#define __NR_chmod 15
#define __NR_lchown 16
#define __NR_break 17
#define __NR_oldstat 18
#define __NR_lseek 19
```

```

#define __NR_getpid 20
#define __NR_mount 21
#define __NR_umount 22
#define __NR_setuid 23
#define __NR_getuid 24
#define __NR_stime 25
#define __NR_ptrace 26
#define __NR_alarm 27
#define __NR_oldfstat 28
#define __NR_pause 29
#define __NR_ftime 30
#define __NR_stat 31
#define __NR_gtty 32

```

You may have also noticed that there is a 64 bit file with the same functions in a different order. I would have to write a separate book on the 64 bit calls because they use different registers beside different numbers.

In any case, these are the same system calls regardless of whether you are calling them from a C program or an Assembly program. In chapters 1 and 2, we have already used two system calls in our Hello World program. The EAX register was loaded with 4 for the write call and then later was loaded with 1 for the exit call to end the program.

Getting the system call numbers into EAX is easy enough but how did I know which arguments go in which registers?

As it turns out, most Linux distributions have manual pages on all these calls. To access these manual pages, you always enter “man 2 name\_of\_the\_function”. For example the write call can be obtained this way.

```
man 2 write
```

It is a big page to read but I do suggest reading it all for a proper understanding of how it works. Included is the C Programming Language signature for the function.

```
write(int fd, const void buf[.count], size_t count);
```

Notice that this takes 3 arguments. The first is the file descriptor number. In the case of standard output, it is 1. Next, it needs a pointer with bytes which will be written to standard output. The final argument is to tell it how many bytes to write from this address.

Using this information, we can form a complete program in C.

## C Hello World with Write System Call

```

#include <unistd.h>

int main(int argc, char *argv[])
{
    write(1, "Hello World!\n", 13);
    _exit(0);
}

```

```
}
```

If you are on a Linux system, which you should be for learning from this book, you can use the following command to compile and run it.

```
gcc -Wall -ansi -pedantic main.c -o main && ./main
```

The reason I show a C programming example in a book about assembly is because C is the native language for the Linux operating system but all the system calls are also accessible from Assembly language as well. Look again at the C program above and then look again at the program from chapter 1.

## FASM Hello World with Write System Call

format ELF executable

```
mov eax,4    ; invoke SYS_WRITE (kernel opcode 4 on 32 bit systems)
mov ebx,1    ; write to the STDOUT file
mov ecx,msg  ; pointer/address of string to write
mov edx,13   ; number of bytes to write
int 80h
```

```
mov eax,1    ; function SYS_EXIT (kernel opcode 1 on 32 bit systems)
mov ebx,0    ; return 0 status on exit - 'No Errors'
int 80h
```

```
msg db 'Hello World!',0Ah,0
```

These two programs are the EXACT same thing written in two different languages. Also notice that the arguments for the write call are the same as in the C signature. The registers are EBX,ECX,EDX. This is because B,C,D is the order of the alphabet!

Of course this consistency is not followed past 3 arguments. After that, the order is ESI,EDI,EBP. However this is rare because most functions do not use more than 3 arguments.

The write call technically uses 4 registers because EAX must have the system call number before we call int 0x80.

This means we still have 3 registers that are not used by system calls and can be used for other purposes like holding temporary numbers we need to use later.

I often refer to this online list to help me remember which registers are used as which arguments.

<https://www.chromium.org/chromium-os/developer-library/reference/linux-constants/syscalls/#x86-32-bit>

It may be a reference for Chromium OS but this is still a Linux compatible system and so it is valid for all Linux distributions as long as they support 32 bit system calls. These functions we are calling are part of the Linux kernel which is actually used on all kinds of operating systems that are not called Linux. Android is a popular example.

I have explained the way that system calls work, specifically the “write” call. The “exit” call is also included because it is how you must end a program in Assembly.

But there is also the problem that system calls are inconvenient. Having to keep in my head what registers to load with what can get pretty confusing. That is why I created a function that uses the write call but manages printing strings and automatically tells the write call how long the string is.

In the next chapter, I will show you the putstring function. But remember, all of the code for input and output in a Linux system is going to require one of the system calls. Knowing when to use the system calls becomes important from the perspective of a C or Assembly programmer.

## Chapter 4: putstring

Perhaps it is God's sense of humor that I am beginning chapter 4 with a function that uses function number 4 of interrupt 0x80.

The write system call is the most useful of all the calls because it is the only way to get output from a program. You have already seen examples of this call being used in the first three chapters. However, the downside of this system call is that you must tell it exactly how many bytes of a string to write. It makes no assumptions about how long the string is.

If you are coming from a C programming background, you will know that it is a common convention that strings are an array of bytes ending in 0. I agree with this convention because it just makes sense. The byte 0 doesn't map to a printable character, and the number zero generally has the sense of "nothing" because there are no more bytes in the string to print.

It might make sense to my mind, but I have to logically get the computer to understand it. To achieve this, I will show you a C program first and then the Assembly version that does the same thing.

### C unistd putstring

```
#include <unistd.h>

int putstring(const char *s)
{
    int count=0;          /*used to count how many bytes will be written*/
    const char *p=s;      /*pointer used to find terminating zero of string*/
    while(*p){p++;}       /*loop until zero found and immediately exit*/
    count=p-s;            /*count is the difference of pointers p and s*/
    write(1,s,count);     /*the unix system call way of writing the bytes*/
    return count;         /*return how many bytes were written*/
}

int main(int argc, char *argv[])
{
    putstring("The putstring function can print any string!\n");
    _exit(0);
}
```

The program source above uses the **unistd.h** interface to the Unix Standard way of doing things. Sometimes this is referred to as POSIX. POSIX is the (Portable Operating System Interface), which is a group of standards that Linux and Unix-like systems, such as BSD (and its descendants FreeBSD, NetBSD, and OpenBSD), and even MacOS, which is actually based on Unix.

So basically, the sets of functions available, such as write, read, open, close, lseek, and exit, are part of a special Unix Standard Library that can be used on Linux and every

operating system except for DOS and Windows because those have different standards set by Microsoft.

Because this book is about programming on Linux, I do promote this as the best way to go about programming on Linux. However, I also consider that my readers may be used to a more traditional C Standard Library approach to programming. Therefore, I will also show you the same thing using the `fwrite` function from the **`stdio.h`** header.

## C `stdlib` `putstring`

```
#include <stdio.h>

int putstring(const char *s)
{
    int count=0;                /*used to count how many bytes will be
written*/
    const char *p=s;           /*pointer used to find terminating zero of
string*/
    while(*p){p++;}            /*loop until zero found and immediately
exit*/
    count=p-s;                  /*count is the difference of pointers p and
s*/
    fwrite(s,1,count,stdout);
/*https://cppreference.com/w/c/io/fwrite.html*/
    return count;               /*return how many bytes were written*/
}

int main(int argc, char *argv[])
{
    putstring("The putstring function can print any string!\n");
    return 0;
}
```

If you compare the two C programs, you will see that they are line by line the exact same thing, except they include a different header, different functions to send the output to the terminal, and return from the main function in different ways.

The consistency in behavior is why it is easy to translate programs using Unix calls into C Standard calls and vice versa.

In any case, here is a breakdown of how the `putstring` function works regardless of which version you use.

A char pointer named `s` is passed to the function when it is called.

A count variable is created for use in telling how many bytes need to be printed. This count is initialized to 0 but will be changed later.

A pointer named `p` is set to the same address as `s`.

A short loop occurs, which checks the byte at the address at pointer `p`. If it is zero, the loop ends. Otherwise, `p` is incremented with the `++` operator. Adding 1 to a pointer means

it will point to the next byte. Sooner or later, it will find that a byte is 0 because the C compiler forces any string literal inside double quotes to end in a zero byte.

The count is set to  $p$  minus  $s$ . Yes, you can subtract pointers from each other in the C language. This pointer arithmetic is the fastest way to extract the length of the string. By subtracting the start of the string ( $s$ ) from the place where the 0 was found ( $p$ ), we can know that it will always have the exact length of the string.

As far as the `write` and `fwrite` calls. They are the same thing, but the number of arguments and their order are slightly different. See the following links for a more accurate breakdown.

<https://man7.org/linux/man-pages/man2/write.2.html>

<https://cppreference.com/w/c/io/fwrite.html>

After the data is written, the `putstring` function returns the number of bytes written. You can ignore this most of the time, but it can be helpful if you need the length of the string to be saved for some operation later. Technically, the C `strlen` function can also be used for finding the length of a string, but there is a strategic reason I did not use that function in my implementation of `putstring`.

The reason is that when we are coding in Assembly, C functions don't exist unless we specifically link assembly programs to the C library. I recommend against this and therefore won't teach you how to do this. More importantly, it is unnecessary because we can always write our own functions in Assembly that run much faster and do exactly what we want.

Now I have shown you two possible interfaces in C for writing a function named “`putstring`” that counts the length of a string and prints it to standard output. All of this effort was made so that when you see the Assembly version, you already have a basic idea of how it works, so that you won't be overwhelmed at how complicated it looks.

And don't worry, I will still be explaining more about it after the code for those who still may not get it.

## Assembly `putstring`

format ELF executable

main:

```
mov eax,string0
call putstring
```

```
mov eax,1 ; invoke SYS_EXIT (kernel opcode 1)
mov ebx,0 ; return 0 status on exit - 'No Errors'
int 80h
```

```
string0 db 'The putstring function can print any string! ',0Ah,0
```

```

putstring:

push eax
push ebx
push ecx
push edx

mov ebx,eax

putstring_strlen_start:

cmp [ebx],byte 0
jz putstring_strlen_end
inc ebx
jmp putstring_strlen_start

putstring_strlen_end:
sub ebx,eax

mov edx,ebx ;number of bytes to write
mov ecx,eax ;pointer/address of string to write
mov ebx,1   ;write to the STDOUT file
mov eax,4   ;invoke SYS_WRITE (kernel opcode 4 on 32 bit systems)
int 80h     ;system call to write the message

pop edx
pop ecx
pop ebx
pop eax

ret

```

The Assembly program that uses `putstring` follows all the same logic as the two C programs earlier in this chapter.

However, you may notice that the `main` function appears first, and then the `putstring` function appears after the `exit` call to end the program, and also after where `string 0` is defined.

Most C compilers force you to define a function before it can be called. FASM has no such restriction, and I frequently place all my data and other functions after the `main` function.

Because function 1 of Interrupt 0x80 is the `exit` call, this means the program will end and not accidentally run anything after it.

It is possible to define the functions and the data before the `main` function, much like you would do in the C language, but this means that you would need to do a “`jmp main`” instruction to skip executing those. Therefore, I reversed the order from the C convention because it saves a few bytes of space and because it protects me from running code that I didn’t mean to. Writing Assembly language is very prone to error because the source

typically takes more lines, and it is easier to get lost and forget what I was doing.

The `putstring` function is written in a way that it expects the address of the string we want to print to be placed in the `eax` register before it is called. I could have used any of the registers but I chose `eax` because 'A' is the first letter of the alphabet and so it is considered the first and most important in my code.

At the beginning and end of the `putstring` function, you may notice the `push` and `pop` instructions. These save and restore the values of registers to the "stack". The stack is a LIFO (Last In First Out) data structure which is useful because the registers are modified whenever we make a system call. By pushing registers in the order `eax,ebx,ecx`,and `edx`, we are saving them to a memory location which is indexed by the Stack Pointer or `esp`. To restore them in the proper order, we have to `pop` them back in the reverse order of `edx,ecx,ebx`,and `eax`.

I understand this is a lot of information that seems pointless, but that is because it requires a good understanding of pointers. Note however that other languages use the stack when you call functions but they hide this from the users. Assembly requires getting comfortable with pointers.

## Pointers vs Integers

One thing that sometimes trips up new assembly programmers is how registers can sometimes function as pointers and other times as plain integers.

For example

```
mov eax,string0
```

Means we are moving the address of the string into the `eax` register. The registers starting with the letter E all store 32 bits of data. They are integers, or simply numbers. However, addresses are also numbers, and therefore it is totally normal to use the same register as both a pointer to an address and yet also use it as a regular number elsewhere.

For example

```
cmp [ebx],byte 0
```

Means we are comparing the address at `ebx` with 0. We know that `ebx` is being used as a pointer because it is in the brackets. Here, `ebx` functions just like `"*p"` does in the C version because `ebx` is being used as the pointer, just like `p` was in the C language. Notice also that it is required to include the "byte" specifier so the assembler knows we are searching for an 8-bit value (byte) instead of a 16-bit (word) or 32-bit (dword).

But I think the final best example of how integers and pointers are mixed is looking at the part of the `putstring` function that takes place after a zero is found.

```
sub ebx,eax ;subtract eax from ebx
```

```
mov edx,ebx ;number of bytes to write  
mov ecx,eax ;pointer of string to write
```

```
mov ebx,1    ;write to the STDOUT file
mov eax,4    ;function number 4
int 80h
```

Registers `eax` and `ebx` are both used as pointers but then the usage of `ebx` suddenly changes. We subtract `eax` from `ebx` to get the number of bytes we need to print and then we copy `ebx` to `edx`.

This is the same as

```
count=p-s;
```

Was in the original C program. However, we can also think of it as

```
`edx=ebx-eax;'
```

Registers are still just special variables that can be used for anything we like. Unlike C, there is no difference between an “int” and an “int\* “. The C compiler will complain at you if you try to reuse the same variable as a different type. Sometimes you can convince it to let you do it anyway with typecasting.

But Assembly is the Minecraft Creative Mode of programming. There are no limitations other than those imposed on you by your operating system and hardware. To be honest

To drive home the point of why Assembly language is powerful but also dangerous, allow me to share the ancient wisdom of a programming master.

“C makes it easy to shoot yourself in the foot; C++ makes it harder, but when you do it blows your whole leg off” - Bjarne Stroustrup [Quotes](#)

And from my experience in Assembly language, I can tell you, Assembly programming is like searching for documentation on how to shoot yourself in the foot. Then Google censors your search results because it thinks you are going to self-harm. Suddenly, the government labels you a terrorist and then drops a bomb on your house until your legs are blown off, and so are everyone else’s in your neighborhood.

In this chapter, I spent a lot more time explaining the behavior of the `putstring` function than I did in the DOS edition of this book. This function is foundational to the other functions in this book because whenever we need to print a number or an error message, the only real way to do it is to first define a string in the memory and print it using `putstring`. Without this function I would have to specifically count the length by hand of each function and load the registers again each time.

The purpose of creating a function in Assembly, or any other language, is to have a piece of reusable code that can be used at your convenience. It saves space when programming and also allows errors to be fixed because if the code that handles input or output is isolated in a function, then fixing that function fixes the rest of the program automatically.

Now that I have taught you the magical properties of the `putstring` function, I will use the

next chapter to show you how I print integers in multiple bases!

## Chapter 5: intstr and putint

In the last chapter, we used a function named putstring to print strings of text. This is nice but in order to do useful things in assembly, we need to be able to print numbers.

If you are coming from the C language, you might remember the printf function and expect something similar. However, printf is not as good as what I will be showing you in both C and assembly. Printf can only print integers in base ten (decimal), eight (octal), and sixteen (hexadecimal)

Instead, I will introduce two new functions that can be used along with putstring in order to print any integer in any base from two to thirty-six. The default base is two (binary) because it is my favorite but it can be changed.

First, here is the C program that shows how these functions are written and used in C. Keep in mind that putstring is the same as it was in the last chapter. Whether you use the unistd or stdlib form of output will not matter here. However, this is a Linux book so I will be using the unistd interface with POSIX system calls from now on.

### C putint

```
#include <unistd.h>

int putstring(const char *s)
{
    int count=0;          /*used to count how many bytes will be written*/
    const char *p=s;      /*pointer used to find terminating zero of string*/
    while(*p){p++;}       /*loop until zero found and immediately exit*/
    count=p-s;            /*count is the difference of pointers p and s*/
    write(1,s,count);     /*the unix system call way of writing the bytes*/
    return count;         /*return how many bytes were written*/
}

#define usl 0x100 /*usl stands for Unsigned or Universal String
Length.*/
char int_string[usl+1]; /*global string which will be used to store
string of integers. Size is usl+1 for terminating zero*/

/*radix or base for integer output. 2=binary, 8=octal, 10=decimal,
16=hexadecimal*/
int radix=2;
/*default minimum digits for printing integers*/
int int_width=1;

char *intstr(unsigned int i)      /*Chastity's supreme integer to string
conversion function*/
{
    int width=0;                  /*the width or how many digits including
prefixed zeros are printed*/
    char *s=int_string+usl;      /*a pointer starting to the place where
```

```

we will end the string with zero*/
*s=0; /*set the zero that terminates the
string in the C language*/
while(i!=0 || width<int_width) /*loop to fill the string with every
required digit plus prefixed zeros*/
{
    s--; /*decrement the pointer to go left for
correct digit placing*/
    *s=i%radix; /*get the remainder of division by the
radix or base*/
    i/=radix; /*divide the input by radix*/
    if(*s<10){*s+='0';} /*convert digits 0 to 9 to the ASCII
character for that digit*/
    else{*s=*s+'A'-10;} /*for digits higher than 9, convert to
letters starting at A*/
    width++; /*increment the width so we know when
enough digits are saved*/
}
return s; /*return this string to be used by
putstr,printf,std::cout or whatever*/
}

void putint(unsigned int i)
{
    putstring(intstr(i));
}

int main(int argc, char *argv[])
{
    int a=0;

    putstring("The putstring function can print any string!\n");
    putstring("The intstr function can convert an integer to a string!\n");
    putstring("The putint function calls intstr and putstring to print an
integer\n");

    while(a<0x10)
    {
        putint(a);
        putstring("\n");
        a++;
    }

    _exit(0);
}

```

After the putstring function was defined, I defined some global variables which control the details of where the digits for integers are stored and which number base or radix will be used to convert an integer into a string.

The width variable determines how many minimum digits are going to be printed. A width of 8 for example would mean that extra zeros would be prefixed if the number was naturally less than 8 digits. Most of the time this can be left as 1 so that only the

minimum number of required digits is used.

I will admit that the `intstr` function which uses these variables looks quite complicated to a beginner programmer. However, this difficulty does not come from the C language.

The reason that this function is complex is because it requires the human reading it to understand the nature of number systems themselves. If you don't know about bases other than ten, you probably haven't thought about it much. In fact, most people don't even take the time to analyze the base ten system of numbers they are already using. For now, I suggest learning about the binary numeral system from other sources for clarification, though I may be including more information about it in this book later about number bases and the pros and cons of them.

The last C program I showed you will print the following text when compiled and run.

The `putstring` function can print any string!

The `intstr` function can convert an integer to a string!

The `putint` function calls `intstr` and `putstring` to print an integer

```
0
1
10
11
100
101
110
111
1000
1001
1010
1011
1100
1101
1110
1111
```

Naturally, I am going to show you the assembly program that does the same exact thing as the C version above.

## **FASM putint**

format ELF executable

main:

```
mov eax,string0
call putstring
mov eax,string1
call putstring
mov eax,string2
call putstring
```

```
mov eax,0
```

```

loop0:
call putint
call putline
inc eax
cmp eax,0x10
jnz loop0

mov eax,1 ; invoke SYS_EXIT (kernel opcode 1)
mov ebx,0 ; return 0 status on exit - 'No Errors'
int 80h

string0 db 'The putstring function can print any string!',0Ah,0
string1 db 'The intstr function can convert an integer to a
string!',0Ah,0
string2 db 'The putint function calls intstr and putstring to print an
integer',0Ah,0

putstring:

push eax
push ebx
push ecx
push edx

mov ebx,eax

putstring_strlen_start:

cmp [ebx],byte 0
jz putstring_strlen_end
inc ebx
jmp putstring_strlen_start

putstring_strlen_end:
sub ebx,eax

mov edx,ebx ;number of bytes to write
mov ecx,eax ;pointer/address of string to write
mov ebx,1 ;write to the STDOUT file
mov eax,4 ;invoke SYS_WRITE (kernel opcode 4 on 32 bit systems)
int 80h ;system call to write the message

pop edx
pop ecx
pop ebx
pop eax

ret

; This is the location in memory where digits are written to by the
intstr function
; The string of bytes and settings such as the radix and width are
global variables defined below.

```

```

int_string db 32 dup '?' ;enough bytes to hold maximum size 32-bit
binary integer

int_string_end db 0 ;zero byte terminator for the integer string

radix dd 2 ;radix or base for integer output. 2=binary, 8=octal,
10=decimal, 16=hexadecimal
int_width dd 1 ;default width of integers. Extra zeros prefixed if more
than 1

;this function creates a string of the integer in eax
;it uses the above radix variable to determine base from 2 to 36
;it then loads eax with the address of the string
;this means that it can be used with the putstring function

intstr:

mov ebx,int_string_end-1 ;find address of lowest digit(just before the
newline 0Ah)
mov ecx,1

digits_start:

mov edx,0;
div dword [radix]
cmp edx,10
jb decimal_digit
jnb hexadecimal_digit

decimal_digit: ;we go here if it is only a digit 0 to 9
add edx,'0'
jmp save_digit

hexadecimal_digit:
sub edx,10
add edx,'A'

save_digit:

mov [ebx],dl
cmp eax,0
jz intstr_end
dec ebx
inc ecx
jmp digits_start

intstr_end:

prefix_zeros:
cmp ecx,[int_width]
jnb end_zeros
dec ebx
mov [ebx],byte '0'
inc ecx

```

```
jmp prefix_zeros
end_zeros:
```

mov eax,ebx ; now that the digits have been written to the string,  
display it!

```
ret
```

```
; function to print string form of whatever integer is in eax
; The radix determines which number base the string form takes.
; Anything from 2 to 36 is a valid radix
; in practice though, only bases 2,8,10,and 16 will make sense to other
programmers
; this function does not process anything by itself but calls the
combination of my other
; functions in the order I intended them to be used.
```

```
putint:
```

```
push eax
push ebx
push ecx
push edx
```

```
call intstr
```

```
call putstring
```

```
pop edx
pop ecx
pop ebx
pop eax
```

```
ret
```

```
line db 0Ah,0 ;a string containing only a newline
```

```
;the next function which pushes eax to the stack
;moves the address of the line string and prints it with putstring
;then it pops the original value of eax back from the stack before the
function returns
;this allows me to print a newline anywhere in the code without a single
register changing
```

```
putline:
push eax
mov eax,line
call putstring
pop eax
ret
```

Although this program is longer than the C version, it is doing all of the same things. This program is very simple in terms of what it does. Here is a brief breakdown of it.

It sets `eax` to the address of 3 different strings and each time calls `putstring` to print those strings until the terminating zero is found. The `putstring` function was described in chapter 4 and I promise you that you will not want to forget it because it is the top link in a chain of dependencies.

Next, `eax` is set to 0 before a loop labeled as “`loop0`”. During this loop, the `eax` register is used when calling `putint` to print the `eax` register in the default base of binary. The `putline` function is called next to print a newline character. The reason this is done is because `eax` cannot be modified inside the loop or the program would crash. Therefore I wrote a function for printing a line which changes `eax` temporarily but will leave it as it was so that the loop is still functioning correctly.

Next, `eax` is incremented with the “`inc`” instruction. This is the same as adding 1 to a variable as you saw in the `putstring` when I incremented `ebx` in the same way.

The next instruction of “`cmp eax,0x10`” compares `eax` with the number 16. It updates flags but does not modify anything else so that the next instruction of “`jnz loop0`” will operate correctly. If `eax` was not equal to 16 in the last comparison, then it will jump to the `loop0` label.

## Side rant on condition jumps

You might wonder, why is the conditional jump called “`jnz`” when it means jump if not equal? Technically you can also change it to “`jne`” and it will still assemble and run.

The way Assembly language works is that when you compare something with “`cmp`”, it subtracts the second number from the first number to get a result but it does not save that result. Therefore, only when `eax` equals 16 will the the comparison of 16 minus 16 equal zero.

I will be covering more about conditional jumps and other things in the next chapter, but for now you can consider the “`jnz`” to be the same as “`jne`” and write whichever one you like best. Similarly, you can also modify the last program to use “`jb`” which means jump if below. The opposite of “`jb`” would be “`ja`” which means jump if above.

## How the `intstr` function works

Mastery of conditional jumps is essential for the operation of the `intstr` function in this chapter that converts integers to strings. This function contains a loop which divides `eax` with “`div`” and the remainder is stored in `edx`. Depending on the value of `dl` (the lowest byte of `edx`), we need to jump to different places for different bases.

For example, if the remainder is 0 to 9, we can consider it a decimal digit. Then we have to add the ASCII value for ‘0’ which is not the same as the number zero but is instead forty eight. Then it will display as the digit that we expect it to. However, if the digit is ten or above, then we have to represent it somehow using a convenient list of recognized

symbols. Letters A to Z of the alphabet were the obvious choice when these decisions were made decades ago. Adding the ASCII value of ‘A’ or sixty-five works for making the digit display as something we can understand. This is how the hexadecimal system works, for example. The `intstr` function is my largest function so far because it has to handle all these different scenarios that can happen.

For greater understanding, I give you the following table of convenient characters for integers.

## Digit Character Table

| <b>bin</b> | <b>hex</b> | <b>dec</b> | <b>character</b> |
|------------|------------|------------|------------------|
| 00110000   | 30         | 048        | 0                |
| 00110001   | 31         | 049        | 1                |
| 00110010   | 32         | 050        | 2                |
| 00110011   | 33         | 051        | 3                |
| 00110100   | 34         | 052        | 4                |
| 00110101   | 35         | 053        | 5                |
| 00110110   | 36         | 054        | 6                |
| 00110111   | 37         | 055        | 7                |
| 00111000   | 38         | 056        | 8                |
| 00111001   | 39         | 057        | 9                |
| 01000001   | 41         | 065        | A                |
| 01000010   | 42         | 066        | B                |
| 01000011   | 43         | 067        | C                |
| 01000100   | 44         | 068        | D                |
| 01000101   | 45         | 069        | E                |
| 01000110   | 46         | 070        | F                |
| 01000111   | 47         | 071        | G                |
| 01001000   | 48         | 072        | H                |
| 01001001   | 49         | 073        | I                |
| 01001010   | 4A         | 074        | J                |
| 01001011   | 4B         | 075        | K                |
| 01001100   | 4C         | 076        | L                |
| 01001101   | 4D         | 077        | M                |
| 01001110   | 4E         | 078        | N                |
| 01001111   | 4F         | 079        | O                |
| 01010000   | 50         | 080        | P                |
| 01010001   | 51         | 081        | Q                |
| 01010010   | 52         | 082        | R                |
| 01010011   | 53         | 083        | S                |
| 01010100   | 54         | 084        | T                |
| 01010101   | 55         | 085        | U                |
| 01010110   | 56         | 086        | V                |
| 01010111   | 57         | 087        | W                |

| <b>bin</b> | <b>hex</b> | <b>dec</b> | <b>character</b> |
|------------|------------|------------|------------------|
| 01011000   | 58         | 088        | X                |
| 01011001   | 59         | 089        | Y                |
| 01011010   | 5A         | 090        | Z                |

If you are feeling overwhelmed at the point, I don't blame you. I am hitting you with more information than most programming books attempt to teach. So far the goal has only been to get you started with functions you can use to print strings and numbers to the console. Without doing this foundational step, this book would be like a book on the C Programming Language without including how to use printf.

In Assembly, there is no printf. The programmer is responsible for printing the information by setting up registers, memory, and making system calls. This takes a lot of math and is not for the average human who doesn't like low level details.

If you're still with me, I have some good news for you. The next chapter will be a low easier! I will explaining every single assembly instruction that most programs need and each instruction will have a short example that can be copied and pasted directly from the digital versions of this book or typed by hand if you have a paperback edition.

However, those programs will use the putstring and putint functions a lot, and at some point you will thank me for spending 5 chapters on how to assemble programs that allow managing output to your Linux terminal.

## Chapter 6: Chastity's Intel Assembly Reference

I use a very small subset of the Intel 8086 family instruction set. This is both because I want to limit it to my small memory (my brain memory, not computer memory). If you are like me and have a tendency to forget things, then Assembly language is actually very good because there is not a lot to remember when compared to bigger high level languages like C++ or Java. And if you do forget, this chapter will function as the definitive guide for performing math using Assembly language for Linux

**Important note. All program listings in this chapter assume that you also included the `putstring`, `intstr`, and `putint` functions as shown in chapters 2 and 3. This can be done by including external files or just copy pasting their text after the system exit call from `eax=1` and interrupt `80h`. This keeps the examples brief by not repeating functions you should already have from chapters 4 and 5.**

At the end of each example, you will see a line that reads  
`include 'chastelib32.asm'`

The [chastelib32.asm](#) file is a file containing the functions listed in chapters 4 and 5 but also includes a lot more commentary than I have included in those chapters. For your convenience, you can download it from my repository and/or view the entire source code of it by looking at the end of chapter 10. My functions provide a useful base which to can use to add, delete, or even improve upon mine. Not only do the examples in this reference chapter use them to show output, but so will the core four programs: `chastack`, `chastext`, `chastecmp`, and `chastehex`.

**But** you don't need the `chastelib32.asm` either because you can put the required functions in a file of your choice. As long as you ***included*** them in your source after the exit call, these examples will all work.

### **mov**

The `mov` instruction copies a number from one location to another. In the FASM and NASM assemblers, the instruction always takes the form

```
mov destination, source
```

Think of it as “`destination=source`” as you would write in C. For example, in the following program which prints the number 8, we see that most of the required data is set up with `mov` instructions.

```
format ELF executable
```

```
main:
```

```
mov dword [radix],10  
mov dword [int_width],1
```

```

mov eax, 3
mov ebx, 5
add eax, ebx

call putint
call putline

mov eax, 1
mov ebx, 0
int 80h

```

```
include 'chastelib32.asm'
```

That program also contains the call, int, and add instructions to make a program that does something useful. However, mov instructions take up the largest part of any program. Whether you are filling a register with a number, another register, or a memory location, the mov instruction is the way to do it.

## add

Next to mov, you will see that add is going to be your friend in Assembly a lot. In the previous example, we saw that 3 and 5 were added to make 8. Just like mov, add follows the same rules.

```
add destination, source
```

- Destination is left of Source and separated by a comma
- Destination and Source can be registers and memory locations
- But Destination and Source cannot both be memory locations
- Source can also be a regular integer value

Most instructions that take two arguments follow these same rules. Once you have mastered mov and add, you can handle almost anything in a program because you know the basic rules.

There is also the “inc” instruction which takes only one item and adds 1 to it. This is just a shorter way of saying “add Destination,1”

## sub

As its name implies, sub will subtract the Source from the Destination.

```
sub destination, source
```

Since it follows the same rules as mov and add (starting to see a pattern yet?), subtraction is just as easy as addition.

Just as “add” has “inc”, “sub” has the “dec” instruction which subtracts 1. Adding or subtracting 1 are probably the most common thing ever done while programming in any language.

Just as a review of the mov,add,sub instructions, here is a small program to show their effect.

```
format ELF executable
```

```
main:
```

```
mov dword [radix],10
mov dword [int_width],1
```

```
mov eax,8
call putint
call putline
add eax,eax
call putint
call putline
sub eax,4
call putint
call putline
```

```
mov eax,1
mov ebx,0
int 80h
```

```
include 'chastelib32.asm'
```

That program will output the following.

```
8
16
12
```

This is because we set eax to 8, then we added eax to itself, and finally we subtracted 4 from eax. Once you think about how easy this is, read on to see how multiplication and division work.

## mul

The mul instruction is slightly different than The previous instructions. It takes only one operand which must be either a register or memory location. It multiplies eax by the value of this operand. If the value is too large to fit within the eax register, it puts the higher bits into edx.

## div

The div instruction divides eax by the operand you give it (the divisor). However, division is a tricky operation because not every number divides evenly into another. It is also more complicated by the fact that the edx register is assumed to be the upper half of the bits in the dividend while eax is the lower bits of the dividend.

I know it sounds complicated but it is easier than I can explain. I can illustrate this with a small program that multiplies and divides!

format ELF executable

main:

```
mov dword [radix],10
mov dword [int_width],1
```

```
mov eax,12
call putint
call putline
mov ebx,5
mul ebx
call putint
call putline
mov ebx,8
mov edx,0
div ebx
call putint
call putline
mov eax,edx
call putint
call putline
```

```
mov eax,1
mov ebx,0
int 80h
```

include 'chastelib32.asm'

The output of that program is this:

```
12
60
7
4
```

This is because 12 was multiplied by 5 to get 60. Then we attempted to divide 60 by 8. It goes in only 7 times (which equals 56). This means the remainder is 4, which is stored in the edx register after the division.

You may also notice in the source above that I set edx to zero before the div instruction. If this is not done, the edx might have mistakenly had another number and been interpreted as part of the dividend.

I also think some terminology about division is helpful here.

- Dividend: The number we are dividing from.
- Divisor: The number we are dividing the dividend by. How many times does this number subtract from the dividend?
- Quotient: The result of the division.
- Remainder: What is left over if the divisor could not divide perfectly into the dividend.

As much as I love math, I find some of these terms confusing when I try to explain them

in English. Let's face it, I am better at Assembly Language and the C Programming Language than I am with English, but it looks like you're stuck with me because normal people are not autistic enough to care!

Division is repeated subtraction, just as multiplication is repeated addition. Most of the time, modern CPU hardware can multiply and divide faster by the `mul` and `div` instructions than if you have manually made loops of repeating addition or subtraction, but the result would have been the same.

6502 CPUs (used for the Nintendo Entertainment System) did not have multiplication and division instructions, and so programmers really did use loops of adding and subtracting to get the results needed. On Intel CPUs, `mul` and `div` are available, and I suggest using them, but please don't forget what these operations really are.

For a more in depth explanation of the `mul` and `div` instructions, I will include those written by Tomasz Grysztar (creator of the FASM assembler) in the official "flat assembler 1.73 Programmer's Manual"

---

*mul performs an unsigned multiplication of the operand and the accumulator. If the operand is a byte, the processor multiplies it by the contents of AL and returns the 16-bit result to AH and AL. If the operand is a word, the processor multiplies it by the contents of AX and returns the 32-bit result to DX and AX.*

*div performs an unsigned division of the accumulator by the operand. The dividend (the accumulator) is twice the size of the divisor (the operand), the quotient and remainder have the same size as the divisor. If divisor is byte, the dividend is taken from AX register, the quotient is stored in AL and the remainder is stored in AH. If divisor is word, the upper half of dividend is taken from DX, the lower half of dividend is taken from AX, the quotient is stored in AX and the remainder is stored in DX.*

---

Perhaps you can see that Assembly language is nothing more than a fancy calculator, except better. This is because there is no question which order the operations take place in. There is no need for mnemonics like "Please excuse my dear Aunt Sally" to remind us "Parentheses, Exponents, Multiplication and Division (from left to right), and Addition and Subtraction".

There are still two more instructions before we can construct loops in Assembly programs.

## **cmp**

The `cmp` instruction compares two operands but does not do any math with them. They remain unchanged but modify flags in the processor that allow us to jump based on certain conditions.

## jmp

The jmp instruction jumps to another location regardless of any conditions. It has a family of other jump instructions that jump only if certain conditions are true. In fact many of them have multiple names for the same operation. For example je and jz both jump if the two numbers compared would be zero if they were subtracted. This would only be true if they are the same.

Here is a small chart or table for the conditional jumps I use in my programs.

### Conditional Jumps Table

| Instruction | Meaning           |
|-------------|-------------------|
| je/jz       | jump if equal     |
| ja          | jump if above     |
| jb          | jump if below     |
| jne/jnz     | jump if not equal |
| jna         | jump if not above |
| jnb         | jump if not below |

Aside from those main 6 conditional jumps that I have memorized, there also exists jumps for signed/negative numbers which I have not not talked about yet in this book yet. A small table is below.

### Signed Number Jumps Table

| Instruction | Meaning             |
|-------------|---------------------|
| js          | jump if signed      |
| jns         | jump if not signed  |
| jg          | jump if greater     |
| jl          | jump if less        |
| jng         | jump if not greater |
| jnl         | jump if not less    |

Personally I don't agree with the way negative numbers are represented in computers but I know that understanding the context of signed vs unsigned is important for more complex programs. Once again, I recommend the FASM programmers manual for details that I have excluded for the purpose of keeping this book short.

However, I will be using the js and jns instructions in some programs in this book. It is important to understand that the Linux kernel returns negative numbers in the eax registers when there is an error. The most common case is when you try to open a filename that does not exist or that you don't have permission to open even if it does exist. As an example, you would compare eax with 0 to update the flags and then jump according to the result.

```
cmp eax,0 ;compare eax with zero to update the flags
js error_yes
```

```
jns error_no
```

The Intel processors also have the “neg” instruction for converting between positive and negative. But for the most part, my programs do not use negative numbers and the first table for unsigned integer conditional jumps will be all you need.

The following program can print a message telling you whether `eax` is less than , equal to, or more than `ebx`. Upon this foundation all the conditional jumps in my programs and functions are based.

```
format ELF executable
main:
```

```
mov dword [radix],10
mov dword [int_width],1
```

```
mov eax,5
mov ebx,8
cmp eax,ebx
jb less
je same
ja more
```

```
less:
mov eax,string_less
jmp the_end
```

```
same:
mov eax,string_same
jmp the_end
```

```
more:
mov eax,string_more
jmp the_end
```

```
the_end:
call putstring
```

```
mov eax,1
mov ebx,0
int 80h
```

```
string_less db 'eax is less than ebx',0Ah,0
string_same db 'eax is the same as ebx',0Ah,0
string_more db 'eax is more than ebx',0Ah,0
```

```
include 'chastelib32.asm'
```

Personally, I think that the Assembly system of conditional jumps makes a lot of sense. Other programming languages such as BASIC and C have “goto” statements that work like this. For example, `if(eax<ebx){goto less;}`.

Modern programming languages tend to discourage the use of `goto` or not allow it at all. However, these languages still use jumps I have described in this section because it is required by the hardware. Both “if” and “while” statements are written by using the conditional jump statements most relevant to what you are trying to do.

The only thing I have found difficult about jumps in assembly is remembering which acronym means which condition. However, since I created the chart in this chapter, now I can refer to it, and you can too! As long as I keep these six main types of conditions in my head and am working with unsigned numbers, I can write almost any assembly program from scratch.

## **push/pop**

The push and pop instructions are something you have already seen in my code. They operate on what is called the “stack”. Basically, when you push something, it is like pushing a box of cereal onto a shelf at Walmart. The last item pushed is at the front and will be the first item a customer sees. This is what is called a Last In First Out.

Not only is the stack useful for saving the value of registers temporarily as I do, but without it, it would not be possible to have callable functions. When you call a function with “call”, it is the same as a “jmp” to that location except that it pushes the address where the program was before the call. The “ret” instruction returns to the location that called the function and then proceeds to instructions after it.

The sp register, as I mentioned in chapter 1, is the stack pointer. Every time you push a value, it stores it at the address the stack pointer is pointing to and then subtracts the size of the native dword size. For example, this is always 16 bits in the context of DOS programming for 16 bit .com files. This means that you can use it with the other registers to save their value for later.

In the next chapter, I will show a useful example of the push and pop instructions and explain a little bit more about this.

## **Take it slow**

I know I hit you with a lot of information in this chapter, but trust me, I am intentionally leaving out a lot because I don’t want this book to be the size of the Intel® 64 and IA-32 Architectures Software Developer Manuals.

<https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html>

There are hundreds of instructions for Intel machines and yet if you combine the instructions I have described in this chapter with the “call”, “int”, and “ret” instructions required for calling functions for input and output, you will see that it is possible to write almost any program I want with these instructions.

I am sharing what I have learned from reading the Intel Manuals and the API references available for DOS so that you don’t have to spend as much time figuring these things out as I did. What I can tell you, though, is that the result was worth it because I have been able to write programs to accomplish tasks faster than my C programs could. At the same time, the Assembly versions took longer to write than the C versions did. This is the price

I must pay to have high performing code.

Also, there are some bitwise instructions by the names of AND,OR,XOR,NOT,SHL,SHR that are sometimes useful for making programs faster and smaller. However, these only make sense in the context of the Binary Numeral System and I suspect that the average reader of this book does not have the 25 years of experience in Binary math that I do.

I will be explaining more about these operations in a later chapter because they help a lot when trying to optimize programs for size and speed. However they can make programming LOOK complicated and scare away potentially great new programmers who are just trying to learn to apply the 4 regular arithmetic operations of addition, subtraction, multiplication, and division which apply to all number bases.

## Chapter 7: Open and Close System Calls

In this chapter, I will be showing you two programs. One of them writes a message to a text file. The other will read a text file to find out what is inside it.

### FASM Create a New File

```
format ELF executable
main:
```

```
mov dword [radix],10
mov dword [int_width],1
```

```
;Linux system call to open a file
```

```
mov eax,5           ;invoke SYS_OPEN (kernel opcode 5)
mov ebx,filename    ;path to filename should be in eax before this
function was called
mov ecx,1010        ;flags (in base 8) for open mode 0101 = 0100
(O_CREAT) + 1 (O_WRONLY)
mov edx,6440        ;permissions of the new file
int 80h             ;call the kernel
```

```
mov [filedesc],eax
call putint
mov eax,string1
call putstring
```

```
mov eax,string0      ;mov to eax the address of string to write
mov ecx,eax          ;pointer/address of string to write
call strlen          ;get length of the string in eax
mov edx,eax          ;copy length from eax to edx
mov ebx,[filedesc]   ;write to the file descriptor
mov eax,4            ;invoke SYS_WRITE (kernel opcode 4 on 32 bit systems)
int 80h              ;system call to write the message
```

```
mov ebx,[filedesc]   ;file number to close
mov eax,6            ;invoke SYS_CLOSE (kernel opcode 6)
int 80h              ;call the kernel
```

```
mov eax,1
mov ebx,0
int 80h
```

```
filename db 'newfile.txt',0
filedesc dd 0 ; file descriptor
```

```
string0 db 'This string writes to the file.',0Ah,0
string1 db '=file descriptor',0Ah,0
```

```
;The strlen function gets the length of string in eax and returns it in
eax
```

```
;This is the same algorithm used in my putstring function

strlen:

push ebx
mov ebx,eax ; copy eax to ebx. ebx will be used as index to the string

strlen_start: ; this loop finds the length of the string

cmp [ebx],byte 0 ; compare byte at address ebx with 0
jz strlen_end ; if comparison was zero, jump to loop end
inc ebx
jmp strlen_start

strlen_end:
sub ebx,eax ;subtract start pointer from current pointer to get length
of string
mov eax,ebx ;copy the string length back to eax
pop ebx

ret

include 'chastelib32.asm'
```

## Syscall Reference

This link will help you understand the numbers you need to include for the open and close system calls.

<https://www.chromium.org/chromium-os/developer-library/reference/linux-constants/syscalls/#x86-32-bit>

Additionally, on most Linux systems you can see the manual pages for open and close with these commands.

```
man 2 open
man 2 close
```

The manual pages, while helpful for C programmers are not very helpful for Assembly programmers. To find out which numbers are associated with which constants, you need to file the file which contains them.

Use this command find the filename containing file control constants.

```
find /usr/include -name "fcntl-linux.h"
```

The file might be in slightly different locations on different systems, but it will contain the following values plus more that are not applicable to this chapter.

| Constant | Octal Value |
|----------|-------------|
| O_RDONLY | 00          |
| O_WRONLY | 01          |
| O_RDWR   | 02          |
| O_CREAT  | 0100        |

The next example is a bit more complicated than the last. It includes more error checking because when you are trying to open a file, there is a chance that the file doesn't exist.

This next program will attempt to open the "newfile.txt" that we created with the last one. This means that if you renamed or deleted it, then it will print an error message because it couldn't open a file that doesn't exist.

## FASM Open Existing File

```
format ELF executable
main:
```

```
mov dword [radix],10
mov dword [int_width],1
```

```
;Linux system call to open an existing file
```

```
mov eax,5           ;invoke SYS_OPEN (kernel opcode 5)
mov ebx,filename    ;path to filename should be in eax before this
function was called
mov ecx,0           ;flags for open mode 0=O_RDONLY
mov edx,0           ;permissions irrelevant because we are opening a file
that exists
int 80h             ;call the kernel
```

```
push eax            ;save eax on the stack
mov [filedesc],eax ;also copy file descriptor to memory
call putint
mov eax,string1
call putstring
pop eax             ;restore eax from the stack
```

```
cmp eax,0 ;compare eax with zero to update flags
jns cat      ;if eax is not negative/signed there was no error
```

```
;Otherwise, if it was signed, then this code will display an error
message.
```

```
mov eax,file_error
call putstring
```

```
jmp the_end ;end the program because we failed at opening the file
```

```
cat:
```

```
mov edx,1           ;number of bytes to read
mov ecx,tempbyte     ;address to store the bytes
mov ebx,[filedesc]   ;move the opened file descriptor into EBX
mov eax,3           ;invoke SYS_READ (kernel opcode 3 on 32 bit
systems)
int 80h             ;call the kernel
```

```
cmp eax,0
```

```
jnz print_byte ;if more than zero bytes read, proceed to display
jmp the_end ;otherwise, end the program
```

;this point is reached if file was read from successfully

print\_byte:

```
mov edx,1          ;number of bytes to write
mov ecx,tempbyte   ;address of the bytes
mov ebx,1          ;write to standard out instead of the file
descriptor we read from
mov eax,4          ;invoke SYS_WRITE (kernel opcode 4 on 32 bit
systems)
int 80h           ;call the kernel
```

jmp cat

cat\_end:

```
mov ebx,[filedesc] ;file number to close
mov eax,6          ;invoke SYS_CLOSE (kernel opcode 6)
int 80h           ;call the kernel
```

the\_end:

```
mov eax,1
mov ebx,0
int 80h
```

```
tempbyte db 0
filename db 'newfile.txt',0
filedesc dd 0 ; file descriptor
file_error db 'error while opening file',0Ah,0
```

```
string1 db '=file descriptor',0Ah,0
```

```
include 'chastelib32.asm'
```

Another thing you may have noticed if you assembled and ran these two programs in this chapter is that they both print the number of the file descriptor. If you see it print a huge number over 4 billion, then it is probably a negative number that indicates an error has occurred. In the program for opening an existing file, I used the “jns” instruction which means “Jump if Not Signed”.

I haven’t described negative numbers in this book yet because it can confuse newcomers. However, these negative numbers are used by the Linux operating system to describe errors that can happen in some system calls.

Specifically in the case of the “open” call, a file might not exist or we might not have permission to open it because it belongs to a different user. Either way, we can’t open it.

This chapter may have only introduced two new programs, but it also introduced 3 system calls. I already described the write call when I used it in creating the putstring function from chapter 4. This gives us the four fundamental file functions of the Linux

## Kernel

| <b>Call Name</b> | <b>Call Number</b> |
|------------------|--------------------|
| read             | 3                  |
| write            | 4                  |
| open             | 5                  |
| close            | 6                  |

When I introduced the write call in chapter 4, I was writing to standard output (file descriptor 1). However, from the perspective of the Linux kernel, stdout is just a special case of a file. Standard input is file descriptor 0 and it can be used to get information from the user with the keyboard.

However, when getting user input, error checking must be carefully done, even more so than when reading from another file. This is because humans are unpredictable and you have to plan for every possible input they may produce.

Getting user input will be explained in the next chapter. For now, just look over the examples in this chapter and think about the fact that all programs are just made of system calls for reading and writing files. You can open files, write to them, read from them, and close them.

## Chapter 8: User Input

The first seven chapters have been about teaching the basics of Assembly and getting output of strings and numbers to the screen. All those steps were required for learning Assembly. However, at some point, when you have a program that is meant to do something, you need to have a way for other people, especially those who are not programmers, to be able to give input to direct what the program does.

There are two main ways of doing this in a console program. The first way is have the program ask for the user to type something from the keyboard and then wait until they write something and press enter. The next program will achieve this. Copy this and try it out and then I will explain after the code how it works.

### FASM Keyboard Input

```
format ELF executable
```

```
main:
```

```
mov dword [radix],10
mov dword [int_width],1
```

```
loop_input:
```

```
mov eax,string0
call putstring
```

```
call getstring
```

```
mov esi,eax      ;mov the string address in eax to esi
mov edi,string3  ;mov the "exit" string address to edi
call strcmp      ;call the function to compare the strings and return eax
cmp eax,0        ;if eax is 0, the strings are the same
jz the_end       ;go to the_end if the user typed "exit"
```

```
mov eax,string1
call putstring
```

```
mov eax,buf
call putstring
call putline
```

```
mov eax,string2
call putstring
```

```
mov eax,[count]
dec eax
call putint
call putline
```

```

jmp loop_input

the_end:
mov eax,1
mov ebx,0
int 80h

string0 db 'Enter a string from the keyboard: ',0
string1 db 'string: ',0
string2 db 'length: ',0
string3 db 'exit',0

buf db 0x100 dup '?'
count dd 0
last_char db 0

;summary
;the getstring function is the reverse function of putstring
;instead of printing a string to standard output
;it reads a string from standard input (AKA the keyboard)

;details
;the getstring function is designed to get a string of text
;which is terminated by whitespace or any non printable character
;the idea is that multiple strings can be passed on one line
;separated by spaces, similar to command line arguments
;this function was written for the specific purpose of converting any of
;my programs that used command line arguments to read from stdin instead

getstring:

mov [count],0 ;set count of characters read during this function to zero
mov edx,1     ;number of bytes to read
mov ecx,buf   ;address to store the bytes

getstring_chars:

mov ebx,0     ;read from stdin
mov eax,3     ;invoke SYS_READ (kernel opcode 3)
int 80h       ;call the kernel

cmp eax,1     ;was 1 character read?
jz getstring_read_yes ;if yes, process this character and add to string
ret           ;otherwise exit the function now

getstring_read_yes:
add [count],eax ;add how many characters we have read
mov al,[ecx]    ;mov last character read into al register

;check if this character is in the proper range to be part of the string

cmp al,0x21    ;compare with 0x21 (!=exclamation)
jb getstring_end ;jump if below to getstring_end label
cmp al,0x7E    ;compare with 0x7E (tilde)

```

```

ja getstring_end ;jump if above to getstring_end label

;if neither jump happened, keep the character and

inc ecx          ;increment address where next byte is read from
jmp getstring_chars ;jump back to start of loop and keep reading

getstring_end:

mov [last_char],al ;save the last character read
mov byte[ecx],0 ;terminate this string with a zero

mov eax,buf ;mov the buffer address to eax for returning the string

ret

```

```

;Short Description of strcmp:
;strcmp compares the string at esi to the one at edi
;eax returns 0 if the strings are the same and non zero if different
;the algorithm is simple but I will explain it for those who are
confused

```

```

;Long Description of strcmp:
;eax is initialized to zero
;a byte from each string is loaded into the al and bl registers
;the bytes are compared. if they are different, then we jump to the end
;However, if they are the same, then we check if one of them is zero
;for this purpose it doesn't matter whether we compare al or bl with
zero
;because it is known that they are the same if the jnz did not take
place
;if it is zero, this also jumps to the end of the function
;If neither jump took place, then we jump to the start of the loop
;but when the function finally ends bl will be subtracted from al
;this ensures that the function returns zero if the final characters are
the same
;ebx,esi,and edi are preserved but eax is the return value
;also, the sub instruction at the end of the function also updates the
flags
;so you can "jz" or "jnz" to a label after calling this function based
on results
;This makes strcmp as useful for strings as the Intel "cmp" instruction
is for integers

```

```

strcmp:

```

```

push ebx
push esi
push edi

```

```

mov eax,0

```

```

strcmp_start:

```

```

;read a byte from each string
mov al,[edi]
mov bl,[esi]
cmp al,bl
jnz strcmp_end

cmp al,0
jz strcmp_end

inc edi
inc esi

jmp strcmp_start

strcmp_end:
sub al,bl

pop edi
pop esi
pop ebx

ret

include 'chastelib32.asm'

```

The `getstring` function uses a `read` system call to read from file descriptor 0 which represents standard input or the keyboard. It reads one character each time with a loop and starts at an address labeled “buf” which was declared as a global variable of 256 bytes which were initialized with question marks. I also defined a variable named `count` which was used to automatically count how many bytes were read.

```

buf db 0x100 dup '?'
count dd 0

```

But I feel that the part of this function that needs the most explaining is this section:

```

cmp al,0x20      ;compare with 0x20 (space)
jb getstring_end ;jump if below to getstring_end label
cmp al,0x7E      ;compare with 0x7E (tilde)
ja getstring_end ;jump if above to getstring_end label

```

Because this range of characters from space to tilde is what I have identified as the acceptable range of characters. There is no standard way that makes sense for all strings. For example, someone may want to make a `getstring` function that only accepts capital letters or that only accepts numbers 0 to 9. I can’t say that there is one way that is the best.

The program listed above will keep running the loop until the user types “exit” as the string. Each time after it gets the string, it compares the what the user entered to the “exit” string. If the `strcmp` function returns 0, it means the two strings are the same.

This particular variant of `strcmp` is based off of the C function of the same name. You may also remember that I wrote a `strlen` function for the first example in chapter 7 when I

had a string that I wanted to write to a new file.

I believe that using conventional names of C functions is a good idea because C programmers who read my books will already be familiar with that function and what it does in the C programming language.

In any case, “exit” was the perfect name for a command to “exit” the program. It is also how you log out of a Linux terminal and is the official name for the system call that exits every program in this book!

Although using the keyboard for input during a running program is a great interactive way of doing things, there is one way that I enjoy even more. The next program is one that I wrote long before I started writing this book and has been referred to as “chastearg” on my blog and the Flat Assembler Forum. It prints the command line arguments when you add them after the name of the program.

## FASM Command Line Arguments

```
format ELF executable
entry main
```

```
include 'chastelib32.asm'
```

```
main:
```

```
pop eax                ;pop the number of arguments from the stack
mov [argc],eax         ;save the argument count for later
```

```
pop eax                ;pop argument 0 (name of the program)
dec [argc]             ;subtract 1 from argument count
```

```
putarg:
```

```
cmp [argc],0          ;check for remaining arguments
jz putarg_end          ;if none, end the loop and stop printing
pop eax                ;pop the next argument off the stack
call putstring         ;print the string and a new line
call putline           ;print the string and a new line
dec [argc]             ;subtract 1 from argument count
jmp putarg             ;jump to the beginning of the loop
```

```
putarg_end:
```

```
mov eax, 1             ; invoke SYS_EXIT (kernel opcode 1)
mov ebx, 0             ; return 0 status on exit - 'No Errors'
int 0x80
```

```
argc dd 0
```

## What is a Command Line Argument?

People who come from a Windows environment may not even know what a command line argument is because they are used to pointing and clicking with a mouse. You can't enter an argument this way. For clarification on this topic, here is some terminal text to clarify what arguments are.

```
fasm main.asm
flat assembler  version 1.73.30  (16384 kilobytes memory)
2 passes, 481 bytes.
chmod +x main
./main this program has command line arguments
this
program
has
command
line
arguments
```

When we run fasm and give it the name of the Assembly file we want to assemble, the file is an argument or an option we provide to it. In the above example, main.asm is the file I provide to fasm as an argument.

After the file is assembled, I run the chmod command with the arguments “+x” and “main” which adds the execution permission to the main executable that was just created.

Finally, running “./main” followed by more words on the same line causes Linux to interpret them as arguments. They are pushed onto the stack.

When a program begins on Linux, you can access the number of how many arguments were passed to the program by getting the first number you pop off the stack. In the chastearg program, there is a loop that keeps track of how many arguments are left. While there are some remaining, it keeps popping them into the eax register and calling putstring until there are none left.

## Arguments vs Keyboard Input

The primary difference between input from the keyboard during a program and passing arguments is that the arguments do not stop the execution of a program and wait for anything. If you have an install script which is meant to compile and install a large program, it is better not to pause it for any reason unless an error happens. Arguments are best in this case so that someone can pass information to it that they want the program to know.

Keyboard input does have a benefit though. For example, suppose that you ask the user to input a number and then they accidentally input a string that is not recognizable as a number. With keyboard input, you can tell them they made a mistake and ask them to try again. With arguments, you cannot edit them during the program because they are only pushed at the start when the program is run from the terminal.

Only you can decide which of these methods your program needs, but I hope that my explanation and my strcmp function is helpful for you when you try to write a program that needs input to do different things conditionally.

Later in this book, I will present a calculator written in Assembly language that builds from this chapter's keyboard input loop. However, we are not ready for that until I teach you how to separate regular strings from numbers. That will be the subject of the next chapter and I can promise you it is simultaneously the hardest task but also the most useful feature you will need for writing any program that has to read numbers.

## Chapter 9: Numbers and Strings

One of the things about Assembly language, and programming in general, is that everything is really a number at the lowest level. Strings are not some magical thing that is different from a number. The very first program in this book declared a “Hello World!” string which was printed with the Linux Write system call.

Here is the program again so that I can explain more of what is happening behind the scenes when assembling source files with FASM.

```
format ELF executable
```

```
mov eax,4    ; invoke SYS_WRITE (kernel opcode 4 on 32 bit systems)
mov ebx,1    ; write to the STDOUT file
mov ecx,msg  ; pointer/address of string to write
mov edx,13   ; number of bytes to write
int 80h

mov eax,1    ; function SYS_EXIT (kernel opcode 1 on 32 bit systems)
mov ebx,0    ; return 0 status on exit - 'No Errors'
int 80h
```

```
msg db 'Hello World! ',0Ah,0
```

Specifically, the line here is what we need to understand:

```
msg db 'Hello World! ',0Ah,0
```

Here we see that “msg” is the label used for this data location. A colon (:) is optional and not required for data declaration labels.

The “db” keyword says that we are going to define 1 or more bytes to be sent to the assembled output file.

The message was written in quotes as a convenience that FASM allows. However, these characters translate directly to 8-bit bytes. You can replace the line with the following equivalent and the program will print the same message because the data is the same thing.

```
msg db
0x48,0x65,0x6C,0x6C,0x6F,0x20,0x57,0x6F,0x72,0x6C,0x64,0x21,0x0A,0x00
```

What you see above are Hexadecimal constants that begin with “0x” to tell the assembler that we are about to defined numbers as hexadecimal (base sixteen) rather than decimal (base ten). You can also define constants in hex the same way in the C programming language.

The data declaration statements for integers in FASM are the following.

| Constant | Size | datatype name |
|----------|------|---------------|
| db       | 1    | byte          |
| dw       | 2    | word          |
| dd       | 4    | dword         |

| Constant | Size | datatype name |
|----------|------|---------------|
| dq       | 8    | qword         |

Carefully defining the correct size of data is important for Assembly. Otherwise you will not be able to load the data into a register. For example:

You can load the **al** and **ah** registers with something declared with a **db** statement.

You can load the **ax** register with something declared with a **dw** statement.

You can load the **eax** register with something declared with a **dd** statement.

You can load the **rax** register with something declared with a **dq** statement. However, this would work only if using 64-bit programming mode (this will be a subject of a future book).

## Why is this important?

You might see the importance of this, but if you are still clueless as to why we need to know the size or type of integers, you have to consider that the wrong type of data will not work at all for the system calls. At best, the program will do something unexpected, and at worst, it will refuse to assemble at all.

But the main reason I brought this up is because when we get input from a user, as I showed in the last chapter, we need to know that a string is a bunch of numbers.

The programmer can define data for the program in the source code, but the user expects to type in the numbers from the keyboard. If you go back to the [Digit Character Table](#) in chapter 5, you can see which character from the keyboard translate to actual byte values. This information will provide needed context both for the `intstr` function that was described in chapter 5, but also for the reverse function I am going to give you next!

I present to you the “`strint`” function which will be required for the next program!

## strint function

```
;this function converts a string pointed to by eax into an integer
;returned in eax instead
;it is a little complicated because it has to account for whether the
;character in
;a string is a decimal digit 0 to 9, or an alphabet character for bases
higher than ten
;it also checks for both uppercase and lowercase letters for bases 11 to
36
;finally, it checks if that letter makes sense for the base.
;For example, G to Z cannot be used in hexadecimal, only A to F can
;The purpose of writing this function was to be able to accept user
input as integers
;This function is improved with error checking and uses the new
strint_error variable
```

;The program can check this value after the call and see how many errors happened.

strint\_error db 0 ;declare a byte variable that keeps track of errors

strint:

mov ebx,eax ;copy string address from eax to ebx because eax will be replaced soon!

mov eax,0

mov [strint\_error],0 ;set errors to 0 at the start of this function

read\_strint:

mov ecx,0 ; zero ecx so only lower 8 bits are used

mov cl,[ebx]

inc ebx

cmp cl,0 ; compare byte at address edx with 0

jz strint\_end ; if comparison was zero, this is the end of string

;if char is below '0' or above '9', it is outside the range of these and is not a digit

cmp cl,'0'

jb not\_digit

cmp cl,'9'

ja not\_digit

;but if it is a digit, then correct and process the character

is\_digit:

sub cl,'0'

jmp process\_char

not\_digit:

;it isn't a digit, but it could be an alphabet character which is a digit in a higher base

;if char is below 'A' or above 'Z', it is outside the range of these and is not capital letter

cmp cl,'A'

jb not\_upper

cmp cl,'Z'

ja not\_upper

is\_upper:

sub cl,'A'

add cl,10

jmp process\_char

not\_upper:

;if char is below 'a' or above 'z', it is outside the range of these and is not lowercase letter

cmp cl,'a'

jb not\_lower

cmp cl,'z'

```

ja not_lower

is_lower:
sub cl,'a'
add cl,10
jmp process_char

not_lower:

;if we have reached this point, result invalid and end function with
error
jmp strint_end_error

process_char:

cmp ecx,[radix] ;compare char with radix
jnb strint_end_error ;if this value is above or equal to radix, it is
too high despite being a valid digit/alpha

mov edx,0 ;zero edx because it is used in mul sometimes
mul dword [radix] ;mul eax with radix
add eax,ecx

jmp read_strint ;jump back and continue the loop if nothing has exited
it

strint_end_error: ;we jump here if there was an error with one of the
chars
inc [strint_error] ;increment error counter because char invalid

strint_end: ;we jump here when no errors happened

ret

```

The reason that strint is such a big function is because it has to account for every possible character that could be typed by a user as either standard input or as part of the command line arguments.

It moves the eax to ebx to be used as an index register. We will keep reading from the address that ebx points to and incrementing it.

After eax is moved to ebx, it is set to zero so that it can be the receiving number that we return from the function.

It then begins the loop which checks for 3 kinds of data.

- characters in the range '0' to '9' for decimal digits
- characters in the range 'A' to 'Z' for uppercase letter digits of higher bases
- characters in the range 'a' to 'z' for lowercase letter digits of higher bases

Anything that does not meet these strict ranges of characters will result in the function ending. There are two times when this will happen.

The first time is this code near the start of the "read\_strint" loop

```
cmp cl,0 ; compare byte at address edx with 0
jz strint_end ; if comparison was zero, this is the end of string
```

The second time is when ecx was converted to a number based on the value of the digits or letters but that number is invalid for the radix. For example, A to F is valid for hexadecimal but not decimal. Letters G to Z are valid for base 36 but not for hexadecimal.

```
cmp ecx,[radix] ;compare char with radix
jnb strint_end ;if this value is above or equal to radix, it is too high
despite being a valid digit/alpha
```

During each run of the “read\_strint” loop, the ecx register is set to 0 and then the lowest part of it, cl, is used to get the character at [ebx] and then perform the math to transform the valid characters into the actual value we need. This value is then added to eax after we multiply eax by the radix.

```
mov edx,0 ;zero edx because it is used in mul sometimes
mul dword [radix] ;mul eax with radix
add eax,ecx
```

This function uses a lot of conditional jumps and is bigger than the other 3 core functions of chastelib. However, it was written specifically for turning command line arguments into numbers so that I could create mathematical programs based on them.

To conclude this chapter, I will show you a small program that interprets the command line arguments as numbers and adds them all together!

## FASM Add Arguments

```
format ELF executable
entry main
```

```
include 'chastelib32.asm'
```

```
main:
```

```
mov dword[radix],10 ;I can choose the radix for integer output!
mov dword[int_width],1 ;and the width of each integer for padded zeros
```

```
pop eax ;pop the number of arguments from the stack
mov [argc],eax ;save the argument count for later
```

```
pop eax ;pop argument 0 (name of the program)
dec [argc] ;subtract 1 from argument count
```

```
mov ebp,0 ;use the ebp register as our sum
```

```
addarg:
```

```
cmp [argc],0 ;check for remaining arguments
jz addarg_end ;if none, end the loop and stop printing
pop eax ;pop the next argument off the stack
call strint
```

```

add ebp,eax          ;add the converted number to our sum in ebp
dec [argc]           ;subtract 1 from argument count
jmp addarg           ;jump to the beginning of the loop

addarg_end:

mov eax,ebp
call putint_and_line ;print the string and a new line

mov eax, 1           ; invoke SYS_EXIT (kernel opcode 1)
mov ebx, 0           ; return 0 status on exit - 'No Errors'
int 0x80

argc dd 0

```

In case you can't tell, this program is the same as the [FASM Command Line Arguments](#) program with a few additions (see what I did there?) for extracting a number from each argument and adding it to the ebp register to be used as the final sum.

Here is what it looks like when I run the program from my terminal.

```

~/git/Chastity-Code-Cookbook/work$ ./main 1
1
~/git/Chastity-Code-Cookbook/work$ ./main 1 2
3
~/git/Chastity-Code-Cookbook/work$ ./main 1 2 3
6
~/git/Chastity-Code-Cookbook/work$ ./main 1 2 3 4
10
~/git/Chastity-Code-Cookbook/work$ ./main 1 2 3 4 5
15
~/git/Chastity-Code-Cookbook/work$ ./main 1 2 3 4 5 6
21
~/git/Chastity-Code-Cookbook/work$ ./main 1 2 3 4 5 6 7
28
~/git/Chastity-Code-Cookbook/work$ ./main 1 2 3 4 5 6 7 8
36
chastity@chastity-um250:~/git/Chastity-Code-Cookbook/work$

```

Now I can use this program as a super fast method to add numbers from my terminal. But why stop there? Why not build an assembly program that can do all the 4 arithmetic operations?

With a little more work, it is possible! In the next chapter, I will prove it to you!

## Chapter 10: Stack Based Calculator

The program I will present in this chapter is a Stack Based Calculator. This calculator uses a format that is sometimes called Postfix Notation or Reverse Polish Notation.

For a basic breakdown, if you had the chastack program assembled and in your path, you could use this simple expression of adding two numbers 6 and 7:

```
chastack 6 7 add
```

You would get 13 as a result. You can also do

```
chastack 6 7 mul
```

And you will get 42.

You may not appreciate this notation because you are probably used to infix notation that you were taught in school. However, a statement such as “ $3+4*5$ ” confuses humans because they have to remember that the multiplication is supposed to happen before the addition. It is natural to assume that the operations happen from left to right because it is our reading order in English speaking countries. But  $3+4*5$  is not 35 as my mind would assume ( $3+4$  equals 7 and then times 5 equals 35). The convention that society follows says that the answer is 23 due to the  $4*5$  making 20 and then adding the 3.

Postfix notation solves this problem and allows you to choose which order the operations take place. For example, with some commands for chastack, you can do either this:

```
chastack 3 4 add 5 mul
```

This command pushes 3 and 4 to the stack, adds them to get 7, then pushes 5 to the stack and multiplies it by the 7 already on the stack. The result will be 35 because we chose this order of operations.

or you can do:

```
chastack 3 4 5 mul add
```

In the above example, the numbers are the same but the mul command multiplies the 4 and 5 together to get 20, then it adds the 20 to the 3. The result is 23.

Using the stack this way allows us to specify which numbers are on the top of the stack at any point and which operation is used on them.

For more information on RPN notation, read the Wikipedia article.

[https://en.wikipedia.org/wiki/Reverse\\_Polish\\_notation](https://en.wikipedia.org/wiki/Reverse_Polish_notation)

Anyway, the source code is below, and I hope that it helps you understand why this notation is so easy to use.

### FASM chastack

```
format ELF executable
```

```

entry main

include 'chastelib32.asm'

main:

mov dword[radix],10      ;I can choose the radix for integer output!
mov dword[int_width],1  ;and the width of each integer for padded zeros

mov ebp,chastack         ;mov the address of the beginning of the stack to
ebp registers

pop eax                  ;pop the number of arguments from the stack
dec eax                  ;subtract 1 because the program name will be
unused
mov [argc],eax           ;save the argument count for later
pop ebx                  ;pop argument 0 (name of the program, we don't
use it)
cmp eax,0
jnz main_loop           ;if arguments are available, use the main loop

mov eax,string_help
call putstring

main_loop:

cmp [argc],0             ;check for remaining arguments
jz main_loop_end         ;if none, end the loop and stop printing
pop esi                  ;pop the next argument off the stack to esi for
string comparison
dec [argc]               ;subtract 1 from argument count

;Now we process the string we got from the stack
;First, we will try testing for commands
;If any of the predefined strings match the string in esi
;We jump to the label for that command

mov edi,string_setradix
call strcmp
jz command_setradix

mov edi,string_add
call strcmp
jz command_add

mov edi,string_sub
call strcmp
jz command_sub

mov edi,string_mul
call strcmp
jz command_mul

mov edi,string_div

```

```

call strcmp
jz command_div

mov edi,string_rem
call strcmp
jz command_rem

;The default command is to turn the argument into a number and push to
stack

command_num:

mov eax,esi          ;mov the string to eax for processing numbers
call strint          ;try to get a number from the string pointed to by
eax
cmp [strint_error],0 ;did we have zero errors in the strint function?
jz num_push          ;if there were no errors, push this to stack

mov eax,string_err
call putstring        ;print error message
mov eax,esi
call putstring        ;print which command failed
call putline
jmp num_push_end      ;skip the push because this can't be used

num_push:             ;push the number to the fake stack
add ebp,4             ;increment the pointer by the size of the native
int for this mode
mov [ebp],eax          ;mov the value we converted from the string with
strint
num_push_end:
jmp main_loop         ;once value is pushed, continue the program

;These are the labels and code for each of the commands
;When a command is done, we jump back to the beginning of the loop
;the add,sub,mul,div,rem commands are pretty self explanatory
;but I will provide comments for these and other commands

;pop top of stack and set the current radix to it
;it has error checking and leaves the radix as is
;unless at least one number is on the stack
command_setradix:
cmp ebp,chastack       ;is ebp above the address of stack start?
jna change_radix_no    ;if not above, we cannot use it to set the radix
change_radix_yes:
mov eax,[ebp]          ;get the top of stack
mov [radix],eax        ;change the radix
mov dword[ebp],0       ;erase the top of stack
sub ebp,4              ;subtract pointer
jmp main_loop          ;and continue main_loop as normal
change_radix_no:
mov eax,string_err1    ;get error message for less than 1 numbers on
stack
call putstring         ;print error message

```

```

mov eax,esi                ;get name of the command used
call putstring             ;print which command failed
call putline
jmp main_loop

;add number on top of stack to the one below it
command_add:
mov eax,[ebp]
sub ebp,4
add [ebp],eax
jmp memory_check ;check stack for errors after this command

;subtract number on top of stack from the one below it
command_sub:
mov eax,[ebp]
sub ebp,4
sub [ebp],eax
jmp memory_check ;check stack for errors after this command

;multiply number on top of stack by the one below it
command_mul:
mov ebx,[ebp]
sub ebp,4
mov eax,[ebp]
mov edx,0 ;zero edx before multiply
mul ebx ;multiply eax with value in ebx
mov [ebp],eax
jmp memory_check ;check stack for errors after this command

;divide number on top of stack into the one below it
command_div:
mov ebx,[ebp]
sub ebp,4
mov eax,[ebp]
mov edx,0 ;zero edx before divide
div ebx ;divide eax with value in ebx
mov [ebp],eax ;store quotient on stack
jmp memory_check ;check stack for errors after this command

;divide number on top of stack into the one below it
;but leave remainder instead of quotient
command_rem:
mov ebx,[ebp]
sub ebp,4
mov eax,[ebp]
mov edx,0 ;zero edx before divide
div ebx ;divide eax with value in ebx
mov [ebp],edx ;store remainder on stack
jmp memory_check ;check stack for errors after this command

;check if the stack has enough space for the last command
;this will print an error if less than two numbers were on the stack
;when using one of the math commands above
memory_check:

```

```

cmp ebp,chastack      ;is ebp above the address of stack start?
jna print_stack_error ;if not above, explain error to user
mov dword[ebp+4],0    ;if no error, erase the old top of stack
jmp main_loop         ;and continue main_loop as normal
print_stack_error:
mov eax,string_err2   ;get error message for less than 2 numbers on stack
call putstring        ;print error message
mov eax,esi           ;get name of the command used
call putstring        ;print which command failed
call putline
add ebp,4             ;increment the pointer to what it was before the
failed command
jmp main_loop

main_loop_end:

putstack:
cmp ebp,chastack      ;is ebp equal to the address of stack start?
jz putstack_end       ;if it is, end the putstack loop

mov eax,[ebp]
sub ebp,4

call putint_and_line

jmp putstack
putstack_end:

mov eax,1             ;exit (kernel opcode 1 on 32 bit systems)
mov ebx,0             ;return 0 status on exit - 'No Errors'
int 80h              ;system call for 32-bit Linux kernel

argc dd 0

string_err db 'Error: invalid number or command: ',0 ;Generic error
message
string_err1 db 'Error: need one number on stack for command: ',0 ;math
fail error when less than one number on the stack
string_err2 db 'Error: need two numbers on stack for command: ',0 ;math
fail error when less than two numbers on the stack

string_setradix db 'setradix',0
string_add db 'add',0
string_sub db 'sub',0
string_mul db 'mul',0
string_div db 'div',0
string_rem db 'rem',0

string_help db 'chastack is a stack based command line calculator',0xA
db 'Numbers are pushed on the stack and commands can do
math.',0xA
db 'Commands are add,sub,mul,div,rem',0xA
db 'Example: "chastack 3 4 5 add mul"',0xA,0,0

```

```
;strcmp compares the string at esi to the one at edi
;eax returns 0 if the strings are the same and 1 if different
;the algorithm is simple but I will explain it for those who are
confused
```

```
;eax is initialized to zero
;a byte from each string is loaded into the al and bl registers
;the bytes are compared. if they are different, then we jump to the end
;However, if they are the same, then we check if one of them is zero
;for this purpose it doesn't matter whether we compare al or bl with
zero
;because it is known that they are the same if the jnz did not take
place
;if it is zero, this also jumps to the end of the function
;If neither jump took place, then we jump to the start of the loop
;but when the function finally ends bl will be subtracted from al
;this ensures that the function returns zero if the final characters are
the same
;ebx,esi,and edi are preserved but eax is the return value
;also, the sub instruction at the end of the function also updates the
flags
;so you can "jz" or "jnz" to a label after calling this function based
on results
```

```
strcmp:
```

```
push ebx
push esi
push edi
```

```
mov eax,0
```

```
strcmp_start:
```

```
;read a byte from each string
mov al,[edi]
mov bl,[esi]
cmp al,bl
jnz strcmp_end
```

```
cmp al,0
jz strcmp_end
```

```
inc edi
inc esi
```

```
jmp strcmp_start
```

```
strcmp_end:
sub al,bl
```

```
pop edi
pop esi
pop ebx
```

```
ret
```

```
;because the actual hardware stack is used to process the command line  
arguments.  
;I allocate memory for a virtual stack that we can index as if it was  
the real stack  
;I name it "chastack" for Chastity's stack.
```

```
chastack: rd 0x100
```

One thing that seems odd about this program is that the virtual stack I use increases the ebp pointer when numbers are added to the stack and decreases them when numbers are removed. This is opposite of the way the real hardware stack works on Intel machines. In any case, the result is basically the same.

The program pushes each number from the command line arguments onto the stack. It uses the `strint_error` variable to determine whether the `strint` function failed. If there was even one invalid character, it will print an error message.

However, it checks for strings matching one of the commands “add,” “sub”, “mul”, “div”, or “rem”. If any of these are encountered by the string comparison function (`strcmp`), it will use the top two numbers on the stack for that operation and then leave the result.

The program usually achieves this by moving the top memory locations into registers, decrementing ebp, and then operating those register and storing the result on the stack.

This program heavily depends on the concept of a stack as well as the `strint` function for getting a number from a string or checking the strings for commands with `strcmp`. This kind of calculator is easier to write than an infix notation because a program has a harder time figuring out “Please excuse my dear Aunt Sally” than it does a simple linear order of operations left to right.

The main difference my calculator has is that it uses names of assembly instructions as the operators, with the exception of “rem”, which gets the remainder from a `div` instruction when it is stored in the `edx` register. Using these three letter words is convenient in the context of a book on Assembly language.

There are entire programming languages like Forth, dc, STOIC, and Factor which use the concept of a stack to perform math in this order. Really, the stack is just a special case of an array which can be implemented in any programming language, but is more easily understood in Assembly.

When I wrote this program, I wanted to use the real stack for doing the math, but that doesn’t work when the arguments have to be popped off the stack in the first place to operate the calculator.

Though it could be said that a program which read from standard input or a file would be able to use the real stack in the way I intended, but it would not be as convenient as one that can be scripted with arguments as the one I have presented in this chapter.

Whether you like my calculator or not, you have to admit that this has all the elements expected of a standard Linux command line tool.

- input via command arguments so a user can direct the program for what they need.
- output so the user can see the result
- error handling so that the programmer or user can see when something is wrong
- A proper exit condition for the program to avoid an infinite loop. In this case, running out of arguments.

Before I end this chapter, I will include the contents of the “chastelib32.asm” file that was included in the chystack program.

```
{id: chastelib32-asm} ## chastelib32.asm
; chastelib assembly header file for 32 bit Linux
; This file is where I keep the source of my most important Assembly
functions
; These are my string and integer output and conversion routines.

; To simplify documentation. The Accumulator/Arithmetic register
; (ax,eax,rax) depending on bit size shall be referred to as register A
; for the description of these core functions because the A register
; is treated special both by the Intel company and my code;

; putstring; Prints a zero terminated string from the address pointer to
by A register.
; intstr;    Converts the number in A into a zero terminated string and
points A to that address
; putint;    Prints the integer in A by calling intstr and then
putstring.
; strint;    Converts the zero terminated string into an integer and
sets A to that value

; Now, the source of the functions begins, with comments included for
parts that I felt needed explanation.

putstring:

push eax
push ebx
push ecx
push edx

mov ebx,eax                ;copy eax to ebx to be used as index to the
string

putstring_strlen_start: ;this loop finds the length of the string as
part of the putstring function

cmp [ebx],byte 0           ;compare byte at address ebx with 0
jz putstring_strlen_end ;if comparison was zero, jump to loop end
because we have found the length
inc ebx
```

```

jmp putstring_strlen_start

putstring_strlen_end:
sub ebx,eax ;subtract start pointer from current pointer to get length
of string

;Write string using Linux Write system call.
;Reference for 32 bit x86 syscalls is below.
;https://www.chromium.org/chromium-os/developer-library/reference/linux-constants/syscalls/#x86-32-bit

mov edx,ebx      ;number of bytes to write
mov ecx,eax      ;pointer/address of string to write
mov ebx,1        ;write to the STDOUT file
mov eax,4        ;write (kernel opcode 4 on 32 bit systems)
int 80h          ;system call for 32-bit Linux kernel

pop edx
pop ecx
pop ebx
pop eax

ret ;this is the end of the putstring function return to calling
location

; This is the location in memory where digits are written to by the
intstr function
; The string of bytes and settings such as the radix and width are
global variables defined below.

int_string db 32 dup '?' ;reserve bytes for characters string for 32-bit
binary integer

int_string_end db 0 ;zero byte terminator for the integer string

radix dd 2      ;radix or base for integer output. 2=binary, 8=octal,
10=decimal, 16=hexadecimal
int_width dd 8 ;default width of integers. Extra zeros prefixed if more
than 1

;this function creates a string of the integer in eax
;it uses the above radix variable to determine base from 2 to 36
;it then loads eax with the address of the string
;this means that it can be used with the putstring function

intstr:

mov ebx,int_string_end-1 ;find address of lowest digit(just before the
newline 0Ah)
mov ecx,1

digits_start:

mov edx,0;

```

```

div dword [radix]
cmp edx,10
jb decimal_digit
jnb hexadecimal_digit

```

```

decimal_digit: ;we go here if it is only a digit 0 to 9
add edx,'0'
jmp save_digit

```

```

hexadecimal_digit:
sub edx,10
add edx,'A'

```

```

save_digit:

```

```

mov [ebx],dl
cmp eax,0
jz intstr_end
dec ebx
inc ecx
jmp digits_start

```

```

intstr_end:

```

```

prefix_zeros:
cmp ecx,[int_width]
jnb end_zeros
dec ebx
mov [ebx],byte '0'
inc ecx
jmp prefix_zeros
end_zeros:

```

```

mov eax,ebx ;point eax register to this string for putstring

```

```

ret

```

```

;function to print string form of whatever integer is in eax
;The radix determines which number base the string form takes.
;Anything from 2 to 36 is a valid radix
;in practice though, only bases 2,8,10,and 16 will make sense to other
programmers
;this function does not process anything by itself but calls the
combination of my other
;functions in the order I intended them to be used.

```

```

putint:

```

```

push eax
push ebx
push ecx
push edx

```

```

call intstr

```

```
call putstring
```

```
pop edx  
pop ecx  
pop ebx  
pop eax
```

```
ret
```

```
;this function converts a string pointed to by eax into an integer  
returned in eax instead  
;it is a little complicated because it has to account for whether the  
character in  
;a string is a decimal digit 0 to 9, or an alphabet character for bases  
higher than ten  
;it also checks for both uppercase and lowercase letters for bases 11 to  
36  
;finally, it checks if that letter makes sense for the base.  
;For example, G to Z cannot be used in hexadecimal, only A to F can  
;The purpose of writing this function was to be able to accept user  
input as integers  
;This function is improved with error checking and uses the new  
strint_error variable  
;The program can check this value after the call and see how many errors  
happened.
```

```
strint_error db 0 ;declare a byte variable that keeps track of errors
```

```
strint:
```

```
mov ebx,eax ;copy string address from eax to ebx because eax will be  
replaced soon!  
mov eax,0  
mov byte[strint_error],0 ;set errors to 0 at the start of this function
```

```
read_strint:
```

```
mov ecx,0 ;zero ecx so only lower 8 bits are used  
mov cl,[ebx]  
inc ebx  
cmp cl,0 ;compare this byte with 0  
jz strint_end ; if comparison was zero, this is the end of string
```

```
;if char is below '0' or above '9', it is outside the range of these and  
is not a digit  
cmp cl,'0'  
jb not_digit  
cmp cl,'9'  
ja not_digit
```

```
;but if it is a digit, then correct and process the character  
is_digit:  
sub cl,'0'  
jmp process_char
```

```

not_digit:
;it isn't a decimal digit, but it could be perhaps an alphabet character
;which could be a digit in a higher base like hexadecimal
;we will check for that possibility next

;if char is below 'A' or above 'Z', it is outside the range of these and
is not capital letter
cmp cl, 'A'
jb not_upper
cmp cl, 'Z'
ja not_upper

is_upper:
sub cl, 'A'
add cl, 10
jmp process_char

not_upper:

;if char is below 'a' or above 'z', it is outside the range of these and
is not lowercase letter
cmp cl, 'a'
jb not_lower
cmp cl, 'z'
ja not_lower

is_lower:
sub cl, 'a'
add cl, 10
jmp process_char

not_lower:

;if we have reached this point, result invalid and end function with
error
jmp strint_end_error

process_char:

cmp ecx, [radix] ;compare char with radix
jnb strint_end_error ;if this value is above or equal to radix, it is
too high despite being a valid digit/alpha

mov edx, 0 ;zero edx because it is used in mul sometimes
mul dword [radix] ;mul eax with radix
add eax, ecx

jmp read_strint ;jump back and continue the loop if nothing has exited
it

strint_end_error: ;we jump here if there was an error with one of the
chars
inc byte[strint_error] ;increment error counter because char invalid

```

```

strint_end: ;we jump here when no errors happened

ret

;The utility functions below simply print a space or a newline.
;these help me save code when printing lots of strings and integers.

space db ' ',0 ;a string containing only a space

putspace:
push eax
mov eax,space
call putstring
pop eax
ret

line db 0Ah,0 ;a string containing only a newline

;the next function which pushes eax to the stack
;moves the address of the line string and prints it with putstring
;then it pops the original value of eax back from the stack before the
function returns
;this allows me to print a newline anywhere in the code without a single
register changing

putline:
push eax
mov eax,line
call putstring
pop eax
ret

;a function for printing a single character that is the value of al

char: db 0,0

putchar:
push eax
mov [char],al
mov eax,char
call putstring
pop eax
ret

;a small function just for the common operation of
;printing an integer followed by a space
;this saves a few bytes in the assembled code
;by reducing the number of function calls in the main program

putint_and_space:
call putint
call putspace
ret

```

```
;a small function just for the common operation of  
;printing an integer followed by a line feed  
;this saves a few bytes in the assembled code  
;by reducing the number of function calls in the main program
```

```
putint_and_line:  
call putint  
call putline  
ret
```

```
;a small function just for the common operation of  
;printing a string followed by a line feed  
;this saves a few bytes in the assembled code  
;by reducing the number of function calls in the main program  
;it also means we don't need to include a newline in every string!
```

```
putstr_and_line:  
call putstring  
call putline  
ret
```

That header file will be reused in most other programs in this book.

## Chapter 11 A Note on Portability

Assembly has a bad reputation for not being portable. However, portability is not as hard to achieve with Assembly language as it used to be.

First, almost all machines you will buy in a store contain either Intel chips or ARM chips. This means that the number of processors we need to be concerned with is very limited.

I can't speak about ARM Assembly because the only working machine I have with an ARM CPU can't connect to the internet (long story). However, because of the dominance of Intel on desktop and laptop PCs, everything in this book will apply to most users.

Second, I am not concerned with porting between operating systems because Linux and other Unix-Like systems use the same set of system calls. Linux is free in the sense of Libre (freedom) and gratis (no cost) because distros can be downloaded and copied to CDs, DVDs, bootable USB drives, and even run in PC emulators like QEMU.

Third, I have successfully ported software from Assembly to both DOS and Windows because only the system calls need to be rewritten that conform to the operating system API. The programs in the DOS edition of Assembly Arithmetic Algorithms all started out as Linux programs originally, for example.

But in this chapter, I want to talk about a specific portability issue: Portability between 32-bit and 64-bit modes. These two modes of loading and running a program are both supported by the FASM assembler. A future book will focus on the particulars of 64 bit Assembly programming for Linux, but you can consider this chapter a preview and a migration guide if you have a program that you need to convert from running in 32 bit mode to 64 bit mode.

### Difference between 32 and 64 bit modes

The 8 general purpose registers are extended to 64 bits. The E is changed to R. Therefore, they are the same as described in chapter 1 except each of them are 64-bits.

- RAX: The Accumulator Register
- RBX: The Base Register
- RCX: The Count Register
- RDX: The Data Register
- RSI: The Source Index
- RDI: the Destination index
- RBP: The Base Pointer
- RSP: The Stack Pointer

Besides these 8 registers, which can be used in the same way as their equivalent E?? 32 bit-names, there are also 8 more 64 bit registers named R8 to R15. You can think of them as extra storage space available to you in 64 bit mode.

The pusha/pushad and popa/popad instructions no longer exist in 64 bit mode. I haven't mentioned these instructions before nor have I used these in any of my programs because I was aware that it would break compatibility with 64-bit code.

But for historical purposes, what these instructions could do in 16 and 32 bit modes was push or pop all the 8 general purpose registers at the same time. This means that instead of pushing and popping eax,ebx,ecx,edx each on separate lines, I could have just replaced it with pusha at the beginning of a function and popa at the end of a function.

Doing such a thing would have reduced the size of my programs by a few bytes, but when I wrote this book I prioritized portability between the 3 platforms that matter to me.

- 16-bit DOS (available via emulators on every computer)
- 32-bit Linux (the convention used in this book)
- 64-bit Linux (used for a future book and special programs)

For the most part, I can port any program between these platforms fairly easily with only a few small changes.

## Converting 32 to 64 bit mode

Step 1: Replace every 'e' in register names to 'r'. This makes them switch to the full 64 bit register. This is very easy.

Step 2: Change the system call convention to the new 64 bit mode which uses different registers but has the same calls available.

Between these two basic steps, step 2 is a lot harder. To make this easy, I have created some tables and information on the arguments required.

{id: chastitys-linux-system-call-table} ## Chastity's Linux System Call Table

The following tables show the main 6 system calls that all of my programs use. 32 bit calls are accessed with "int 0x80" to call the kernel with an interrupt. 64 bit calls are accessed with the "syscall" instruction. As long as you load the right data into the registers described, you can use these calls for useful programs.

### 32-bit Intel System Calls for Linux

| Number | Name  | eax  | ebx      | ecx    | edx    |
|--------|-------|------|----------|--------|--------|
| 1      | exit  | 0x01 | status   |        |        |
| 3      | read  | 0x03 | fd       | buf    | count  |
| 4      | write | 0x04 | fd       | buf    | count  |
| 5      | open  | 0x05 | filename | flags  | mode   |
| 6      | close | 0x06 | fd       |        |        |
| 19     | lseek | 0x13 | fd       | offset | whence |

## 64-bit Intel System Calls for Linux

| Number | Name  | rax  | rdi      | rsi    | rdx    |
|--------|-------|------|----------|--------|--------|
| 60     | exit  | 0x3C | status   |        |        |
| 0      | read  | 0x00 | fd       | buf    | count  |
| 1      | write | 0x01 | fd       | buf    | count  |
| 2      | open  | 0x02 | filename | flags  | mode   |
| 3      | close | 0x03 | fd       |        |        |
| 8      | lseek | 0x08 | fd       | offset | whence |

The following names for the arguments are based on the Linux manual pages for those system calls. You can access the same from any Linux distro. For example, “man 2 write” will show the C function signature for the write call.

**status** for exit call is a byte number from 0 to 255 to be returned to the operating system. Shell scripts use this to know if something went wrong when running the program. By convention, a return of 0 means no errors happened.

**fd** refers to a number which is used as a file descriptor. fd 0 is stdin and fd 1 is stdout. Other file descriptors are returned from an open call. Any fd is returned in the eax register in Assembly language. This same fd must be saved somewhere and used whenever you want to read, write, close, or lseek using the file you opened.

**buf** refers to pointer which will be read into with a read call or written from using a write call.

**count** refers to how many bytes are going to be read or written with the read or write calls.

### Values for the flags argument to open.

```
O_RDONLY    00
O_WRONLY    01
O_RDWR      02
O_CREAT     0100
```

### Values for the mode argument to open.

- S\_IRWXU 00700 user (file owner) has read, write, and execute permission
- S\_IRUSR 00400 user has read permission
- S\_IWUSR 00200 user has write permission
- S\_IXUSR 00100 user has execute permission
- S\_IRWXG 00070 group has read, write, and execute permission
- S\_IRGRP 00040 group has read permission
- S\_IWGRP 00020 group has write permission
- S\_IXGRP 00010 group has execute permission
- S\_IRWXO 00007 others have read, write, and execute permission
- S\_IROTH 00004 others have read permission

- S\_IWOTH 00002 others have write permission
- S\_IXOTH 00001 others have execute permission

**offset** is the address you want to go to in the file. It is used in the lseek call and depends on the **whence** argument

## Values for the whence argument to lseek.

- SEEK\_SET 0 Seek from beginning of file.
- SEEK\_CUR 1 Seek from current position.
- SEEK\_END 2 Seek from end of file.

Numbers used for file flags or modes are prefixed with zeros because they are octal constants and this is how they are represented in the C Programming Language. However, in Assembly, you need to look up your Assembler's rules for octal constants. For example, in FASM, they must end with the letter 'o'.

## Actual Conversion Example

I realize that the information in my tables and descriptions of data for them won't work for everyone. You need a real example to see how it is possible to convert a program to 64 bit. Therefore, I present to you the 64-bit Linux edition of the chystack program in the last chapter. I have ensured that each line of both the 32-bit and 64-bit source files are doing the exact same thing.

### FASM chystack 64-bit

```
format ELF64 executable
entry main
```

```
include 'chastelib64.asm'
```

```
main:
```

```
mov qword[radix],10      ;I can choose the radix for integer output!
mov qword[int_width],1  ;and the width of each integer for padded zeros

mov rbp,chystack        ;mov the address of the beginning of the stack to
rbp registers

pop rax                  ;pop the number of arguments from the stack
dec rax                  ;subtract 1 because the program name will be
unused
mov [argc],rax           ;save the argument count for later
pop rbx                  ;pop argument 0 (name of the program, we don't
use it)
cmp rax,0
jnz main_loop            ;if arguments are available, use the main loop

mov rax,string_help
```

```

call putstring

main_loop:

cmp [argc],0          ;check for remaining arguments
jz main_loop_end      ;if none, end the loop and stop printing
pop rsi               ;pop the next argument off the stack to rsi for
string comparison
dec [argc]            ;subtract 1 from argument count

;Now we process the string we got from the stack
;First, we will try testing for commands
;If any of the predefined strings match the string in rsi
;We jump to the label for that command

mov rdi,string_setradix
call strcmp
jz command_setradix

mov rdi,string_add
call strcmp
jz command_add

mov rdi,string_sub
call strcmp
jz command_sub

mov rdi,string_mul
call strcmp
jz command_mul

mov rdi,string_div
call strcmp
jz command_div

mov rdi,string_rem
call strcmp
jz command_rem

;The default command is to turn the argument into a number and push to
stack

command_num:

mov rax,rsi           ;mov the string to rax for processing numbers
call strint           ;try to get a number from the string pointed to by
rax
cmp [strint_error],0 ;did we have zero errors in the strint function?
jz num_push           ;if there were no errors, push this to stack

mov rax,string_err
call putstring        ;print error message
mov rax,rsi
call putstring        ;print which command failed

```

```

call putline
jmp num_push_end      ;skip the push because this can't be used

num_push:              ;push the number to the fake stack
add rbp,8              ;increment the pointer by the size of the native
int for this mode
mov [rbp],rax          ;mov the value we converted from the string with
strint
num_push_end:
jmp main_loop          ;once value is pushed, continue the program

;These are the labels and code for each of the commands
;When a command is done, we jump back to the beginning of the loop
;the add,sub,mul,div,rem commands are pretty self explanatory
;but I will provide comments for these and other commands

;pop top of stack and set the current radix to it
;it has error checking and leaves the radix as is
;unless at least one number is on the stack
command_setradix:
cmp rbp,chastack       ;is ebp above the address of stack start?
jna change_radix_no    ;if not above, we cannot use it to set the radix
change_radix_yes:
mov rax,[rbp]          ;get the top of stack
mov [radix],rax        ;change the radix
mov qword[rbp],0       ;erase the top of stack
sub rbp,8              ;subtract pointer
jmp main_loop          ;and continue main_loop as normal
change_radix_no:
mov rax,string_err1    ;get error message for less than 1 numbers on
stack
call putstring         ;print error message
mov rax,rsi            ;get name of the command used
call putstring         ;print which command failed
call putline
jmp main_loop

;add number on top of stack to the one below it
command_add:
mov rax,[rbp]
sub rbp,8
add [rbp],rax
jmp memory_check ;check stack for errors after this command

;subtract number on top of stack from the one below it
command_sub:
mov rax,[rbp]
sub rbp,8
sub [rbp],rax
jmp memory_check ;check stack for errors after this command

;multiply number on top of stack by the one below it
command_mul:
mov rbx,[rbp]

```

```

sub rbp,8
mov rax,[rbp]
mov rdx,0      ;zero rdx before multiply
mul rbx        ;multiply rax with value in rbx
mov [rbp],rax
jmp memory_check ;check stack for errors after this command

;divide number on top of stack into the one below it
command_div:
mov rbx,[rbp]
sub rbp,8
mov rax,[rbp]
mov rdx,0 ;zero rdx before divide
div rbx   ;divide rax with value in rbx
mov [rbp],rax ;store quotient on stack
jmp memory_check ;check stack for errors after this command

;divide number on top of stack into the one below it
;but leave remainder instead of quotient
command_rem:
mov rbx,[rbp]
sub rbp,8
mov rax,[rbp]
mov rdx,0 ;zero rdx before divide
div rbx   ;divide rax with value in rbx
mov [rbp],rdx ;store remainder on stack
jmp memory_check ;check stack for errors after this command

;check if the stack has enough space for the last command
;this will print an error if less than two numbers were on the stack
;when using one of the math commands above
memory_check:
cmp rbp,chastack      ;is ebp above the address of stack start?
jna print_stack_error ;if not above, explain error to user
mov qword[ebp+8],0    ;if no error, erase the old top of stack
jmp main_loop         ;and continue main_loop as normal
print_stack_error:
mov rax,string_err2   ;get error message for less than 2 numbers on stack
call putstring        ;print error message
mov rax,rsi           ;get name of the command used
call putstring        ;print which command failed
call putline
add rbp,8             ;increment the pointer to what it was before the
failed command
jmp main_loop

main_loop_end:

putstack:
cmp rbp,chastack ;is rbp equal to the address of stack start?
jz putstack_end ;if it is, end the putstack loop

mov rax,[rbp]
sub rbp,8

```

```
call putint_and_line
```

```
jmp putstack  
putstack_end:
```

```
mov rax,0x3C      ;exit (kernel opcode 0x3C on 64 bit systems) (60  
decimal)  
mov rdi,0         ;return 0 status on exit - 'No Errors'  
syscall          ;system call for 64-bit Linux kernel
```

```
argc dq 0
```

```
string_err db 'Error: invalid number or command: ',0 ;Generic error  
message  
string_err1 db 'Error: need one number on stack for command: ',0 ;math  
fail error when less than one number on the stack  
string_err2 db 'Error: need two numbers on stack for command: ',0 ;math  
fail error when less than two numbers on the stack
```

```
string_setradix db 'setradix',0  
string_add db 'add',0  
string_sub db 'sub',0  
string_mul db 'mul',0  
string_div db 'div',0  
string_rem db 'rem',0
```

```
string_help db 'chastack is a stack based command line calculator',0xA  
            db 'Numbers are pushed on the stack and commands can do  
math.',0xA  
            db 'Commands are add,sub,mul,div,rem',0xA  
            db 'Example: "chastack 3 4 5 add mul"',0xA,0,0
```

```
;strcmp compares the string at rsi to the one at rdi  
;rax returns 0 if the strings are the same and 1 if different  
;the algorithm is simple but I will explain it for those who are  
confused
```

```
;rax is initialized to zero  
;a byte from each string is loaded into the al and bl registers  
;the bytes are compared. if they are different, then we jump to the end  
;However, if they are the same, then we check if one of them is zero  
;for this purpose it doesn't matter whether we compare al or bl with  
zero  
;because it is known that they are the same if the jnz did not take  
place  
;if it is zero, this also jumps to the end of the function  
;If neither jump took place, then we jump to the start of the loop  
;but when the function finally ends bl will be subtracted from al  
;this ensures that the function returns zero if the final characters are  
the same  
;rbx,rsi,and rdi are preserved but rax is the return value  
;also, the sub instruction at the end of the function also updates the  
flags
```

;so you can "jz" or "jnz" to a label after calling this function based on results

strcmp:

```
push rbx
push rsi
push rdi
```

```
mov rax,0
```

strcmp\_start:

```
;read a byte from each string
mov al,[rdi]
mov bl,[rsi]
cmp al,bl
jnz strcmp_end
```

```
cmp al,0
jz strcmp_end
```

```
inc rdi
inc rsi
```

```
jmp strcmp_start
```

```
strcmp_end:
sub al,bl
```

```
pop rdi
pop rsi
pop rbx
```

```
ret
```

;because the actual hardware stack is used to process the command line arguments.

;I allocate memory for a virtual stack that we can index as if it was the real stack

;I name it "chastack" for Chastity's stack.

```
chastack: rq 0x100
```

## **chastelib64.asm**

; chastelib assembly header file for 64 bit Linux

; This file is where I keep the source of my most important Assembly functions

; These are my string and integer output and conversion routines.

; To simplify documentation. The Accumulator/Arithmetic register

; (ax,eax,rax) depending on bit size shall be referred to as register A

; for the description of these core functions because the A register

```
; is treated special both by the Intel company and my code;

; putstring; Prints a zero terminated string from the address pointer to
by A register.
; intstr;    Converts the number in A into a zero terminated string and
points A to that address
; putint;    Prints the integer in A by calling intstr and then
putstring.
; strint;    Converts the zero terminated string into an integer and
sets A to that value

; Now, the source of the functions begins, with comments included for
parts that I felt needed explanation.
```

```
putstring:
```

```
push rax
push rdi
push rsi
push rdx
```

```
mov rdi,rax          ;copy eax to rdi to be used as index to the
string
```

```
putstring_strlen_start: ;this loop finds the length of the string as
part of the putstring function
```

```
cmp [rdi],byte 0      ;compare byte at address rdi with 0
jz putstring_strlen_end ;if comparison was zero, jump to loop end
because we have found the length
inc rdi
jmp putstring_strlen_start
```

```
putstring_strlen_end:
sub rdi,rax ;subtract start pointer from current pointer to get length
of string
```

```
;Write string using Linux Write system call.
;Reference for 64 bit x86 syscalls is below.
;https://www.chromium.org/chromium-os/developer-library/reference/linux-constants/syscalls/#x86\_64-64-bit
```

```
mov rdx,rdi          ;number of bytes to write
mov rsi,rax          ;pointer/address of string to write
mov rdi,1            ;write to the STDOUT file
mov rax,1            ;write (kernel opcode 1 on 64 bit systems)
syscall              ;system call for 64-bit Linux kernel
```

```
pop rdx
pop rsi
pop rdi
pop rax
```

```
ret ;this is the end of the putstring function return to calling
```

location

; This is the location in memory where digits are written to by the  
intstr function  
; The string of bytes and settings such as the radix and width are  
global variables defined below.

int\_string db 64 dup '?' ;reserve bytes for characters string for 64-bit  
binary integer

int\_string\_end db 0 ;zero byte terminator for the integer string

radix dq 2 ;radix or base for integer output. 2=binary, 8=octal,  
10=decimal, 16=hexadecimal

int\_width dq 8 ;default width of integers. Extra zeros prefixed if more  
than 1

;this function creates a string of the integer in rax  
;it uses the above radix variable to determine base from 2 to 36  
;it then loads rax with the address of the string  
;this means that it can be used with the putstring function

intstr:

mov rbx,int\_string\_end-1 ;find address of lowest digit(just before the  
newline 0Ah)  
mov rcx,1

digits\_start:

mov rdx,0;  
div qword [radix]  
cmp rdx,10  
jb decimal\_digit  
jnb hexadecimal\_digit

decimal\_digit: ;we go here if it is only a digit 0 to 9  
add rdx,'0'  
jmp save\_digit

hexadecimal\_digit:  
sub rdx,10  
add rdx,'A'

save\_digit:

mov [rbx],dl  
cmp rax,0  
jz intstr\_end  
dec rbx  
inc rcx  
jmp digits\_start

intstr\_end:

```

prefix_zeros:
cmp rcx,[int_width]
jnb end_zeros
dec rbx
mov [rbx],byte '0'
inc rcx
jmp prefix_zeros
end_zeros:

```

```

mov rax,rbx ;point eax register to this string for putstring

```

```

ret

```

```

;function to print string form of whatever integer is in rax
;The radix determines which number base the string form takes.
;Anything from 2 to 36 is a valid radix
;in practice though, only bases 2,8,10,and 16 will make sense to other
programmers
;this function does not process anything by itself but calls the
combination of my other
;functions in the order I intended them to be used.

```

```

putint:

```

```

push rax
push rbx
push rcx
push rdx

```

```

call intstr
call putstring

```

```

pop rdx
pop rcx
pop rbx
pop rax

```

```

ret

```

```

;this function converts a string pointed to by rax into an integer
returned in rax instead
;it is a little complicated because it has to account for whether the
character in
;a string is a decimal digit 0 to 9, or an alphabet character for bases
higher than ten
;it also checks for both uppercase and lowercase letters for bases 11 to
36
;finally, it checks if that letter makes sense for the base.
;For example, G to Z cannot be used in hexadecimal, only A to F can
;The purpose of writing this function was to be able to accept user
input as integers
;This function is improved with error checking and uses the new
strint_error variable

```

;The program can check this value after the call and see how many errors happened.

strint\_error db 0 ;declare a byte variable that keeps track of errors

strint:

mov rbx,rax ;copy string address from rax to rbx because rax will be replaced soon!

mov rax,0

mov byte[strint\_error],0 ;set errors to 0 at the start of this function

read\_strint:

mov rcx,0 ;zero rcx so only lower 8 bits are used

mov cl,[rbx]

inc rbx

cmp cl,0 ;compare this byte with 0

jz strint\_end ; if comparison was zero, this is the end of string

;if char is below '0' or above '9', it is outside the range of these and is not a digit

cmp cl,'0'

jb not\_digit

cmp cl,'9'

ja not\_digit

;but if it is a digit, then correct and process the character

is\_digit:

sub cl,'0'

jmp process\_char

not\_digit:

;it isn't a decimal digit, but it could be perhaps an alphabet character

;which could be a digit in a higher base like hexadecimal

;we will check for that possibility next

;if char is below 'A' or above 'Z', it is outside the range of these and is not capital letter

cmp cl,'A'

jb not\_upper

cmp cl,'Z'

ja not\_upper

is\_upper:

sub cl,'A'

add cl,10

jmp process\_char

not\_upper:

;if char is below 'a' or above 'z', it is outside the range of these and is not lowercase letter

cmp cl,'a'

jb not\_lower

```

cmp cl,'z'
ja not_lower

is_lower:
sub cl,'a'
add cl,10
jmp process_char

not_lower:

;if we have reached this point, result invalid and end function with
error
jmp strint_end_error

process_char:

cmp rcx,[radix] ;compare char with radix
jnb strint_end_error ;if this value is above or equal to radix, it is
too high despite being a valid digit/alpha

mov rdx,0 ;zero rdx because it is used in mul sometimes
mul qword [radix] ;mul rax with radix
add rax,rcx

jmp read_strint ;jump back and continue the loop if nothing has exited
it

strint_end_error: ;we jump here if there was an error with one of the
chars
inc byte[strint_error] ;increment error counter because char invalid

strint_end: ;we jump here when no errors happened

ret

;The utility functions below simply print a space or a newline.
;these help me save code when printing lots of strings and integers.

space db ' ',0 ;a string containing only a space

putspace:
push rax
mov rax,space
call putstring
pop rax
ret

line db 0Ah,0 ;a string containing only a newline

;the next function which pushes rax to the stack
;moves the address of the line string and prints it with putstring
;then it pops the original value of rax back from the stack before the
function returns
;this allows me to print a newline anywhere in the code without a single

```

register changing

```
putline:
push rax
mov rax,line
call putstring
pop rax
ret
```

;a function for printing a single character that is the value of al

```
char: db 0,0
```

```
putchar:
push rax
mov [char],al
mov rax,char
call putstring
pop rax
ret
```

;a small function just for the common operation of  
;printing an integer followed by a space  
;this saves a few bytes in the assembled code  
;by reducing the number of function calls in the main program

```
putint_and_space:
call putint
call putspace
ret
```

;a small function just for the common operation of  
;printing an integer followed by a line feed  
;this saves a few bytes in the assembled code  
;by reducing the number of function calls in the main program

```
putint_and_line:
call putint
call putline
ret
```

;a small function just for the common operation of  
;printing a string followed by a line feed  
;this saves a few bytes in the assembled code  
;by reducing the number of function calls in the main program  
;it also means we don't need to include a newline in every string!

```
putstr_and_line:
call putstring
call putline
ret
```

## Pros and Cons of Conversion to 64 Bits

Of all the programs I have written in Assembly, only chystack actually benefits from having an increased bit size for registers. This means larger numbers can be added, subtracted, multiplied, and divided.

On the flip side, I have never converted this program to 16-bit DOS because doing so would reduce the capabilities of it to work with only 16-bit numbers. Since I have to run DOS in an emulator, I also don't gain any convenience or speed. For the most part the 32 bit edition is more than I need.

But this calculator was actually written as a teaching tool to help people learn that there are other ways of building a calculator. Stack based Reverse Polish Notation is the best way based on my experience.

## Chapter 12: The Lseek System Call

In the last chapter, I showed you tables of my favorite 6 system calls and the arguments that you need to put in which registers. However, of the 6 system calls, there is only one that has not been included in any program in this book so far.

The “lseek” call is named as a short form of “long seek”. This is because it was a 32-bit upgrade from “seek” which took a 16 bit number as an offset to seek to.

Anyway, the lseek function is the equivalent of a “goto” or “jmp”, except it moves the file pointer associated with a particular file descriptor. Whenever we have used the read or write system calls, the pointer automatically goes to the next position in the file after the data that was read or written.

For example, if you were reading a file, you would open it and then the pointer would start at address 0. You would read bytes until the read call returned 0 in the eax register as the number of bytes read. If 0 bytes were read, you are at the end of the file.

Similarly, if you were creating and writing to a new file, you would typically write to it starting at the beginning and then close it whenever you were done. The end of the file would be wherever you were when you stopped.

For 95 percent of cases for reading or writing files, you go from address 0 to EOF (End Of File). In fact, this is the recommended way for most programs and naturally is the reason the file pointer is updated for you automatically.

To understand some basic terminology, the words **pointer**, **address**, and **offset** all mean exactly the same thing. It just means a place in a file instead of memory in this specific context.

Although the need for lseek is quite rare, there will be programs later in this book that can’t operate without it. That is why I will help you understand it with a basic example.

First, create a small text file named “openthis.txt” by redirecting the output of the echo command, or just use your favorite text editor to write whatever you feel like!

```
echo "This is a normal file. Nothing suspicious at all." > openthis.txt
```

Now that we have a file to open, we can write a small program to open this file and change all lowercase letters into uppercase letters.

### FASM cat upper

format ELF executable

```
main:                                ;the main function of the assembly program

mov dword[radix],16                  ;I can choose the radix for integer output!
mov dword[int_width],1               ;and the width of each integer for padded zeros
```

```

;Linux system call to open a file

mov ecx,2          ;open file in read and write mode
mov ebx,filename   ;filename should be in eax before this function was
called
mov eax,5          ;invoke SYS_OPEN (kernel opcode 5)
int 80h           ;call the kernel

mov dword [fd],eax ;save the file descriptor returned in eax

cmp eax,0
jns cat_upper ;if eax is not negative/signed there was no error

;Otherwise, if it was signed/negative, display an error message.

neg eax           ;turn the negative in eax positive
call putint_and_space ;print the number in eax (error code) and space
mov eax,open_error ;get the error message I defined
call putstr_and_line ;print the error message and newline

jmp main_end      ;end the program because we can't open the file

cat_upper:

;use the read call to read 1 byte from the file

mov eax,3          ;invoke SYS_READ (kernel opcode 3)
mov ebx,[fd]       ;move the opened file descriptor into ebx
mov ecx,buf        ;address to store the byte
mov edx,1          ;number of bytes to read
int 80h           ;call the kernel

cmp eax,0          ;were zero bytes read?
jz cat_upper_end   ;if yes, then end the loop

;otherwise we assume 1 byte was read like we requested
;we print it to standard output so we can see what was in the file
;ecx and edx remain the same as in the read call above

mov eax,4          ;invoke SYS_WRITE (kernel opcode 4 on 32 bit
systems)
mov ebx,1          ;write to stdout
int 80h           ;call the kernel

mov al,[buf]       ;move the byte we just read and wrote to al register

;if char is below 'a' or above 'z', it is not a lowercase letter
;if outside this range, we skip to the "al_is_not_lower" label

cmp al,'a'
jb al_is_not_lower
cmp al,'z'
ja al_is_not_lower

```

```
;otherwise, the al register contains a lowercase letter
;we will convert it to an uppercase letter with this formula
;and send it back to [buf]
```

```
sub al,'a'
add al,'A'
mov [buf],al
```

```
;next, we use an lseek call to go back to the offset we were at
;before we read the character in the first place
```

```
mov eax,19          ;invoke SYS_LSEEK (kernel opcode 19)
mov ebx,[fd]        ;move the opened file descriptor into ebx
mov ecx,[offset]    ;move the file cursor to this address
mov edx,0           ;whence argument (SEEK_SET)
int 80h             ;call the kernel
```

```
;and then we write the uppercase letter at the same place
;where it was a lowercase in the file!
```

```
mov eax,4           ;invoke SYS_WRITE (kernel opcode 4 on 32 bit
systems)
mov ebx,[fd]        ;move the opened file descriptor into ebx
mov ecx,buf         ;address to store the byte
mov edx,1           ;number of bytes to write
int 80h             ;call the kernel
```

```
al_is_not_lower:
```

```
inc dword[offset]   ;increment offset
```

```
jmp cat_upper
```

```
cat_upper_end:
```

```
mov ebx,[fd]        ;file number to close
mov eax,6           ;invoke SYS_CLOSE (kernel opcode 6)
int 80h             ;call the kernel
```

```
main_end:
```

```
mov eax,1           ;exit (kernel opcode 1 on 32 bit systems)
mov ebx,0           ;return 0 status on exit - 'No Errors'
int 80h             ;system call for 32-bit Linux kernel
```

```
include 'chastelib32.asm'
```

```
filename db 'openthis.txt',0 ;name of the file
fd dd 0              ;a place for the file descriptor
buf db 0             ;a buffer containing 1 byte
offset dd 0          ;offset to be used in lseek
```

```
open_error db 'error opening file',0
```

The special thing about this program is that it displays what the file originally contained on the first run before it was changed. Each time after that, the result will be different because it rewrites the file. Here are the result on my machine.

```
$ fasm main.asm
flat assembler version 1.73.30 (16384 kilobytes memory)
3 passes, 695 bytes.
$ chmod +x main
$ ./main
This is a normal file. Nothing suspicious at all.
$ ./main
THIS IS A NORMAL FILE. NOTHING SUSPICIOUS AT ALL.
```

## How it works

The open,close,read,write, and exit calls have been used before, but the key element that this program requires as the final piece is the lseek call. Here is that section again:

```
mov eax,19          ;invoke SYS_LSEEK (kernel opcode 19)
mov ebx,[fd]        ;move the opened file descriptor into ebx
mov ecx,[offset]    ;move the file cursor to this address
mov edx,0           ;whence argument (SEEK_SET)
int 80h             ;call the kernel
```

- 19 is the call number as you can see in the [Linux System Call Table](#)
- fd is the file descriptor of the open file
- offset is a variable we use to keep the address we will read next
- 0 is the whence argument, which means that we will go to offset bytes from the beginning of the file

I always used 0 as the whence argument because I like absolute addresses rather than relative. The offset variable is always incremented during this loop.

```
inc dword[offset] ;increment offset
```

However, the offset is not used until the program detects that a character is in the lowercase range from ‘a’ to ‘z’. Only then does the lseek call happen so we can go backwards to the address where the byte came from in the file. Otherwise, it is skipped but the file pointer is automatically incremented when the read call reads exactly 1 byte.

## File Open Modes and Security

But another thing that matters is that we passed 2 as the flags for the open call. This mode means that we opened the file to allow for reading and writing. The simple fact is that you can open all files this way if you wish. However, it might be recommended to use 0 instead for programs that only read files. There are two main reasons for this.

- To prevent accidentally writing to files that you did not mean to change.
- The open call will complete fail if you have read permission but not write permission yet try to open it for both reading and writing.

There are two more programs that will use the lseek call in this book. Starting in the next chapter, I will show you a program that only reads a file and yet still requires the lseek

call because it needs to go backwards sometimes. This chapter was meant to introduce `lseek` so you won't be as confused when you see it used in a larger program.

## Chapter 13 chastext

The chastext program is a basic find and replace program. Nearly every text editor has the ability to find a string and replace all occurrences of it. This feature is so useful that most of us take it for granted and don't appreciate how this works from a programming perspective.

### FASM chastext

```
;Linux 32-bit Assembly Source for chastext
;a basic text search and replace program
format ELF executable
entry main

include 'chastelib32.asm'

main:

pop eax
mov [argc],eax ;save the argument count for later

cmp dword [argc],1
ja help_skip ;if more than 1 argument is given, skip the help message
and process the other arguments

help:
mov eax,help_message
call putstring
jmp main_end
help_skip:

pop eax ;pop the next arg which is the name of the program we
are running

get_filename:
pop eax ;pop the next arg which is the name of the file we will
open

mov [filename],eax ; save the name of the file we will open to read

arg_open_file:

;Linux system call to open a file

mov ecx,0 ;open file in read only mode
mov ebx,eax ;filename should be in eax before this function was
called
mov eax,5 ;invoke SYS_OPEN (kernel opcode 5)
int 80h ;call the kernel

cmp eax,0
```

```

jns file_open_no_errors ;if eax is not negative/signed there was no
error

;Otherwise, if it was signed, then this code will display an error
message.

mov eax,open_error_message
call putstr_and_line

jmp main_end ;end the program because we failed at opening the file

file_open_no_errors:

mov [fd],eax      ;save the file descriptor number for later use

;before we just textdump or "cat" the file, we need to check for the
existence of more arguments which will modify the output

cmp dword[argc],3
jb search_skip

pop eax ;pop the next arg which is the string we are searching for
mov [string_search],eax

search_skip:

cmp dword[argc],4
jb replace_skip

pop eax ;pop the next arg which is the string we are searching for
mov [string_replace],eax

replace_skip:

;now we begin displaying the file but also searching for the search
string if it exists. We will check for these based on the number of
arguments like we did earlier

textdump:

;if only there are only 2 arguments (name of program plus input file)
;then we do a loop that ignores searching and replacing
;this loop will read one character from the file and then send it to
stdout
;until there are no more bytes to display
;but if there are above 2 arguments, we skip this loop and go to search
mode

cmp dword[argc],2 ;test arguments 2=only filename given
ja search_mode    ;but if above 2, then go to search mode because a
search string was given

;This loop is the same as the Linux 'cat' command
;or the DOS 'type' command for a single file

```

;it will read one byte and echo it to standard output until EOF

cat:

```
mov edx,1          ;number of bytes to read
mov ecx,buf        ;address to store the bytes
mov ebx,[fd]       ;move the opened file descriptor into EBX
mov eax,3          ;invoke SYS_READ (kernel opcode 3 on 32 bit systems)
int 80h            ;call the kernel
```

```
mov [count],eax
```

```
cmp eax,0
jnz file_success   ;if more than zero bytes read, proceed to display
```

```
jmp main_end ;otherwise, end the program
```

; this point is reached if file was read from successfully

file\_success:

;print the last read character to stdout by switching to write call

```
mov ebx,1          ;write to the STDOUT file
mov eax,4          ;invoke SYS_WRITE (kernel opcode 4 on 32 bit
systems)
int 80h            ;call the kernel
```

```
jmp cat
```

search\_mode:

;this is the beginning of search mode

;it handles the file by seeking and reading to search every position for the search string

;first, seek to the offset we initialized to zero

;this variable will be added to depending on actions taken

```
mov edx,0          ;whence argument (SEEK_SET)
mov ecx,[offset]   ;move the file cursor to this address
mov ebx,[fd]       ;move the opened file descriptor into EBX
mov eax,19         ;invoke SYS_LSEEK (kernel opcode 19)
int 80h            ;call the kernel
```

;obtain the length of the search string using my strlen function

```
mov eax,[string_search]
call strlen        ;get the length of the search string
```

;use the length of the string we are searching for as the number of bytes to read at this location

```
mov edx,eax        ;number of bytes to read
mov ecx,buf        ;address to store the bytes
mov ebx,[fd]       ;move the opened file descriptor into EBX
```

```

mov eax,3           ;invoke SYS_READ (kernel opcode 3)
int 80h            ;call the kernel

mov [count],eax     ;store how many bytes were read with that last read
operation

mov ebx,buf         ;move the address of bytes read into ebx
add ebx,eax         ;add number of bytes read (return value of read
function in eax)
mov byte[ebx],0     ;terminate the string with zero

cmp eax,edx ;if the number of bytes is not what we expected to read, end
this loop
jnz textdump_end

;move our two strings into the esi and edi registers for comparison
;with my custom written strcmp function

mov esi,[string_search]
mov edi,buf
call strcmp ;compare these two strings

cmp eax,0           ;test if they are the same (if eax returned zero)
jnz not_match      ;if they are not a match go to that section for printing
a character

;but if they are a match, then we either quote them
;or replace them if a replacement string is available

;but regardless of which action we do, since a match was found, let us
add this count to the offset
;so that we read from beyond this point next time the textdump loop
starts
mov eax,[count]
add [offset],eax

cmp dword[argc],4   ;if less than 4 args, no replacement exists, so we
quote the strings
jb print_quotes

;otherwise, we will print the replacement string instead of the
original!

mov eax,[string_replace]
call putstring ;print the string

jmp textdump ;restart the main loop

print_quotes:

;print quotes around matched string
mov al,'" '
call putchar

```

```

mov eax,buf
call putstring ;print the string

mov al, '"'
call putchar

jmp textdump ;restart the main loop

not_match:

;Instead of calling the putchar function in the case of no match,
;I do a system call to print 1 byte to standard output
;This is simple and also compatible with binary files we want to replace
text in.
;But it only works if the search and replace strings are of the same
length

mov eax,4          ;invoke SYS_WRITE (kernel opcode 4 on 32 bit
systems)
mov ebx,1          ;write to the STDOUT file
mov ecx,buf        ;pointer/address of string to write
mov edx,1          ;number of bytes to write == 1
int 80h            ;system call to write the message

add [offset],1 ;add 1 to the file address so we don't read this same
position again

jmp textdump

textdump_end:

;print the remaining bytes, if any, left after the main loop ended
;mov eax,buf
;call putstring

mov eax,4          ;invoke SYS_WRITE (kernel opcode 4 on 32 bit systems)
mov ebx,1          ;write to the STDOUT file
mov ecx,buf        ;pointer/address of string to write
mov edx,[count]    ;number of bytes to write == last read call result
int 80h            ;system call to write the message

main_end:

;this is the end of the program
;we close the open file and then use the exit call

;Linux system call to close a file

mov ebx,[fd]       ;file number to close
mov eax,6          ;invoke SYS_CLOSE (kernel opcode 6)
int 80h            ;call the kernel

mov eax, 1         ;invoke SYS_EXIT (kernel opcode 1)
mov ebx, 0         ;return 0 status on exit - 'No Errors'

```

int 80h

;the strlen and strcmp are named after the equivalent C functions  
;but are written from scratch by me based on their expected behavior

;The strlen function gets the length of string in eax and returns it in  
eax

;This is the same algorithm used in my putstring function

strlen:

push ebx

mov ebx,eax ;copy eax to ebx. ebx will be used as index to the  
string

strlen\_start: ;this loop finds the length of the string

cmp byte[ebx],0 ;compare byte at address ebx with 0

jz strlen\_end ;if comparison was zero, jump to loop end

inc ebx

jmp strlen\_start

strlen\_end:

sub ebx,eax ;subtract start pointer from current pointer to get  
length of string

mov eax,ebx ;copy the string length back to eax

pop ebx

ret

;strcmp compares the string at esi to the one at edi

;eax returns 0 if the strings are the same and 1 if different

;the algorithm is simple but I will explain it for those who are  
confused

;eax is initialized to zero

;a byte from each string is loaded into the al and bl registers

;the bytes are compared. if they are different, then we jump to the end

;However, if they are the same, then we check if one of them is zero

;for this purpose it doesn't matter whether we compare al or bl with  
zero

;because it is known that they are the same if the jnz did not take  
place

;if it is zero, this also jumps to the end of the function

;If neither jump took place, then we jump to the start of the loop

;but when the function finally ends bl will be subtracted from al

;this ensures that the function returns zero if the final characters are  
the same

;ebx,esi,and edi are preserved but eax is the return value

;also, the sub instruction at the end of the function also updates the  
flags

;so you can "jz" or "jnz" to a label after calling this function based  
on results

```

strcmp:

push ebx
push esi
push edi

mov eax,0

strcmp_start:

;read a byte from each string
mov al,[edi]
mov bl,[esi]
cmp al,bl
jnz strcmp_end

cmp al,0
jz strcmp_end

inc edi
inc esi

jmp strcmp_start

strcmp_end:
sub al,bl

pop edi
pop esi
pop ebx

ret

help_message db 'chastext by Chastity White Rose',0Ah,0Ah
db '"cat" a file:',0Ah,0Ah,9,'chastext file',0Ah,0Ah
db 'search for a string:',0Ah,0Ah,9,'chastext file search',0Ah,0Ah
db 'replace string:',0Ah,0Ah,9,'chastext file search replace',0Ah,0Ah
db 'Find or replace any string!',0Ah,0

open_error_message db 'error while opening file',0

offset dd 0 ;file address defaults to zero AKA beginning of file

;variables for managing arguments and files
argc rd 1
filename rd 1 ; name of the file to be opened
fd rd 1 ; file descriptor
count rd 1

string_search rd 1 ; place to hold the search string pointer
string_replace rd 1 ; place to hold the replacement string pointer

;where we will store data from the file
buf db 0xA4 dup 0

```

To understand exactly how powerful this program is, I have prepared a text file and a script which can transform it into something quite different! Here is the text file of a famous tongue twister about someone who sells seashells on the seashore!

## **seashell.txt**

```
She sells seashells by the seashore,  
The shells she sells are seashells, I'm sure.  
So if she sells seashells on the seashore,  
Then I'm sure she sells seashore shells.
```

And here is a script that runs the chastext program multiple times and changes the text in several stages.

## **chastext test script**

```
#!/bin/sh  
./main seashell.txt  
echo  
./main seashell.txt 'She' 'Chastity' > 0.txt  
./main 0.txt 'sells' 'writes' > 1.txt  
./main 1.txt 'seashells' 'assembly books' > 2.txt  
./main 2.txt 'seashore' 'computer' > 3.txt  
./main 3.txt 'shells' 'books' > 4.txt  
./main 4.txt 'by the' 'on her' > 5.txt  
./main 5.txt
```

If you assembled the program under the name “main” and then run this script like:

```
sh textdemo.sh
```

The output will be as follows:

```
She sells seashells by the seashore,  
The shells she sells are seashells, I'm sure.  
So if she sells seashells on the seashore,  
Then I'm sure she sells seashore shells.
```

```
Chastity writes assembly books on her computer,  
The books she writes are assembly books, I'm sure.  
So if she writes assembly books on the computer,  
Then I'm sure she writes computer books.
```

The idea behind the way I have written chastext is that it makes use of file redirection so that we can make changes in stages one command and file at a time. If anything were to go wrong, we still have the original file unmodified.

The seashell poem is just one example, but I suppose this could also be used to change lyrics for song parodies. This program is as capable of transforming text as your average text editor with a find and replace feature.

There is a program named “sed” on Linux that can also do find and replace on text files, but I just found it to be a fun exercise to write my own. Also, my program is faster and smaller because it has less work to do than the same thing written in C. But if you are

curious, here is the same thing done using sed.

```
#!/bin/sh
cat seashell.txt
cp seashell.txt 0.txt
echo
sed 's/She/Chastity/g' -i 0.txt
sed 's/sells/writes/g' -i 0.txt
sed 's/seashells/assembly books/g' -i 0.txt
sed 's/seashore/computer/g' -i 0.txt
sed 's/shells/books/g' -i 0.txt
sed 's/by the/on her/g' -i 0.txt
cat 0.txt
```

## How does chastext work?

chastext uses almost every concept I have taught in this book so far. It uses all 6 system calls in my tables and it accepts user input from command line arguments to control which file is opened and what changes are made.

It will display a help message if you don't add any arguments.

If you provide one argument, it uses that string as a filename to open and then displays all of it using the “cat” loop.

If you provide a second argument after it, then it will search the file for this string and will display the whole file with the string you gave in quotes wherever it is found.

But the real magic is when you give it a third argument and it will replace the search string with whatever your third argument is.

Although it does not write any data back to the original file, it still requires lseek because it has to increment the address 1 byte at a time. If I were to search for my name: “chastity”, it would read 8 bytes from the file and then see if all of those bytes are equal to “chastity”. Regardless of whether there is a match or not, the act of performing the read call would still increment the file pointer by 8 bytes.

But in the case where we don't have a match, we then need to go backwards to read the next position 1 byte after the last address. Without lseek, we would be permanently stuck beyond this point.

I tried writing this program without lseek several times but then it kept missing strings that I intended to replace. It turns out that the only other way to do it would be to load the entire file into memory because that would also allow random access moving forward or backward. Seeking in a file allows us to process information larger than the available RAM. If I had a 16 gigabyte file and wanted to replace all occurrences of one string with another, I am confident my program could do it.

## Fun Fact About Chastext

The `strlen` and `strcmp` functions were written in Assembly specifically for this program. While writing `chastext`, it was the first time I needed to obtain the length of the search string first and then I needed to read that many bytes from the input file for comparison with `strcmp`.

I could have named these functions anything I wanted, but because functions of these same names are part of the C standard library, my readers with a background in C programming would have no trouble understanding what they were meant to do.

## Chapter 14: Customizing the Linux PATH

I have covered a lot of Assembly tricks and because I have been using Linux for so long, I sometimes forget that not everybody is as familiar with the command line as I am. I hope that I am not going too fast and assuming too much about the capabilities of my readers.

Yes, this book is not exactly for beginners to Linux. I can't answer questions about the Linux Kernel. Not even Linus Torvalds himself could explain it all because the project has grown far beyond him with many contributors.

But what is accessible to users like you and I is that around the Linux kernel, huge operating systems are built using hundreds of command line utilities. Some of the more common tools are:

- `ls`: list files in a directory
- `cp`: copy files
- `mv` move files
- `cat`: display text files
- `sed`: transform text such as search and replace
- `rm`: delete files

There are better books and free online resources for explaining these commands to Linux beginners. But the good news is that most of us don't require memorizing all the commands. Most people can learn them as they go and Google search what they don't know.

For example, in chapter 6, I described the ten basic Intel instructions (counting all jumps as one category of instructions). However, Intel CPUs have over 1500 instructions.

Similarly, there are over 460 Linux system calls but I have only ever used 6 of them for every program I needed to write.

I have purposefully limited using any more Intel instructions or system calls than strictly necessary to avoid any confusion.

I bring this up to show you that even if you find the information a bit hard to follow, I have been trying to make it easy.

There is one key piece of advice that I do want to share in this chapter that is not about Assembly programming specifically but is about Linux usage in general. This is also something that not everyone will tell you but is a lesson I have learned from experience.

To have a consistent development environment, or even a gaming environment on Linux, it is often helpful to know where your executables are located and also to install your own programs to a common location.

Sometimes, Linux users forget what software they have installed because of the

convenience of package managers that install them to places already part of the PATH variable. However, you will need to know what the PATH is and how to change it for whatever you need at the time.

## What is the PATH?

The PATH is the name of an environment variable that is a list of directories (or folders to use terminology from Windows). Directories that are in this list are searched whenever you type a command.

But the good news is that you can create your own directory and add programs to it. I will teach you how with an example of my own.

The “~” directory is a shortcut to the home directory of the user currently logged in. For example, on my system, the command

```
echo ~
```

Will return

```
/home/chastity
```

With this information, here is what I chose to use as my executable directory. I created a directory named “.bin” in my home like this:

```
mkdir ~/.bin
```

And then I set the path to it like this:

```
PATH=~/.bin/
```

What this does is set my PATH to only this directory. All other commands on the system are unavailable when I do this. This works great for Assembly programs from this book because they depend on the Linux kernel but don’t depend on shared libraries.

However, if you want to keep access to all other commands but you want to also have a special .bin directory, then you need to edit the “~/.bashrc” in your home directory.

If you have a text editor like nano for example, you can edit it with:

```
nano ~/.bashrc
```

And add this line to it.

```
export PATH=$PATH:~/.bin/
```

What this command does is cause the “~/.bin” directory to be added to whatever PATH was already defined by your system. The “.bashrc” file is a config file for the bash shell, which is the most popular but there are other shells that some distros use as the default. If you use a different shell, see the documentation for it. In any case, you should still set a custom directory for programs you write to keep them separate from your distribution’s package manager.

## Why did you choose .bin instead of bin?

Files and directories starting with a dot on Linux are hidden files. I don't always want certain things to show in the file managers when I am looking through hundreds of directories on my system, which has become quite complex BTW. I have just about every compiler and assembler for languages I haven't learned but might someday.

But the important thing is that you can choose any name for your directory. When you write programs, you should remember where your directory is and then choose a suitable name for it.

For example, if you assembled the chastext program from chapter 12 and the executable was currently named "main", you would copy and rename it with the "cp" command like this:

```
cp main ~/.bin/chastext
```

Once you have an executable file in a directory that is part of your path, you can run it from anywhere in the system. Naturally, the FASM and NASM assemblers are part of my path on Debian Linux because the "apt" package manager put them in the system wide "/usr/bin" directory. However, my own programs I keep in the "~/.bin" directory I have described. This prevents the need for entering my root password and accidentally making a mistake that makes my system unbootable. I always keep my stuff separate from the locations chosen by the package maintainers for my own safety and because I don't want to brick the machine I use for all my writing.

## A Note to Windows Users

If you are a current or former Windows user, you may remember that Windows also has a PATH variable deep in the control panel settings. Personally I have forgotten how to access it because it has been awhile since I used Windows. However, it gives you a warning that you can cause problems if you carelessly mess with the PATH.

But the method I have described in this chapter is completely safe because it does not mess with your system directories and is completely reversible. Either your temporary path goes away when you exit out of your terminal or you can change it to automatically be what you want in the ".bashrc" file.

## Why is the PATH so important?

The reason I dedicated a chapter to explaining the PATH variable is precisely because Linux commands need to be accessible from wherever you are. When I am moving around in my repositories and compiling C programs or assembling programs with FASM or NASM, I can trust them all to be in my path.

Trust me, this will become more obvious as this book continues because I will be including all 4 of my core programs. Two of these you have already seen and two are still

yet to come.

## The Core Four Assembly Programs

| <b>Name</b> | <b>Function</b>                    |
|-------------|------------------------------------|
| chastack    | RPN calculator with software stack |
| chastext    | search and replace for text files  |
| chastecmp   | compare two files in hexadecimal   |
| chastehex   | seek and edit files in hexadecimal |

## Chapter 15: chastecmp

In this chapter, I will show you the source code of a file comparison program. This program is meant to find which bytes are different between two files that are similar but contain a few differences.

I will use text files for my examples in this chapter, but the program actually does a binary file comparison and displays the different bytes in hexadecimal because it is a universally understood shorthand for binary that most C and Assembly programmers are already familiar with.

First, here is the source code of chastecmp, which is the short name for “Chastity’s Comparison tool”. The name is also meant to refer to the “cmp” instruction, which is used a lot more in this program because it is essential.

### FASM chastecmp source

```
;Linux 32-bit Assembly Source for chastecmp
format ELF executable

main:

;radix will be 16 because this whole program is about hexadecimal
mov dword[radix],16 ; can choose radix for integer input/output!
mov dword[int_width],1

pop eax ;get the number of arguments
dec eax ;subtract 1 because we will ignore the name of the program
pop ebx ;pop program name into a register to delete it from stack

cmp eax,2 ;do we have two arguments to be used as filenames?
jb help
mov dword[offset],0 ;assume the offset is 0,beginning of file
jmp arg_open_file_1

help:
mov eax,help_message
call putstring
jmp main_end

arg_open_file_1:
pop eax
mov [filename1],eax ; save the name of the file we will open to read
call putstring ;print the name of the file we will try opening

mov ecx,0 ;open file in read mode
mov ebx,eax ;move filename for system call
mov eax,5 ;invoke SYS_OPEN (kernel opcode 5)
int 80h ;call the kernel
```

```

cmp eax,0
js file_error_display ;end program if the file can't be opened
mov [fd1],eax ; save the file descriptor number for later use
mov eax,file_open
call putstr_and_line

arg_open_file_2:
pop eax
mov [filename2],eax ; save the name of the file we will open to read

call putstring ;print the name of the file we will try opening

mov ecx,0 ;open file in read mode
mov ebx,eax ;move filename for system call
mov eax,5 ;invoke SYS_OPEN (kernel opcode 5)
int 80h ;call the kernel

cmp eax,0
js file_error_display ;end program if the file can't be opened
mov [fd2],eax ; save the file descriptor number for later use
mov eax,file_open
call putstr_and_line

files_compare:

file_1_read_one_byte:
mov edx,1 ;number of bytes to read
mov ecx,buf1 ;address to store the bytes
mov ebx,[fd1] ;move the opened file descriptor into EBX
mov eax,3 ;invoke SYS_READ (kernel opcode 3)
int 80h ;call the kernel

;eax will have the number of byte read after system call
mov [count1],eax ;we save the number of byte read for later
cmp eax,0
jnz file_2_read_one_byte ;unless zero bytes were read, proceed to read
from next file

mov eax,[filename1]
call putstring
mov eax,end_of_file_string
call putstr_and_line

;Even if we have reached the end of the first file,
;we still proceed to read a byte from the second file
;to see if it also ends at the same address

file_2_read_one_byte:
mov edx,1 ;number of byte to read
mov ecx,buf2 ;address to store the bytes
mov ebx,[fd2] ;move the opened file descriptor into EBX
mov eax,3 ;invoke SYS_READ (kernel opcode 3)
int 80h ;call the kernel

```

```

;eax will have the number of bytes read after system call
mov [count2],eax ;we save the number of bytes read for later
cmp eax,0
jnz check_both_bytes ;unless zero bytes were read, proceed to compare
bytes from both files

mov eax,[filename2]
call putstring
mov eax,end_of_file_string
call putstr_and_line

jmp main_end ;we have reach end of one file and should end program

check_both_bytes:

;we add the number of bytes read from both files
mov eax,[count1]
add eax,[count2]
cmp eax,2
jnz main_end

compare_bytes:

mov al,[buf1]
mov bl,[buf2]

;compare the two bytes and skip printing them if they are the same
cmp al,bl
jz bytes_are_same

;print the address and the bytes at that address
mov eax,[offset]
mov dword[int_width],8
call putint_and_space
mov dword[int_width],2
mov eax,0
mov al,[buf1]
call putint_and_space
mov al,[buf2]
call putint_and_line

bytes_are_same:

inc dword[offset]

jmp files_compare

file_error_display:

mov eax,file_error
call putstr_and_line

main_end:

```

```

;this is the end of the program
;we close the open files and then use the exit call

mov ebx,[fd1] ;file number to close
mov eax,6     ;invoke SYS_CLOSE (kernel opcode 6)
int 80h       ;call the kernel

mov ebx,[fd2] ;file number to close
mov eax,6     ;invoke SYS_CLOSE (kernel opcode 6)
int 80h       ;call the kernel

mov eax, 1    ; invoke SYS_EXIT (kernel opcode 1)
mov ebx, 0    ; return 0 status on exit - 'No Errors'
int 80h

include 'chastelib32.asm'

;variables for displaying information
help_message db 'chastecmp by Chastity White Rose',0Ah,0Ah
db 9,'chastecmp file1 file2',0Ah,0Ah
db 'Differing bytes are shown in hexadecimal',0Ah
db 'until the EOF has been reached.',0Ah,0

file_open db ' opened',0
file_error db ' error',0
end_of_file_string db ' EOF',0

db 23 dup 0 ;fill with extra space to match 1024 executable size

;variables for managing files
filename1 dd ? ;name of the file to be opened
filename2 dd ? ;name of the file to be opened
fd1 dd ?      ;file descriptor 1
fd2 dd ?      ;file descriptor 2
buf1 db ?     ;store byte from file 1 here
buf2 db ?     ;store byte from file 2 here
count1 dd ?
count2 dd ?
offset dd ?

```

## How to use chastecmp

Using the chastecmp program requires two filenames to be passed as command-line arguments. Although you can use any files you have, it makes sense to use a simple example with text files because they are so easy to create with the echo command.

Run these commands to create the two files.

```

echo "chandler is my birth name" > file1.txt
echo "chastity is my trans name" > file2.txt

```

Now that the files exist

```
./main file1.txt file2.txt
```

If you have created these files and run the `chastecmp` program on them, you will see this result:

```
file1.txt opened
file2.txt opened
00000003 6E 73
00000004 64 74
00000005 6C 69
00000006 65 74
00000007 72 79
0000000F 62 74
00000010 69 72
00000011 72 61
00000012 74 6E
00000013 68 73
file1.txt EOF
file2.txt EOF
```

## How does `chastecmp` work?

This program is much simpler than `chastack` or `chastext`, but it is close to 180 lines and still has some logic to follow. First thing it does is check to see how many command-line arguments were passed to the program. Since the name of the program always counts as 1, we subtract from this number and also pop the next argument into `ebx` just to get rid of it. The actual register used doesn't matter in this case as long as it is not `eax`, which holds the number of arguments.

The `eax` register is compared with 2. If this number is below 2, then there are not enough arguments to continue the program, and it will end. Otherwise, it will proceed to use the open call with both filenames and assume these files exist. If they do not exist, it will print the filename and then say error.

If both files are opened, it will keep reading 1 byte from each file descriptor and store each in its own buffer of 1 byte. If the two bytes are the same, they will be ignored. However, if they are different, the address and the values of both bytes at that address will be displayed.

The variable “offset” is used to keep track of which address we are at in both files, but it isn't used to `lseek` in this program because we are going from beginning to end.

If at any time the read system call returns 0, a message is displayed with the filename and EOF to tell the user that the end of that file has been reached.

In the example I just used, both files are the same length of 26 bytes and will reach the end at the same time.

## But why should I care?

The average person probably does not know why it matters to see the hexadecimal differences between two files. I know it seems silly, especially for small text files as I

used in this chapter's examples. However, I can give two examples of times I have used this information.

The first example is relevant to Chapter 2, where I presented the header file "chaste-elf-32.nasm" which can be included to make a loadable program using the NASM assembler.

I read the specification document for ELF files to describe what the fields were named and what the values meant. However, this informational alone was not enough for me to successfully create the custom ELF header. I had to create ELF executable files with FASM because it has this feature built in. By creating slightly different programs, I was able to compare the binary differences in the different source files fed to FASM. The chastecmp program was extremely helpful to me as I used it hundreds of times in reverse engineering the ELF format.

One of my discoveries was that when the size of a program increased, either by adding more code or adding more data statements, there was a number in the header that also increased. As it turns out, the memory size of the file increased even when data reservation keywords (such as rb, rw, rd, and rq) were used, even when the size of the file itself didn't.

The specification could tell me a lot, but without the example ELF headers FASM was already creating, I would not have been able to create dynamic headers to match programs written in FASM. I probably spent 12 hours on that project, but at least I can assembler any of my programs with NASM if I make the necessary syntax changes.

But perhaps a more fun example, and also the reason I got started with programming, was that I used a file comparison tool to cheat at a Norse mythology game years ago. The game was called Castle of the Winds, and it ran on Windows 3.1, 98, and even XP.

One of the features of that specific game was that it let you save the game at any time. I remember that I had 5 mana points. I saved the first file and then cast the magic arrow spell to spend one point. I then saved a second file and ran the Windows "fc" command to compare the two files in binary mode.

```
fc /b 1.cwg 2.cwg
```

It told me the address of the byte that had changed from 5 to 4. I then opened this in a hex editor named XVI32 and changed this byte to different values.

In time, I was able to not only change my mana points but also hit points and experience points to make myself invincible in that game.

I didn't really know much about hexadecimal at this point, but by trial and error, I accidentally started understanding it. It was this experience of cheating in a video game that led me to learn about binary and hexadecimal number systems originally.

I had seen for the first time that an understanding of computer arithmetic could allow me to break the rules and do things in a video game that the developer could not predict or prevent me from doing. In those days, I learned to do the same with many video games

and had many fun adventures.

In modern times, developers have gotten smarter and have put measures in place to prevent this form of cheating. Most notably, more games are multiplayer and read data from a server that stores the game data, where no user can hack it.

But you have to understand that back in the 90s, nearly every single player game could be hacked that stored its data locally and didn't connect to the internet. I have had people criticize my habit of cheating in single-player games and say that it ruins the experience of the game.

But what they don't understand is that I didn't care about the video game I was hacking, because Arithmetic had become my favorite game. My love of math was so great that I learned computer programming and had more fun writing programs in BASIC, C, and Assembly than I did playing video games in the first place.

I can't hack most modern games with these tricks, but I have found the art of computer programming, which is much more satisfying than any video game I have played in my life.

In summary, the `chastecmp` program does the same thing as the `fc /b` command from DOS and Windows did. When I switched to Linux as my primary operating system, I wrote my own file comparison tool to always keep the fond memories of my childhood with me.

## Chapter 16: chastehex

In this chapter, I will finally show you the Linux version of chastehex! If you read my DOS edition of Assembly Arithmetic Algorithms, you will already be familiar with it. But make no mistake in thinking the program is written the same as it was in DOS. There are some differences in how it achieves the same behavior with less code.

First, command line arguments are just popped directly from the stack in Linux. You have already seen this from the other programs in this book. In DOS it was not so simple because they were at a special address and I had to process them manually which made the code harder to read. For this reason, the Linux version you are about to read is cleaner and less confusing.

The second thing that makes the Linux version easier to use is that the `strint` function is the same one you have already seen used in chapter 10 of this book. The DOS version had to have a special version of `strint` because the DOS system call for `lseek` had the lower and upper 16 bits split into different registers.

Aside from these technical details caused by the limitations that DOS has, I have also spent time cleaning up and commenting the code for the Linux chastehex program. Despite its optimizations, it is still the largest of all the programs in this book because it has 3 modes of operation. I shall refer to these modes as “dump”, “peek”, and “poke”.

For now, glance over the source code and see if you are clever enough to guess what it does. Either way, I will explain the 3 modes and when each of them are useful!

### FASM chastehex

```
;Linux 32-bit Assembly Source for chastehex  
;a special tool originally written in C  
format ELF executable
```

```
main:
```

```
;radix will be 16 because this whole program is about hexadecimal  
mov dword [radix],16 ; can choose radix for integer input/output!
```

```
pop eax ;get the number of arguments  
dec eax ;subtract 1 because we will ignore the name of the program  
pop ebx ;pop program name into a register to delete it from stack  
mov [argc],eax ;save the argument count for later
```

```
;before we try to get the first argument as a filename, we must check if  
it exists  
cmp dword [argc],0  
jnz arg_open_file
```

```
help:  
mov eax,help_message
```

```

call putstring
jmp main_end

arg_open_file:

pop eax
dec dword [argc]
mov [filename],eax ; save the name of the file we will open to read
call putstr_and_line

;Linux system call to open a file
mov ecx,2 ;open file in read and write mode
mov ebx,eax ;filename should be in eax before this function was called
mov eax,5 ;invoke SYS_OPEN (kernel opcode 5)
int 80h ;call the kernel

cmp eax,0
jns file_open_no_errors ;if eax is not negative/signed there was no
error

;Otherwise, if it was signed, then this code will display an error
message.
neg eax
call putint_and_space
mov eax,open_error_message
call putstr_and_line

jmp main_end ;end the program because we failed at opening the file

file_open_no_errors:

mov [fd],eax ; save the file descriptor number for later use
mov dword [offset],0 ;assume the offset is 0,beginning of file

;check next arg
cmp dword [argc],0 ;if there are no more args after filename, just
hexdump it
jnz seek_offset ;otherwise use next argument as an offset for lseek

hexdump:

mov edx,0x10 ;number of bytes to read
mov ecx,buf ;address to store the bytes
mov ebx,[fd] ;move the opened file descriptor into EBX
mov eax,3 ;invoke SYS_READ (kernel opcode 3)
int 80h ;call the kernel

mov [count],eax

cmp eax,0
jnz file_success ;if more than zero bytes read, proceed to display

;otherwise, display EOF to indicate we have reached the end of file
mov eax,end_of_file_string

```

```

call putstr_and_line

jmp main_end

;this point is reached if file was read from successfully
file_success:
call print_bytes_row
jmp hexdump

;convert argument with strint and lseek to that offset
seek_offset:

pop eax ;pop the argument into eax and process it as a hex number
dec dword [argc]
call strint

;use the hex number as an address to seek to in the file
mov edx,0 ;whence argument (SEEK_SET)
mov ecx,eax ;move the file cursor to this address
mov ebx,[fd] ;move the opened file descriptor into EBX
mov eax,19 ;invoke SYS_LSEEK (kernel opcode 19)
int 80h ;call the kernel

mov [offset],eax ;move the new offset

;check the number of args still remaining
cmp dword [argc],0
jnz next_arg_write ; if there are still arguments, skip this read
section and enter writing mode

read_one_byte:
mov edx,1 ;number of bytes to read
mov ecx,buf ;address to store the bytes
mov ebx,[fd] ;move the opened file descriptor into EBX
mov eax,3 ;invoke SYS_READ (kernel opcode 3)
int 80h ;call the kernel

;eax will have the number of bytes read after system call
cmp eax,1
jz print_byte_read ;if exactly 1 byte was read, proceed to print info

call show_eof

jmp main_end ;go to end of program

;print the address and the byte at that address
print_byte_read:
call print_byte_info

;this section interprets the rest of the args as bytes to write
next_arg_write:
cmp dword [argc],0
jz main_end

```

```

pop eax
dec dword [argc]
call strint ;try to convert string to a hex number

;write that number as a byte value to the file

mov [buf],al

mov edx,1 ;write 1 byte
mov ecx,buf ;pointer/address of byte to write
mov ebx,[fd] ;write to this file descriptor
mov eax,4 ;invoke SYS_WRITE (kernel opcode 4 on 32 bit systems)
int 80h ;system call to write the message

call print_byte_info
inc dword [offset]

jmp next_arg_write

main_end:

;this is the end of the program
;we close the open file and then use the exit call

;Linux system call to close a file

mov ebx,[fd] ;file number to close
mov eax,6 ;invoke SYS_CLOSE (kernel opcode 6)
int 80h ;call the kernel

mov eax, 1 ;invoke SYS_EXIT (kernel opcode 1)
mov ebx, 0 ;return 0 status on exit - 'No Errors'
int 80h

include 'chastelib32.asm'

;a function just for printing the address
;this saves space because all 3 modes use this
print_offset:
mov eax,[offset]
mov dword [int_width],8
call putint_and_space
ret

;this function prints a row of hex bytes
;each row is 16 bytes
print_bytes_row:
call print_offset

mov ebx,buf
mov ecx,[count]
add [offset],ecx
next_byte:
mov eax,0

```

```

mov al,[ebx]
mov dword [int_width],2
call putint_and_space

inc ebx
dec ecx
cmp ecx,0
jnz next_byte

mov ecx,[count]
pad_spaces:
cmp ecx,0x10
jz pad_spaces_end
mov eax,space_three
call putstring
inc ecx
jmp pad_spaces
pad_spaces_end:

;optionally, print chars after hex bytes
call print_bytes_row_text
call putline

ret

;I define a string of 3 spaces as filler when less than 16 bytes are
read
;This makes the text section on the right properly lined up.
space_three db '   ',0

;This function prints the text equivalent of the bytes on the last row
printed.
;It reads how many bytes were read in the last read operation.
;If less than 16 bytes were read, it prints spaces as filler so that
;text can still be printed lined up with all the other rows
;even if less than 16 bytes exist in the current row.
;This situation sometimes happens when we get near the end of the file.
;It also replaces characters that can't be printed with periods -> .

print_bytes_row_text:
mov ebx,buf
mov ecx,[count]
next_char:
mov eax,0
mov al,[ebx]

;if char is below '0' or above '9', it is outside the range of these and
is not a digit
cmp al,0x20
jb not_printable
cmp al,0x7E
ja not_printable

printable:

```

```

; if char is in printable range, keep as is and proceed to next index
jmp next_index

not_printable:
mov al, '.' ; otherwise replace with placeholder value

next_index:
mov [ebx], al
inc ebx
dec ecx
cmp ecx, 0
jnz next_char
mov [ebx], byte 0 ; make sure string is zero terminated

mov eax, buf
call putstring
ret

; function to display EOF with address
show_eof:

call print_offset
mov eax, end_of_file_string
call putstr_and_line

ret

; print the address and the byte at that address
print_byte_info:
call print_offset
mov eax, 0
mov al, [buf]
mov dword [int_width], 2
call putint_and_line

ret

end_of_file_string db 'EOF', 0

help_message db 'chastehex by Chastity White Rose', 0Ah
db 'hexdump a file:', 0Ah, 9, 'chastehex file', 0Ah
db 'read a byte:', 0Ah, 9, 'chastehex file offset', 0Ah
db 'write a byte:', 0Ah, 9, 'chastehex file offset value', 0Ah
db 'The file must exist', 0Ah, 0

; variables for managing arguments and files
argc dd 0
filename dd 0 ; name of the file to be opened
fd dd 0 ; file descriptor
count dd 0
offset dd 0
open_error_message db 'error opening file', 0

; where we will store data from the file

```

```
;17 bytes because 16 bytes read per row plus the terminating 0
;used for the text printing function
buf db 17 dup '?'
```

```
db 0x31 dup 0 ;fill with extra space to match 1280 executable size
```

## dump mode

To describe the 3 modes of chastehex, it is good to use a familiar example. Therefore, I will be using the seashell.txt file from chapter 13.

Assemble the chastehex program from the source and place it in your path as I described in chapter 14.

Run the chastehex program while passing the name of the file as the first argument.

```
chastehex seashell.txt
```

Doing so will execute chastehex in dump mode where it dumps the whole file to the terminal like this:

```
seashell.txt
00000000 53 68 65 20 73 65 6C 6C 73 20 73 65 61 73 68 65 She sells
seashe
00000010 6C 6C 73 20 62 79 20 74 68 65 20 73 65 61 73 68 lls by the
seash
00000020 6F 72 65 2C 0A 54 68 65 20 73 68 65 6C 6C 73 20 ore, .The shells
00000030 73 68 65 20 73 65 6C 6C 73 20 61 72 65 20 73 65 she sells are
se
00000040 61 73 68 65 6C 6C 73 2C 20 49 27 6D 20 73 75 72 ashells, I'm
sur
00000050 65 2E 0A 53 6F 20 69 66 20 73 68 65 20 73 65 6C e..So if she
sel
00000060 6C 73 20 73 65 61 73 68 65 6C 6C 73 20 6F 6E 20 ls seashells on
00000070 74 68 65 20 73 65 61 73 68 6F 72 65 2C 0A 54 68 the
seashore, .Th
00000080 65 6E 20 49 27 6D 20 73 75 72 65 20 73 68 65 20 en I'm sure she
00000090 73 65 6C 6C 73 20 73 65 61 73 68 6F 72 65 20 73 sells seashore
S
000000A0 68 65 6C 6C 73 2E 0A hells..
EOF
```

Notice that the addresses are displayed on the left side, hex view is in the center, and the text approximation is on the right side. This is because I wrote chastehex to follow the convention of other hex editors that do things this way.

## peek mode

Using peek mode is very easy. We simply add an address in hexadecimal of the byte we want to read.

For example, if we want to read the address of the first newline character, we can see from the previous output that it is four bytes after the beginning of the line at 00000020.

In that case, we know it is address 24 and can run this command.

```
chastehex seashell.txt 24
```

Then chastehex will output the address we chose and the value there.

```
seashell.txt  
00000024 0A
```

But if we tried to use an address past the end of the file

```
chastehex seashell.txt BAD
```

It would show EOF, which means End Of File.

```
seashell.txt  
00000BAD EOF
```

## poke mode

But chastehex can do a lot more than just read bytes. We can also edit them anywhere we like, including after the EOF. For example, we will write some bytes starting at address C0, which does not currently exist in the file.

```
chastehex seashell.txt C0 48 65 6C 6C 6F 20 57 6F 72 6C 64 21 0A
```

When you run the above command, you will see this output:

```
seashell.txt  
000000C0 48  
000000C1 65  
000000C2 6C  
000000C3 6C  
000000C4 6F  
000000C5 20  
000000C6 57  
000000C7 6F  
000000C8 72  
000000C9 6C  
000000CA 64  
000000CB 21  
000000CC 0A
```

Now you know which bytes were written to exactly which addresses starting at address C0 hex.

## But what does it all mean?

Just run the chastehex program on the file one more time using only the filename as you did with dump mode before.

```
chastehex seashell.txt
```

And you will see what has changed in the file!

```
seashell.txt  
00000000 53 68 65 20 73 65 6C 6C 73 20 73 65 61 73 68 65 She sells  
seashe  
00000010 6C 6C 73 20 62 79 20 74 68 65 20 73 65 61 73 68 lls by the
```

```

seash
00000020 6F 72 65 2C 0A 54 68 65 20 73 68 65 6C 6C 73 20 ore,.The shells
00000030 73 68 65 20 73 65 6C 6C 73 20 61 72 65 20 73 65 she sells are
se
00000040 61 73 68 65 6C 6C 73 2C 20 49 27 6D 20 73 75 72 ashells, I'm
sur
00000050 65 2E 0A 53 6F 20 69 66 20 73 68 65 20 73 65 6C e..So if she
sel
00000060 6C 73 20 73 65 61 73 68 65 6C 6C 73 20 6F 6E 20 ls seashells on
00000070 74 68 65 20 73 65 61 73 68 6F 72 65 2C 0A 54 68 the
seashore,.Th
00000080 65 6E 20 49 27 6D 20 73 75 72 65 20 73 68 65 20 en I'm sure she
00000090 73 65 6C 6C 73 20 73 65 61 73 68 6F 72 65 20 73 sells seashore
s
000000A0 68 65 6C 6C 73 2E 0A 00 00 00 00 00 00 00 00
hells.....
000000B0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 .....
000000C0 48 65 6C 6C 6F 20 57 6F 72 6C 64 21 0A          Hello World!.
EOF

```

As you can see, that command actually wrote “Hello World!” to a location in the file that didn’t exist before.

Chastehex is a magnificent program because it read or write to any file at any location. Of course the example used in this chapter was only a basic introduction. Making more advanced usage of this requires knowing which addresses you want to modify and the purposes of those bytes. You would normally use chastehex on an executable or save file of a game to modify what you like.

In any case, the example I used of modifying a text file shows you exactly what this program does. It uses all the 6 system calls I described in this book. It is also the only program that does entirely different operations depending on how many arguments there are.

But even a tool like this is only as useful as your understanding of arithmetic and how numbers are actually represented in a specific file format.

Now that all of the large programs have been described in these first 16 chapters, I will use the next chapters to describe some more advanced knowledge on how computers really process numbers. At this point, you have all the information on what the instructions and system calls mean. I have also provided you with many functions which can display numbers and text.

Be sure to look at the [chastelib32.asm](#) file from chapter 10 whenever you forget the way these functions work. From this point, I will be describing the Arithmetic of the Algorithms more than the Assembly language. Assembly is just the programming language we use, but the reason we use it, is the Arithmetic!

## Chapter 17: Integer Sequences and Their Application in Learning

To be a programmer in any language, a person needs more than information. There must be a motivation for something you want to make. The challenge is that when you are a beginner, it can be easy to get discouraged because you won't be making anything big or impressive to other people at the start. All you have to work with in most programming languages is displaying text and numbers.

Later on, you can learn to use third-party libraries or native APIs for your operating system. However, what I have always disliked is that the internals of how they work are hidden or obfuscated so that you don't know how they work.

One of the best ways to get started programming in any language is by writing small programs to print integer sequences. In this chapter, I will be sharing some of my favorite sequences.

Each integer sequence has a page on the Online Encyclopedia of Integer Sequences. These pages can be used as a useful reference for those you are not familiar with.

- [Fibonacci numbers](#)
- [Powers of 2](#)
- [Prime Numbers](#)
- [Factorials](#)

If you have the ebook edition of this book, you will be able to click the links above and learn more about these sequences. Either way, I will show you the code that makes printing these sequences easy, even in Assembly Language.

### How to use these examples

My suggestion is that you download the examples in this chapter from my Github repository rather than trying to type them by hand or copy paste them. That way you can assemble them with FASM and run them on your Linux machine and play around with them.

<https://github.com/chastitywhiterose/Assembly/tree/main/fasm/linux/AAA-Linux-Book-Examples>

The programs in this chapter can produce long lists of numbers and so I can't include all the output in this book. You will have to run them to get the full picture of how magnificent they are! For example, try changing the length in the primes program from 1000 to 1000000. I tested it on my machine and it produced all the prime numbers less than a million very fast!

## Fibonacci numbers

format ELF executable

main:

```
mov dword [radix],10
mov dword [int_width],1
```

```
mov eax,0
mov ebx,1
```

```
Fibonacci:
call putint
call putline
add eax,ebx
push eax
mov eax,ebx
pop ebx
cmp eax,1000
jb Fibonacci
```

```
mov eax,1
mov ebx,0
int 0x80
```

```
include 'chastelib32.asm'
```

When executed, that program will output the following sequence

```
0
1
1
2
3
5
8
13
21
34
55
89
144
233
377
610
987
```

Just by looking at it, hopefully you see the pattern. Each number is the sum of the previous two numbers. The loop in this program needed to swap the numbers in `eax` and `ebx` each time before the next add. Technically, there are two other ways I could have achieved it. I could have used `ecx` as a temporary storage. I also could have used the `xchg` instruction, which does the same thing, but the stack provided me a convenient way of doing it. We only needed to save the `eax` register before it was overwritten with `ebx`, then

we popped into ebx the pushed eax from earlier. None of these methods is more correct than any other, but I tend to favor simplicity and therefore am limiting the type of instructions I use. I also think that using a third register or even another memory location is acceptable for something like this. Still, since the stack is already used for function calls and is an expected part of Assembly, there is no reason not to use it, especially when we only need to save one register.

## Powers of 2

format ELF executable

main:

```
mov dword [radix],10
mov dword [int_width],1
```

```
mov ecx,0
```

```
mov [array],byte 1
```

powers\_of\_two:

```
;this section prints the digits
```

```
mov ebx,[length]
```

```
array_print:
```

```
dec ebx
```

```
mov eax,0
```

```
mov al,[array+ebx]
```

```
call putint
```

```
cmp ebx,0
```

```
jnz array_print
```

```
call putline
```

```
;this section adds the digits
```

```
mov dl,0
```

```
mov ebx,0
```

```
array_add:
```

```
mov eax,0
```

```
mov al,[array+ebx]
```

```
add al,al
```

```
add al,dl
```

```
mov dl,0
```

```
cmp al,10
```

```
jb less_than_ten
```

```
sub al,10
```

```
mov dl,1
```

```
less_than_ten:
```

```
mov [array+ebx],al
```

```
inc ebx
```

```
cmp ebx,[length]
```

```

jnz array_add

cmp dl,0
jz carry_is_zero

mov [array+ebx],1
inc [length]

carry_is_zero:

;keeps track of how many times the loop has run
add ecx,1
cmp ecx,64
jna powers_of_two

mov eax,1
mov ebx,0
int 0x80

include 'chastelib32.asm'

length dd 1
array db 0x100 dup 0

```

The above program will display the powers of two. This sequence is 1,2,4,8,16,32,64,128,256, etc., all the way until 18446744073709551616, which is two to the 64th power.

Under 32-bit Linux, there is no way to reach this number naturally. This algorithm uses something called [Arbitrary Precision Arithmetic](#).

As fancy as that sounds, it is really just using an array of bytes as if they were decimal digits. These lines define variables for the array and the length of the array.

```

length dd 1
array db 0x100 dup 0

```

The “length” starts at 1, and the total size of the array is 0x100 (256 in decimal) bytes, initialized to zero. These variables form the boundaries of several different loops in the program. This loop is what displays the current used parts of the array.

```

;this section prints the digits
mov ebx,[length]
array_print:
dec ebx
mov eax,0
mov al,[array+ebx]
call putint
cmp ebx,0
jnz array_print
call putline

```

The ebx register is set to the current value of length, which will be 1 at the start of the program. Then, ebx is decremented so that it is 1 less than the length. Remember, arrays range from 0 to the length minus 1 in Assembly, just like would be the case in C and

other languages.

During this loop, while ebx is not 0, we set eax to 0 and then load al (the lowest 8 bits of eax), with the byte at the address of the array plus the number in the ebx register. We call the putint function on this value to print the number in al.

After the currently used digits in the array are printed, another loop begins that adds the digits to themselves. Each one is loaded into al, then al is added to itself. The dl register, which is initialized to 0, is the “carry” variable. If the result of al+al is less than 10, we jump to the “less\_than\_ten:” label and write the new digit back to the array in that index. If, however, the digit in al is 10 or above, we have to subtract 10 and then set dl (our carry) to 1 so that the next digit we add to itself will also have the carry added to it.

```
;this section adds the edigits
```

```
mov dl,0
mov ebx,0
array_add:
mov eax,0
mov al,[array+ebx]
add al,al
add al,dl
mov dl,0
cmp al,10
jb less_than_ten
```

```
sub al,10
mov dl,1
```

```
less_than_ten:
mov [array+ebx],al
inc ebx
cmp ebx,[length]
jnz array_add
```

The process of this loop is basically the same way we would add the digits of numbers on paper. However, since we are adding the number to itself, the process is greatly simplified.

But perhaps the final piece of this powers of two program that needs a special mention is the part that expands how many digits are displayed by incrementing the length if a carry of 1 still remains. If the final digit processed had a result of 10 or greater, the carry in dl would have been set to 1, but there would not be a digit to add this to.

```
cmp dl,0
jz carry_is_zero

mov [array+ebx],1
inc [length]

carry_is_zero:
```

We set the byte at “array+ebx” to 1 and then increment the length variable so that the new digit becomes permanently part of the list of bytes that is displayed and added to itself,

plus the carry from each previous digit addition.

I hope I haven't lost you with my explanation of the arbitrary precision Powers of 2 program. The original version was written in my first programming language, QBASIC, and was the first time I had successfully learned how to use arrays.

At 14 years old, I was learning the concepts of arrays and memory addresses for the first time. I remember a very helpful user on the Network54 QBASIC forum explained it over and over again until I understood.

The syntax of the Assembly version of the Powers of 2 algorithm may look strange. Still, it follows all the same steps as the original QBASIC program and the C version, which later became part of "Chastity's Code Cookbook".

## Prime Numbers

format ELF executable

main:

```
mov dword [radix],10
mov dword [int_width],1
```

```
;the only even prime is 2
mov eax,2
call putint
call putspace
```

```
;fill array with zeros up to length
mov ebx,0
array_zero:
mov [array+ebx],0
inc ebx
cmp ebx,length
jb array_zero
```

```
;start by filtering multiples of first odd prime: 3
mov eax,3
```

primes:

```
;print this number because it is prime
call putint
call putspace
```

```
mov ebx,eax ;mov eax to ebx as our array index variable
mov ecx,eax ;mov eax to ecx
add ecx,ecx ;add ecx to itself
```

sieve:

```
mov [array+ebx],1 ;mark element as multiple of prime
add ebx,ecx ;check only multiples of this prime times 2 to exclude even
```

```

numbers
cmp ebx,length
jb sieve

;check odd numbers until we find unused one not marked as multiple of
prime
mov ebx,eax
next_odd:
add ebx,2
cmp [array+ebx],0
jz prime_found
cmp ebx,length
jb next_odd
prime_found:

;get next prime ready to print in eax
mov eax,ebx
cmp eax,length
jb primes
call putline

mov eax,1
mov ebx,0
int 0x80

include 'chastelib32.asm'

length=1000
array rb length

```

The primes program uses a method called the “Sieve of Eratosthenes”. It is an ancient but very fast algorithm to implement in almost any programming language. The program can find all primes less than 1000 in less than a second.

A Sieve is a process of elimination. Imagine you have all the numbers from 0 to 100. A prime number, by definition, has only two factors: itself and 1.

The only even prime number is 2. It is the first prime number because 1 times 2 equals 2. There are only two factors. Since all even numbers like 4,6,8,10,12, etc are multiples of 2. We exclude them from the list of possible primes. The new number, which is not crossed out, is 3. We then cross out all multiples of 3. Then the next number still in the list after 3 is 5. 4 doesn’t exist because we already excluded multiples of 2. 5 is our next prime number after 3 for this reason. We cross out all multiples of 5 like 10,15,20,25, etc. In fact, some of these would have already been excluded because they are multiples of 2. Any number that is not prime is called composite, which means it is composed by multiplying numbers which are prime.

In summary, the primes program has an array of 1000 bytes. We use each of these bytes as items to keep track of whether they are prime or not. Every item in the array starts as 0 (prime until proven otherwise). We print 2 because it is a known even prime. We then do the same for 3 because it is the first odd prime. Then we mark all indexes which are a

multiple of 3 as 1(not prime). We then skip ahead to the next odd index, which is not marked.

The result will be 2 3 5 7 11 13 17 19 23 29 31 37 41 43 47 53 59 61. The list contains all odd numbers after 2, and some of them are excluded because they are multiples of other odd numbers.

9 is not prime because 33 is 9. *21 is not prime because 37 is 21.* And so it continues.

This prime algorithm requires a lot of memory. Specifically in this implementation you need 1 byte for each number up to the limit you are checking so that everything can be marked as prime or composite.

However, this method is fast because it uses only addition and subtraction (excluding the division used in the `intstr` function of my library). On a modern PC running Linux it is easy to allocate gigabytes of memory and find lists of even higher primes.

If your PC is low on memory, you can even use disk space instead by seeking your way through a file and marking bytes of it prime and composite. Although disk space is slower than RAM, it is usually more abundant.

## Factorials

I have another example of Arbitrary Precision Arithmetic. This one generates the factorial sequence. Because multiplication of increasing numbers quickly generates long numbers, the built in integer registers are not equipped to handle numbers this large.

Just like the Powers of 2 program from this chapter, the following program uses arrays of decimal digits. I carefully translated it from the C version in [Chastity's Code Cookbook](#).

format ELF executable

main:

```
mov dword [radix],10
mov dword [int_width],1
```

```
mov eax,0
mov ebx,1
```

```
;fill all 3 array with zeros up to maxlength
```

```
mov ebx,0
```

```
array_zero:
```

```
mov [array_a+ebx],0
```

```
mov [array_b+ebx],0
```

```
mov [array_c+ebx],0
```

```
inc ebx
```

```
cmp ebx,maxlength
```

```
jb array_zero
```

```
mov [array_a],1 ;set low digit of array_a to 1
```

```

mov [array_b],2 ;set low digit of array_b to 2

;Keep track of the currently used length of each array.
;At the start, use only one digit
mov dword [array_a_length],1
mov dword [array_b_length],1
mov dword [array_c_length],1

mov edx,0 ;use edx as a counter for the main loop
main_loop:

;stage 1: display the a array
mov eax,0
mov ebx,[array_a_length]
stage1:
dec ebx
mov al,[array_a+ebx]
call putint
cmp ebx,0
jnz stage1
call putline

;stage 2: multiply the a and b arrays together and store the result in
the c array

mov ebx,0
stage2:

mov eax,0
stage2_multiply:

;we need to get the result of multiplication of the current digit
;indexed in array_a by eax and array_b by ebx
;the only safe way is to back up all the registers
;do a multiply operation, and then restore them

push eax
push ebx
push ecx
push edx

;mov eax and ebx to ecx and edx
;so that we can index the arrays
;using the low parts of eax and ebx as the result
mov ecx,eax
mov edx,ebx
;both eax and ebx are zeroed to avoid conflicts
;only the lowest 8 bits will be loaded from the arrays
;then we will do a multiply instruction
mov eax,0
mov al,[array_a+ecx]
mov ebx,0
mov bl,[array_b+edx]
add ecx,edx ;add edx to ecx before edx is overwritten with mul

```

```
mul ebx ;multiply eax by ebx
```

```
stage2_add_product:
```

```
add al,[array_c+ecx] ;add the byte at this index to al
mov ebx,[radix]      ;set the bl register to the radix
mov edx,0            ;clear edx before division
div ebx              ;divide eax by ebx
mov [array_c+ecx],dl ;move the remainder back to this index
```

```
inc ecx
cmp al,0              ;is the carry or quotient zero?
jnz stage2_add_product ;if not zero, go to next digit and repeat
```

```
cmp ecx,[array_c_length] ;is the index higher than current length of c
array?
jb c_digits_are_enough
mov [array_c_length],ecx ;expand digits
c_digits_are_enough:
```

```
;pop back the original values of the registers
pop edx
pop ecx
pop ebx
pop eax
```

```
inc eax
cmp eax,[array_a_length]
jnz stage2_multiply
```

```
inc ebx
cmp ebx,[array_b_length]
jnz stage2
;end of array multiplication stage
```

```
;stage 3: add 1 to the b array
push edx
mov eax,1 ;set carry to 1
mov ebx,0 ;start at lowest element of b
stage3_add_one_to_b:
add al,[array_b+ebx]
mov edx,0
div dword [radix]
mov [array_b+ebx],dl ;move the remainder back to this index
```

```
inc ebx
cmp al,0              ;is the carry or quotient zero?
jnz stage3_add_one_to_b ;if so, go to next digit and repeat
```

```
cmp ebx,[array_b_length] ;is the index higher than current length of c
array?
jb b_digits_are_enough
mov [array_b_length],ebx ;expand digits
```

```
b_digits_are_enough:
pop edx
```

```
;stage 4: replace array_a with array_c
;and turn array_c to all zeros to be used for next product
```

```
mov ebx,0 ;start at lowest element of both arrays
stage4:
```

```
mov al,[array_c+ebx] ;get element from array_c
mov [array_a+ebx],al ;store it here in array_a
mov [array_c+ebx],0 ;zero the byte in array_c
```

```
;next, expand length of array_a to same as array_c
mov eax,[array_c_length] ;get length of array_c
mov [array_a_length],eax ;set length of array_a
```

```
inc ebx
cmp ebx,maxlength
jnz stage4
```

```
inc edx
cmp edx,64 ;maximum factorial
jnz main_loop
```

```
mov eax,1
mov ebx,0
int 0x80
```

```
maxlength=1000 ;use this as maximum length of all arrays
array_a rb maxlength ;first array
array_b rb maxlength ;second array
array_c rb maxlength ;third array
```

```
;reserve one double word for each variable that will store the length
;the initial value is unknown but will be set in the program
array_a_length rd 1
array_b_length rd 1
array_c_length rd 1
```

```
include 'chastelib32.asm'
```

The factorials program is hard to explain without the ability to show all the math on paper. It uses the traditional method of multiplication as I was taught in school. Each digit in one number is multiplied by each digit in another number. For example, if you start at the one's place in the bottom number and multiply each digit in the top number by that digit, you have a partial result. The digits need to be added together while keeping their place values aligned. For example, consider this:

carries during multiplication

```
11
123
33
```

```

-----
    256 = array a
X   256 = array b
-----
    1536 = product of 6*256
    1280 = product of 5*256 shifted 1 left
    512  = product of 2*256 shifted 2 left

-----
    65536 = sum of the three products above

```

However strange the code may look, the process is very much like a human processing multiplication of numbers on paper. I have an above average ability to visualize it in my head. I hope the above example can help explain what it is doing even for those who don't understand the assembly code.

## Chapter 18: Bitwise Operations for Advanced Nerds

This chapter contains information which will assist you in understanding more about how computers work, but that in general is not required for MOST programming unless you are trying to operate on individual bits.

To start out, I will describe 5 essential bitwise operations independently of any specific programming language. This is because these operations exist in every programming language I know of, including Assembly and C.

After I have explained what the bitwise operations do, I will give examples of how this can be used in Assembly language to substitute for addition and subtraction! You might wonder why you would do this. The fact is that you don't need to but it is a fun trick that only advanced nerds like me do for a special challenge.

### The Bitwise Operations

This chapter explains 5 bitwise operations which operate on the bits of data in a computer. For the purpose of demonstration, it doesn't matter which number the bits represent at the moment. This is because the bits don't have to represent numbers at all but can represent anything described in two states. Bits are commonly used to represent statements that are *true* or *false*. For the purposes of this section, the words AND, OR, XOR are in capital letters because their meaning is only loosely related to the English words they get their name from.

### Bitwise AND Operation

```
0 AND 0 == 0
0 AND 1 == 0
1 AND 0 == 0
1 AND 1 == 1
```

Think of the bitwise AND operation as multiplication of single bits. 1 times 1 is always 1 but 0 times anything is always 0. That's how I personally think of it. I guess you could say that something is true only if two conditions are true. For example, if I go to Walmart AND do my job then it is true that I get paid.

I like to think of the AND operation as the "prefer 0" operation. It will always choose a 0 if either of the two bits is a 0, otherwise, if no 0 is available, it will choose 1.

### Bitwise OR Operation

```
0 OR 0 == 0
0 OR 1 == 1
1 OR 0 == 1
1 OR 1 == 1
```

The bitwise OR operation can be thought of as something that is true if one or two

conditions are true. For example, it is true that playing in the street will result in you dying because you got run over by a car. It is also true that if you live long enough, something else will kill you. Therefore, the bit of your impending death is always 1.

I like to think of the OR operation as the “prefer 1” operation. It will always choose a 1 if one of the two bits is a 1, otherwise, if no 1 is available, it will choose 0.

## Bitwise XOR Operation

```
0 XOR 0 == 0
0 XOR 1 == 1
1 XOR 0 == 1
1 XOR 1 == 0
```

The bitwise XOR operation is different because it isn't really used much for evaluating true or false. Instead, this operation returns 1 if the bits compared are different or 0 if they are the same. This means that any bit, or group of bits, XORed with itself, will always result in 0.

If you look at my XOR chart above, you will see that using XOR of any bit with a 1 causes the result to be the opposite of the original bit.

The XOR operation is the quickest way to achieve this bit inversion. If you have a setting that you want to switch on or off, you can toggle it by XORing that bit with 1.

While the AND, OR, XOR operations can work in the context of individual bits, or groups of them, the next operations, the bit shifts, only make sense in the context of a group of bits. At minimum, you will be operating on 8 bits at a time because a byte is the lowest addressable size of memory.

## Bitwise Left and Right Shift Operations

Consider the case of the following 8 bit binary value:

```
00001000
```

This would of course represent the number 8 because a 1 is in the 8's place value. We can left shift or right shift.

```
00001000 == 8 : is the original byte
```

```
00010000 == 16 : original left shift 1
```

```
00000100 == 4 : original right shift 1
```

That is really all there is to shifts. They can be used to multiply or divide by a power of two. In some cases, this can be faster than using the mul and div instructions described in chapter 6.

## Example 00: Fake Add

The following example shows how it is possible to write an addition routine using a combination of the AND,XOR,SHL operations. In this case, the numbers are shown in decimal to be easier for most people to see that the addition is correct.

format ELF executable

main:

```
mov dword [radix],10
mov dword [int_width],1
```

```
mov edi,1987
mov esi,39
```

```
mov eax,edi
call putint
call putline
mov eax,esi
call putint
call putline
```

```
fake_add:
mov eax,edi
xor edi,esi
and esi,eax
shl esi,1
jnz fake_add
```

```
mov eax,edi
call putint
call putline
mov eax,esi
call putint
call putline
```

```
mov eax,1
mov ebx,0
int 0x80
```

```
include 'chastelib32.asm'
```

If you run it, you will see that the correct result of 2026 which is 1987+39. These are the values we set the edi and esi registers to before simulating addition with these fancy bitwise operations that make even seasoned programmers run scared.

But how does this monstrosity of a program work? You see the AND operation keeps track of whether both bits in each place value are 1 or not. If they both are, this means that we have to “carry” those bits as we would do in an ordinary binary addition. We store the carry in the esi register and then left shift it once each time in the loop. The loop continues until esi equals zero and there are no more bits to invert with XOR.

The fact that it works is easy to work out in my head but I don't blame you if you can't visualize it. However, this shows the power of what bit operations can do, even though you will probably never need to do this.

## Example 01: Fake Sub

In case the fake addition example above wasn't enough for you, here is a slightly modified example that does a fake subtraction operation using the same operations. Try it out and you will see that it subtracts 38 from 2025 and gets the original 1987.

format ELF executable

main:

```
mov dword [radix],10
mov dword [int_width],1
```

```
mov edi,2026
mov esi,39
```

```
mov eax,edi
call putint
call putline
mov eax,esi
call putint
call putline
```

```
fake_sub:
xor edi,esi
and esi,edi
shl esi,1
jnz fake_sub
```

```
mov eax,edi
call putint
call putline
mov eax,esi
call putint
call putline
```

```
mov eax,1
mov ebx,0
int 0x80
```

```
include 'chastelib32.asm'
```

I will try to explain how this works. You see, we first XOR the edi register with the esi register. Then, we AND esi with the new value of edi. This means that the bits in the current place value will only both be 1 if those bits were 0 in edi and then were inverted to 1 by the XOR with esi. This means that at the start of the loop, destination bit=0 and source bit=1. 0 minus 1 means that we need to “borrow” (I hate that term because it is really stealing because we never give it back). We left shift esi as usual and then we keep

XORing the new borrow in esi until it is zero.

Also, you may have noticed that I never used the “cmp” instruction to compare si with zero in this examples. This is because the zero flag is automatically updated with most operations. In fact there are places in my standard library of functions (chastelib) where it wasn’t strictly required to compare with “cmp” but I added it for clarity so I could read my code and more easily remember what I was doing.

But let’s face it, the examples in this chapter are purely for showing off how advanced my knowledge of the binary numeral system and manipulating bits in ways no reasonable person should ever attempt. I must admit, it would be great for an obfuscated code contest to make a program with code that is unreadable to most humans.

And while I am on the subject of making unreadable code, why stop at simulating addition and subtraction? I can do multiplication and division too!

## Example 10 Fake Mul

format ELF executable

main:

```
mov dword [radix],10
mov dword [int_width],1
```

```
mov edi,6
mov esi,7
```

```
mov eax,edi
call putint
call putline
mov eax,esi
call putint
call putline
```

```
fake_mul:
mov eax,0
fake_mul_add_loop:
cmp esi,0
jz fake_mul_add_loop_end
test esi,1
jz skip_add
add eax,edi
skip_add:
shl edi,1
shr esi,1
jmp fake_mul_add_loop
fake_mul_add_loop_end:
mov edi,eax
```

```
mov eax,edi
call putint
```

```
call putline

mov eax,1
mov ebx,0
int 0x80

include 'chastelib32.asm'
```

This fake multiplication uses `eax` as a temporary variable to store the sum of repeated addition. Each time through the loop, we test the low bit of `esi` (source register) and add `edi` (destination register). Each time we double `edi` by left shifting it once while right shifting `esi` once.

To test the low bit of `esi`, this example uses an instruction that has not been presented in this book. However, this is one of those times when it should be used because it is the easiest way to test for a bit.

```
test esi,1
```

This tests the lowest bit of the `esi` register. If the low bit (ones place) is 0, then the jump if zero command will skip past the part where we add `edi` to `eax`.

The purpose of the bit shifts is to reduce the number of times we have to add `edi` to `eax`. Because 6 and 7 are the values we use, the loop will execute and exactly these results will happen.

`eax` starts as 0 but at the end of each cycle, `eax`, `edi`, and `esi` will be these values.

| cycle | eax | edi | esi |
|-------|-----|-----|-----|
| 0     | 6   | 6   | 7   |
| 1     | 18  | 12  | 3   |
| 2     | 42  | 24  | 1   |

In short, because `esi` was 7 and was represented as 111 in binary, because it is the sum of  $4+2+1$ , then the result of 42 is the sum of:

$$(4*6)+(2*6)+(1*6)$$

Because of this optimized algorithm, it took only 3 loop cycles instead of the usual 7 cycles if we had just added 6 to `eax` 7 times. The speed won't be much for such a small number as 42 but for larger numbers, this is the kind of optimization I would want.

## Example 11 Fake Div

For the final example, I have prepared the long division algorithm. I always wondered why they called it long division, but now that I wrote this 70 line long program just to divide 256 by 10, I finally understand.

Despite its complexity, it follows the same method that humans use for doing long division. We extract the bits from high to low positions from the dividend (`edi`) and shift them into the low bits of a temporary number (`ebx`). If this number is large enough (not below divisor) then we subtract the divisor. Each time, we save the resulting bit in the

quotient (eax). 0 means the number was too small to subtract the divisor and 1 means it was big enough to subtract the divisor.

Because this is a 32 bit system, the “fake\_div\_sub\_loop” needs to execute exactly 32 times. For this purpose, ecx was chosen as a counter. By initializing it to 1 and left shifting it each time, it will loop 32 times until ecx will be 0 when an overflow happens and the bit is lost.

The eax,ebx,and ecx registers are pushed and popped in this example, despite not being part of a function call. This is to signal that their purpose was temporary and because of the complexity, I thought it would be a good convention to follow for clarity that they are not the final return values.

format ELF executable

main:

```
mov dword [radix],10
mov dword [int_width],1
```

```
mov edi,256
mov esi,10
```

```
mov eax,edi
call putint
call putline
mov eax,esi
call putint
call putline
```

```
fake_div:
;save registers used in the long division algorithm
push eax
push ebx
push ecx
mov eax,0
mov ebx,0
mov ecx,1
cmp esi,0
jz fake_div_sub_loop_end ;div by 0 invalid
fake_div_sub_loop:
cmp ecx,0
jz fake_div_sub_loop_end
shl eax,1
shl ebx,1
test edi,edi ;test edi with itself to check sign bit
jns skip_or ;skip copy of sign bit if it was 0
or ebx,1 ;store a 1 in low bit of ebx based on sign
skip_or:
shl edi,1

;skip subtraction if ebx is below esi
cmp ebx,esi
```

```

jb skip_sub
sub ebx,esi
or eax,1
skip_sub:

shl ecx,1
jmp fake_div_sub_loop
fake_div_sub_loop_end:

;send results to correct registers and clean up
mov edi,eax ;copy quotient to edi
mov esi,ebx ;copy remainder to esi
;restore registers to their original values
pop ecx
pop ebx
pop eax

mov eax,edi
call putint
call putline

mov eax,esi
call putint
call putline

mov eax,1
mov ebx,0
int 0x80

include 'chastelib32.asm'

```

The results of the fake division program are here:

```

256
10
25
6

```

The program correctly divides 256 by 10 which is 25 but with a remainder of 6. Although this program is probably the most difficult to understand, it gives the correct results.

Which can only mean that the long division algorithm I have in my autistic head works because I used to perform these operations on paper a lot and I know the process so well that I created this example.

## Will I need to know these Algorithms?

Believe it or not, there may be times when you will need to use these algorithms. There are some older processors which don't have multiplication and division instructions. For Intel processors, you can already use the real mul and div instructions.

But I have a feeling that somewhere in the hardware of those instructions on the Intel CPUs, they probably work a lot like the algorithms I wrote for this chapter.

When I promised you Assembly Arithmetic Algorithms, I was not joking. I have spent 26 years learning computer programming and I always liked to learn everything whether other people considered it practical or profitable.

## Chapter 19: Getting Started with Linux

If you are already running a Linux operating system, feel free to skip this chapter. However, if you are reading this book and want to get started with Linux, this chapter is for you. I can tell you how I got started with Linux 21 years ago and offer some tips to get started with easy to use distros (a short term we needs use for specific distributions of Linux.)

This whole book was written on Debian version 12 (bookworm). This has been my Linux distribution on my desktop computer since at least 2024 when Microsoft Windows decided to overwrite my bootloader when I had a dual boot between Windows and Ubuntu. I installed the newest version of Debian available at the time and have never looked back.

I got my start in the Linux world back in 2005 during the time of Ubuntu 5.10 (Breezy Badger). Ubuntu was my first distribution and I used it without problem for years. But since Ubuntu was based on Debian, I decided to go with the original distribution. I highly recommend either of them because both of these projects have great websites with detailed installation guides.

<https://www.debian.org/>  
<https://ubuntu.com/>

If you have an old machine running Windows but it is kind of slow, I recommend following the official instructions and installing one of these distributions of Linux to your hard disk by booting either a live CD or USB drive and following the instructions. I figured it out with no background in Linux when I was 18 years old. They have done great work making Linux easy for beginners.

But what I am going to do is share a method of trying out Linux for people who don't have a spare computer in their house and they can't risk deleting Windows because their job or school requires specific Windows software.

Just as I used a DOS emulator to emulate DOS for the DOS edition of [Assembly Arithmetic Algorithms](#), it is also possible to run Linux in an emulator.

There are some things you should know about running Linux in an emulator.

1. Emulators are slower than real hardware and cannot be used to judge Linux. Just because you ran Linux in an emulator and it was slow does not mean that the operating system is a failure or that you should give up the idea.
2. Emulators have many different options that you should read about and the reason I recommend learning how to use an emulator is because it teaches you to understand hardware better. Files are used as virtual compact disks or hard disks but the emulators see them as being the same as real hardware.

3. Portability is the best reason to use an emulator. Because an emulator can run software for the same machine type or even a different architecture, you can use the information in this book even if you are running an Intel emulator on an ARM or RISC-V processor.

The emulator I will be using for this example is called QEMU.

<https://www.qemu.org/>  
<https://www.qemu.org/docs/master/system/invocation.html>

And the distro I will be using is called Tiny Core Linux.

<http://tinycorelinux.net/>

You will want to go to the downloads page and get the smallest of the three files. It will be named “Core-current.iso”.

Install QEMU on whatever OS you are currently using. QEMU is available for Windows, Mac, and Linux.

Once you have QEMU installed and downloaded “Core-current.iso” to a directory somewhere, you can launch it with QEMU by using this command.

```
qemu-system-x86_64 -drive file=Core-current.iso,media=cdrom
```

You can also use the shorter form of the command which means the same thing.

```
qemu-system-x86_64 -cdrom Core-current.iso
```

Either way, it will boot into a tiny Linux system that is only a command line. You can use standard Linux commands such as `ls`, `cat`, `sed`, `cp`, `rm`, `cd`, and `exit`. There is even the “vi” editor built in. If you have not used the vi or vim editors before, don’t worry about it. However, if you have used them, go ahead and practice by making some text files and saving them.

When you are done using Tiny Core Linux, just use this command:

```
sudo poweroff
```

## How does Tiny Core work?

Everything will shut down and the emulator will also stop. If you clicked on the QEMU Window and your mouse cursor disappeared, press `Ctrl+Alt+G` on your keyboard to grab it back from the emulator so you can do other things while the emulator is still running Tiny Core Linux. `Ctrl+Alt+F` can turn on or off full screen mode. This allows you to see the terminal cover your whole screen and look more like it would if you had booted it natively.

Tiny Core Linux runs entirely in RAM when you run it using only the ISO image for the cdrom drive. Any changes you make will not be saved when you exit the emulator.

I am sure you are wondering what good an operating system is that doesn’t save your

work. For that, I have 2 answers.

1. Because Tiny Core starts fresh each time using only the files in the CD image, it means each time you have a new chance to do something new and test things without any fear of messing up your system.
2. There is a way to install it to a hard disk and configure it to save changes. I will be explaining more about this in this chapter.

## Installing Software Temporarily

Before trying to permanently install Tiny Core to a hard disk, it is important to understand the package manager that it uses. It is called tce-ab. You can run it by its name.

```
tce-ab
```

It has a lot of text with instructions that let you search for and install packages. For example, on my machine when I booted into Tiny Core on QEMU, I was able to search for and install the “nano” text editor and the “nasm” assembler.

You can also directly install programs with the tce-load program. This is often faster than navigating the menus of tce-ab.

```
tce-load -wi nano
```

However, because even previously installed programs will be gone on the next reboot, it is time to teach you how to create a virtual hard disk and reboot into the emulator with a hard disk that we will install to.

## Installing to a Hard Disk

First, exit the emulator and then use the qemu-img command below to create a 1 Gigabyte empty file that will be used as a hard disk.

```
qemu-img create haddisk.img 1G
```

The file will be 1 gigabyte in size. This is large enough to hold software you may want to install on it later but also small enough to load into RAM comfortably or copy to another disk for transferring to another computer. It contains nothing right now but with a modified QEMU command, we can reboot from the cdrom and then install Tiny Core to the virtual hard disk we created.

```
qemu-system-x86_64 -drive file=Core-current.iso,media=cdrom -drive  
file=haddisk.img,format=raw,media=disk
```

The Tiny Core install program is ironically not included in the cdrom file we are booting from. Therefore, it becomes necessary to install the installer!

While you are booted into QEMU with both the cdrom and haddisk images, install the install script with the tce-load command below.

```
tce-load -wi tc-install
```

It will take some time to install the dependencies of the Tiny Core install script. Notable dependencies are Perl (a popular scripting language) and syslinux (the bootloader).

After it finished installing, run this command:

```
sudo tc-install.sh
```

The installer asks a lot of questions about what options you want when you want to install. These are the options I used if you are in doubt or don't understand everything it is asking you.

- i for installing from booted cdrom
- f for frugal hard drive installation
- 1 for whole disk installation
- 2 for sda (first hard disk)
- y for installing the bootloader
- 3 for the ext4 file system

Most other options you can leave blank and press enter.

I know that process may have been a little complex but it highlights the difference between installing Linux and installing Windows.

You don't install Windows. It was already on your computer when you bought it. This means somebody else chose all the options for you.

When you install Linux, you need to know some basic facts or at least google them when in doubt. This takes some time to learn but it gives you unrestricted control in the kind of system you are building for your software development.

In fact Tiny Core provides a helpful book that explains some of these options.

<http://www.tinycorelinux.net/book.html>

The book covers the graphical interface which is available on a much larger ISO file. I chose to go the pure command line only route and adapt the instructions for the tc-install.sh script which only got a passing mention in the book.

When you have completed those steps, it is now possible to boot directly into the hard disk without the cdrom!

```
qemu-system-x86_64 -drive file=harddisk.img,format=raw,media=disk
```

The main benefit of installing to a hard disk instead of just booting from the cd image each time is that any programs you install with tce-ab or tce-load will stay there and be available to use each time you reboot.

## Making a Persistent Home Directory

However, the files in your home directory are deleted each time you reboot. However, there is an easy fix for this. We will open the bootloader configuration and add an option to restore the backup from sda1.

First, open the config file:

```
nano /mnt/sda1/tce/boot/extlinux/extlinux.conf
```

And add this option to the line that starts with APPEND.

```
home=sda1
```

This means that the home directory will become an actual directory on the hard disk instead of a temporary location only loaded into RAM. With this step completed, you can begin programming in Assembly on your Tiny Core Linux system! Time to install the tools.

NASM is already in the repository and this is the standard way to install it.

```
tce-load -wi nasm
```

However, FASM is not available in the repository but it can be easily downloaded from the official website. However, this system only has a terminal with no web browser. Don't worry, there is a way! We can use the wget command that is already in Tiny Core Linux!

```
wget http://flatassembler.net/fasm-1.73.35.tgz
```

After the TGZ (Tar with GZip compression) file downloads, we need to extract the files from it.

```
tar -xf fasm-1.73.35.tgz
```

Now you can change to the fasm directory that was just created.

```
cd fasm
```

Inside are several files including "fasm" (the actual executable that can assembly all the programs in this book), "fasm.txt" the manual in plain text that describes both how to use FASM and also introduces the Intel instruction set. The "license.txt" may also be of interest.

To install fasm, we should add it to somewhere where it can be in the current path. Check what the path is currently with this command.

```
echo $PATH
```

One of the directories that is in the path is "/home/tc/.local/bin". This gives us the information we need to install fasm permanently there. Just copy fasm there like this:

```
cp fasm ~/.local/bin
```

Now enter "fasm" as a command and see what happens. If you see the following output, then it was done correctly!

```
flat assembler version 1.73.35
```

```
usage: fasm <source> [output]
```

```
optional settings:
```

```
-m <limit>      set the limit in kilobytes for the available memory
-p <limit>      set the maximum allowed number of passes
-d <name>=<value> define symbolic variable
-s <file>       dump symbolic information for debugging
```

This means that you can assembly and run any program that I have included in this book,

## [Assembly Arithmetic Algorithms for Linux.](#)

Keep in mind however that programming in this terminal based environment may feel different because you have to use terminal text editors like vi,vim, or nano. These text editors are just as good as graphical ones except that there is no mouse support. However, I was hoping to give you the retro feeling of programming like a crazy person in the 1980s.

This is just the tip of the iceberg about what Tiny Core Linux can do. However, I wanted to at least spend this chapter introducing this lesser known distro because it is small enough that it emulates with QEMU flawlessly.

Although Debian, Ubuntu, Linux Mint, and many others are better than Tiny Core because they have more programs in their repositories, they are too large to emulate because their graphical X Windows System displays use a lot more memory and it is harder for a PC emulator like QEMU to emulate them properly.

Consider Tiny Core Linux as a way to test the waters and get used to running commands in a Linux terminal before you are ready to install a more mainstream distro like Debian. However, once you have made the jump into the wonderful world of Linux, you will not be disappointed as a programmer because there is no shortage of text editors, assemblers, compilers, and even video games. For example, Final Fantasy 6 and Chrono Trigger from my Steam game collection both work on my Debian Linux system. Ironically, my Windows 11 laptop could not run Chrono Trigger.

I wrote this chapter to do my small part in the world to help people discover the things I like about Linux and the freedom it offers.

## **Tiny Core Linux makefile**

The following makefile is something I use to run commands for emulating Tiny Core Linux inside QEMU. I use it to remind myself what commands I use to set up a new installation of Tiny Core Linux.

```
Core-harddisk:
    qemu-system-x86_64 -drive file=harddisk.img,format=raw,media=disk
Core-cdrom:
    qemu-system-x86_64 -drive file=Core-current.iso,media=cdrom
Core-cdrom-short:
    qemu-system-x86_64 -cdrom Core-current.iso
Core-cdrom_and_harddisk:
    qemu-system-x86_64 -drive file=Core-current.iso,media=cdrom -drive
file=harddisk.img,format=raw,media=disk
harddisk:
    qemu-img create harddisk.img 1G
```

Of course, this chapter also serves as a reminder to myself about how to use Tiny Core Linux and also why it is one of my favorite distributions of Linux. I like it because sometimes I just want to have a text-only system upon which I can improve my scripting

and command line usage skills. Modern Linux distros include Graphical User Interfaces where you can point and click your way to doing anything, but from the beginning it was not this way. Therefore, I always promote learning the Linux terminal because it is the one thing that stays consistent and trustworthy no matter which distro you are using at the time.

I will admit that using the command line is a huge learning curve when you are first getting started. Perhaps I found it easier because I grew up with MS-DOS where text commands were all I had. In any case, knowing how to navigate directories on Linux to edit, copy, rename, and delete files is helpful because there are so many different graphical user interfaces and I can't remember which menus to click on.

But the real reason text commands are the best is because text can be copy pasted and then I am able to give you the exact commands to run so that you don't have to spend months google searching for the right documentation on all these different things!

But because I am a nerd, this was not my first time installing a Linux operating system and I promise you that once you have successfully installed Linux, you will find it gets easier each time.

The reason I chose Tiny Core Linux as the example for this chapter was because I have tried many of them and Tiny Core had the best documentation and the core image was extremely small. It was perfect for emulating and the official book gave me what I needed to know to try it myself so I could tell you!

## Chapter 20: chastdin for Linux

For the final chapter of this book, I have created another monster of a program. It uses only concepts that I taught in this book, but may be complex even for assembly programmers more advanced than I am.

It uses keyboard input as taught in chapter 8, where I read from stdin and then exit the program when the user enters “exit” as the string.

But it does a lot more than that. It also takes all of the commands from the chystack program from chapter 10. All the same commands of add,sub,mul,div, and rem work the same as before, except that they are just entered from the keyboard while the program is running instead of as arguments before the program is started.

This is a complete Reverse Polish Notation calculator, and it is the largest Assembly language program I have ever written in my life. The full source code is 24 kilobytes but assembles to only 2 kilobytes.

Like most other programs in this book, it includes “chastelib32.asm” from chapter 10 of this book. However, it also includes a new header file named “chastdin32.asm” which includes the “getstring” and “strcmp” functions from chapter 8 plus a few more that are not used in this book but will be for future programs I write.

The source files are included below and can be copied and pasted, but it might be easier to download them from my repository directly here:

<https://github.com/chastitywhiterose/chastdin>

### main.asm for chastdin

```
format ELF executable
entry main
```

```
include 'chastelib32.asm'
include 'chastdin32.asm'
```

```
main:
```

```
mov dword[radix],10      ;I can choose the radix for integer output!
mov dword[int_width],1   ;and the width of each integer for padded zeros
```

```
mov ebp,chastack          ;mov the address of the beginning of the stack to
ebp registers
```

```
;this program does not read command line arguments
;it only reads from stdin (STanDard INput)
;it is a complete Revese Polish Notation calculator
```

```
call help ;display brief help message at the beginning of program
```

```

mov [last_char],0xA ;set newline as last_char so prompt will display

main_loop:

;show the arrow indicating we wait for the user to enter something
;but only show it when the last character is a newline
;otherwise it will print too many if multiple commands were entered on
the same line
cmp [last_char],0xA
jnz skip_prompt
mov eax,string_prompt
call putstring
skip_prompt:

call getstring ;get string and return address in eax

;if there were 0 characters read in the getstring function
;it means that standard input was redirected from a file or another
command
;or that Ctrl+D was pressed on the keyboard
;we must exit the program now to stop an infinite loop

cmp dword[count],0 ;were there zero characters read?
jz command_exit ;reached end of standard input, exit program

;we must restart the loop in case of an empty string
;if we didn't, strint would read the empty string and return 0
;then zero would be pushed to the stack, which is not what we want
;we can check for an empty string by checking if
;one character was read in the last call to getstring
;on Linux, this will usually be 0x0A or the newline character

cmp dword[count],1 ;was only one character (newline) read?
jz main_loop ;if yes, this was an empty string, retry input

mov esi,eax ;mov string to esi for string comparison

;Now we process the string the user entered
;First, we will try testing for commands
;If any of the predefined strings match the string in esi
;We jump to the label for that command

mov edi,string_setradix
call strcmp
jz command_setradix

mov edi,string_add
call strcmp
jz command_add

mov edi,string_sub
call strcmp
jz command_sub

```

```

mov edi,string_mul
call strcmp
jz command_mul

mov edi,string_div
call strcmp
jz command_div

mov edi,string_rem
call strcmp
jz command_rem

mov edi,string_putstack
call strcmp
jz command_putstack

mov edi,string_clear
call strcmp
jz command_clear

mov edi,string_help
call strcmp
jz command_help

mov edi,string_exit
call strcmp
jz command_exit

;The default command is to turn the argument into a number and push to
stack
command_num:

mov eax,esi           ;mov the string to eax for processing numbers
call strint           ;try to get a number from the string pointed to by
eax
cmp [strint_error],0  ;did we have zero errors in the strint function?
jz num_push           ;if there were no errors, push this to stack

mov eax,string_err
call putstring        ;print error message
mov eax,esi
call putstring        ;print which command failed
call putline
jmp num_push_end      ;skip the push because this can't be used

num_push:             ;push the number to the fake stack
add ebp,4             ;increment the pointer by the size of the native
int for this mode
mov [ebp],eax         ;mov the value we converted from the string with
strint
num_push_end:
jmp main_loop         ;once value is pushed, continue the program

```

```

;These are the labels and code for each of the commands
;When a command is done, we jump back to the beginning of the loop
;the add,sub,mul,div,rem commands are pretty self explanatory
;but I will provide comments for these and other commands

;pop top of stack and set the current radix to it
;it has error checking and leaves the radix as is
;unless at least one number is on the stack
command_setradix:
cmp ebp,chastack      ;is ebp above the address of stack start?
jna change_radix_no   ;if not above, we cannot use it to set the radix
change_radix_yes:
mov eax,[ebp]         ;get the top of stack
mov [radix],eax       ;change the radix
mov dword[ebp],0      ;erase the top of stack
sub ebp,4             ;subtract pointer
jmp main_loop         ;and continue main_loop as normal
change_radix_no:
mov eax,string_err1   ;get error message for less than 1 numbers on
stack
call putstring        ;print error message
mov eax,esi           ;get name of the command used
call putstring        ;print which command failed
call putline
jmp main_loop

;add number on top of stack to the one below it
command_add:
mov eax,[ebp]
sub ebp,4
add [ebp],eax
jmp memory_check ;check stack for errors after this command

;subtract number on top of stack from the one below it
command_sub:
mov eax,[ebp]
sub ebp,4
sub [ebp],eax
jmp memory_check ;check stack for errors after this command

;multiply number on top of stack by the one below it
command_mul:
mov ebx,[ebp]
sub ebp,4
mov eax,[ebp]
mov edx,0             ;zero edx before multiply
mul ebx               ;multiply eax with value in ebx
mov [ebp],eax
jmp memory_check ;check stack for errors after this command

;divide number on top of stack into the one below it
command_div:
mov ebx,[ebp]
sub ebp,4

```

```

mov eax,[ebp]
mov edx,0 ;zero edx before divide
div ebx ;divide eax with value in ebx
mov [ebp],eax ;store quotient on stack
jmp memory_check ;check stack for errors after this command

;divide number on top of stack into the one below it
;but leave remainder instead of quotient
command_rem:
mov ebx,[ebp]
sub ebp,4
mov eax,[ebp]
mov edx,0 ;zero edx before divide
div ebx ;divide eax with value in ebx
mov [ebp],edx ;store remainder on stack
jmp memory_check ;check stack for errors after this command

;check if the stack has enough space for the last command
;this will print an error if less than two numbers were on the stack
;when using one of the math commands above
memory_check:
cmp ebp,chastack ;is ebp above the address of stack start?
jna print_stack_error ;if not above, explain error to user
mov dword[ebp+4],0 ;if no error, erase the old top of stack
jmp main_loop ;and continue main_loop as normal
print_stack_error:
mov eax,string_err2 ;get error message for less than 2 numbers on stack
call putstring ;print error message
mov eax,esi ;get name of the command used
call putstring ;print which command failed
call putline
add ebp,4 ;increment the pointer to what it was before the
failed command
jmp main_loop

command_putstack: ;print all numbers on the stack
push ebp ;save value of ebp
command_putstack_loop:
cmp ebp,chastack ;is ebp equal to the address of stack start?
jz command_putstack_end ;if it is, end the putstack loop
mov eax,[ebp]
sub ebp,4
call putint_and_line
jmp command_putstack_loop
command_putstack_end:
pop ebp ;restore ebp to what it was before this command
jmp main_loop

command_clear: ;erase all numbers on the stack
command_clear_loop:
cmp ebp,chastack ;is ebp equal to the address of stack start?
jz command_clear_end ;if it is, end the clear loop
mov dword[ebp],0
sub ebp,4

```

```

jmp command_clear_loop
command_clear_end:
jmp main_loop

command_help:
call help
jmp main_loop

command_exit: ;end the program

mov eax,1          ;exit (kernel opcode 1 on 32 bit systems)
mov ebx,0          ;return 0 status on exit - 'No Errors'
int 80h           ;system call for 32-bit Linux kernel

argc dd 0

string_setradix db 'setradix',0
string_add db 'add',0
string_sub db 'sub',0
string_mul db 'mul',0
string_div db 'div',0
string_rem db 'rem',0
string_help db 'help',0

string_exit db 'exit',0
string_putstack db '?',0
string_clear db 'clear',0

string_prompt db '-> ',0

string_err db 'Error: invalid number or command: ',0 ;Generic error
message
string_err1 db 'Error: need one number on stack for command: ',0 ;math
fail error when less than one number on the stack
string_err2 db 'Error: need two numbers on stack for command: ',0 ;math
fail error when less than two numbers on the stack

chastdin_help db 'chastdin is a stack based interactive calculator',0xA
               db 'that reads stdin for numbers and commands.',0xA
               db 'Numbers are pushed on the stack for all math.',0xA
               db 'Each line can contain multiple numbers or
commands.',0xA,0xA
               db 'Arithmetic commands are add,sub,mul,div,rem',0xA
               db 'The exit command ends the program',0xA
               db 'The ? command prints the entire stack',0xA
               db 'The setradix command changes the radix for input and
output',0xA,0xA
               db 'See readme.md for full help',0xA,0

;a function to print the help message defined above
;for how to use this calculator program
help:
mov eax,chastdin_help
call putstring

```

ret

```
;This program uses a virtual stack for convenience and portability
;I allocate memory for a virtual stack that we can index as if it was
the real stack
;I name it "chastack" for Chastity's stack.
```

chastack: rd 0x100

## chastdin32.asm

```
;Chastity's Standard Input header file
;The functions here are designed to read strings and numbers from
standard input.
```

```
;getstring ;read characters from stdin until the first whitespace
;getline   ;read characters from stdin until the first
newline,EOF,tab,etc.
;strcmp    ;compare two strings similar to the same function in C
;strlen    ;get length of string similar to the same function in C
```

```
;these variables are used as the default controllers
;for the getstring and getline functions
;buf stores keyboard input during those functions
;count stores how many bytes were read during system read calls
;last_char stores the last character read
;usually this will be a space, tab, or newline
```

```
buf db 0x100 dup '?'
count dd 0
last_char db 0
```

```
;summary
;the getstring function is the reverse function of putstring
;instead of printing a string to standard output
;it reads a string from standard input (AKA the keyboard)
```

```
;details
;the getstring function is designed to get a string of text
;which is terminated by whitespace or any non printable character
;the idea is that multiple strings can be passed on one line
;separated by spaces, similar to command line arguments
;this function was written for the specific purpose of converting any of
my programs that used command line arguments to read from stdin instead
```

getstring:

```
mov [count],0 ;set count of characters read during this function to zero
mov edx,1      ;number of bytes to read
mov ecx,buf    ;address to store the bytes
```

getstring\_chars:

```
mov ebx,0      ;read from stdin
```

```

mov eax,3      ;invoke SYS_READ (kernel opcode 3)
int 80h        ;call the kernel

cmp eax,1      ;was 1 character read?
jz getstring_read_yes ;if yes, process this character and add to string
ret           ;otherwise exit the function now

getstring_read_yes:
add [count],eax ;add how many characters we have read
mov al,[ecx]    ;mov last character read into al register

;check if this character is in the proper range to be part of the string

cmp al,0x21     ;compare with 0x21 (!=exclamation)
jb getstring_end ;jump if below to getstring_end label
cmp al,0x7E     ;compare with 0x7E (tilde)
ja getstring_end ;jump if above to getstring_end label

;if neither jump happened, keep the character and

inc ecx        ;increment address where next byte is read from
jmp getstring_chars ;jump back to start of loop and keep reading

getstring_end:

mov [last_char],al ;save the last character read
mov byte[ecx],0 ;terminate this string with a zero

mov eax,buf ;mov the buffer address to eax for returning the string

ret

;the getline function gets an entire line of text from the keyboard
;calling this function allows for a string that can contain spaces
;it considers as anything outside the range of 0x20 to 0x7E as the end
of line character
;this is because the end of the line might be 0x0A on Linux
;or it might be 0x0D,0x0A on DOS or Windows.
;technically, it means tab will also terminate a line
;the intended use of this function is to read a filename
;This makes sense because filenames can contain spaces

getline:

mov [count],0 ;set count of characters read during this function to zero
mov edx,1     ;number of bytes to read
mov ecx,buf   ;address to store the bytes

getline_chars:

mov ebx,0     ;read from stdin
mov eax,3     ;invoke SYS_READ (kernel opcode 3)
int 80h       ;call the kernel

```

```

cmp eax,1          ;was 1 character read?
jz getline_read_yes ;if yes, process this character and add to string
ret                ;otherwise exit the function now

getline_read_yes:
add [count],eax    ;add how many characters we have read
mov al,[ecx]       ;mov last character read into al register

;check if this character is in the proper range to be part of the string

cmp al,0x20        ;compare with 0x20 (space)
jb getline_end     ;jump if below to getstring_end label
cmp al,0x7E        ;compare with 0x7E (tilde)
ja getline_end     ;jump if above to getstring_end label

;if neither jump happened, keep the character and

inc ecx            ;increment address where next byte is read from
jmp getline_chars  ;jump back to start of loop and keep reading

getline_end:

mov [last_char],al ;save the last character read
mov byte[ecx],0    ;terminate this string with a zero

mov eax,buf ;mov the buffer address to eax for returning the string

ret

;Short Description of strcmp:
;strcmp compares the string at esi to the one at edi
;eax returns 0 if the strings are the same and non zero if different
;the algorithm is simple but I will explain it for those who are
confused

;Long Description of strcmp:
;eax is initialized to zero
;a byte from each string is loaded into the al and bl registers
;the bytes are compared. if they are different, then we jump to the end
;However, if they are the same, then we check if one of them is zero
;for this purpose it doesn't matter whether we compare al or bl with
zero
;because it is known that they are the same if the jnz did not take
place
;if it is zero, this also jumps to the end of the function
;If neither jump took place, then we jump to the start of the loop
;but when the function finally ends bl will be subtracted from al
;this ensures that the function returns zero if the final characters are
the same
;ebx,esi,and edi are preserved but eax is the return value
;also, the sub instruction at the end of the function also updates the
flags
;so you can "jz" or "jnz" to a label after calling this function based
on results

```

;This makes strcmp as useful for strings as the Intel "cmp" instruction is for integers

strcmp:

```
push ebx
push esi
push edi
```

```
mov eax,0
```

strcmp\_start:

```
;read a byte from each string
mov al,[edi]
mov bl,[esi]
cmp al,bl
jnz strcmp_end
```

```
cmp al,0
jz strcmp_end
```

```
inc edi
inc esi
```

```
jmp strcmp_start
```

```
strcmp_end:
sub al,bl
```

```
pop edi
pop esi
pop ebx
```

```
ret
```

;Short Description of strlen:

;The strlen function gets the length of string in eax and returns it in eax

;This is the same algorithm used in my putstring function but is independent of an operating system.

;Long Description of strlen:

;The strlen function is rarely used but there are time when knowing the length of a string is helpful.

;First, I needed it for my chastext program when I wanted to compare strings

;To search for text in a file, I had to first get the length of the string which was being searched for.

;By knowing the string length, I can read that many bytes from the file.

;That way, if fewer bytes were read from the file than required, I can end the program without requiring strcmp to be called.

;Comparing incomplete data would give untrackable results.

;The second time I might need string length is when I am converting a

number to a string in a specific radix using `intstr`.  
 ;If I know how many characters are in the highest number in an integer sequence,  
 ;I can then customize the integer width so that all digits are lined up.  
 ;My `chastelib` library was designed with integer sequences as the priority.

`strlen:`

```
push ebx
mov ebx,eax      ;copy eax to ebx. ebx will be used as index to the
string
```

```
strlen_start:    ;this loop finds the length of the string
```

```
cmp byte[ebx],0 ;compare byte at address ebx with 0
jz strlen_end   ;if comparison was zero, jump to loop end
inc ebx
jmp strlen_start
```

```
strlen_end:
sub ebx,eax      ;subtract start pointer from current pointer to get
length of string
mov eax,ebx      ;copy the string length back to eax
pop ebx
```

```
ret
```

If you have assembled the program, the next thing you need to know is how to use it! Therefore, I wrote the following documentation or “readme” which fully explains how to use it.

## chastdin readme

This is the full documentation for the program known as “chastdin”. This name stands for “Chastity’s STandard INput”. The program serves two purposes.

The first purpose of this program was to test Chastity’s library of functions for reading keyboard input and comparing the strings entered with predefined strings to control program execution.

The second purpose was putting all the functions together to form a complete program that not only tests the standard input library, but also does something useful. When you first run the program, it will display a message like the following

```
chastdin is a stack based interactive calculator
that reads stdin for numbers and commands.
Numbers are pushed on the stack for all math.
Each line can contain multiple numbers or commands.
```

```
Arithmetic commands are add,sub,mul,div,rem
The exit command ends the program
The ? command prints the entire stack
```

The setradix command changes the radix for input and output

This is an accurate, but incomplete documentation of what the program does. Therefore, I have described each command below and how to use it. Some may be obvious while others deserve a longer explanation.

## **add**

The “add” command is a 3 letter word that does exactly when you would expect. Here is an example of how you could use it.

```
1987 40 add ?
```

This line will add the numbers 1987 and 40 to the stack, then it will add them and then use the ? command to display what is on the stack. The result in this case will be 2027 because that is the year that Chastity becomes the 40 year old virgin because she was born in 1987 on May 15th.

## **sub**

The “sub” command is a 3 letter word abbreviation for “subtract”. For example, the following commands will push 100 and 36 to the stack and subtract to get 64.

```
100 36 sub ?
```

One thing to notice is that there are no negative numbers in the chastdin program. So if you already have the number 64 on the stack from the previous commands and then tried to subtract 65:

```
65 sub ?
```

You will get an absurdly high number of 4294967295 if the data type of the implementation is a 32 bit unsigned integer. The specific data type can be changed in the C source code but the 32-bit Linux Assembly version is always this type because of the size of the Intel registers used.

## **mul**

The “mul” command is a 3 letter word abbreviation for “multiply”. For example, the following commands will push 5 and 15 to the stack and multiply to get 75.

```
5 15 mul ?
```

While multiplication is pretty straightforward, division uses two different commands described below.

## **div**

The “div” command is a 3 letter word abbreviation for “divide”. For example, the following commands will push 60 and 7 to the stack and multiply to get the quotient.

```
60 7 div ?
```

The quotient will be 8 because 7 can divide into 60 a maximum of 8 times. However, this is 56 and is not completely equal to 60. There is a separate command to get the remainder of division.

## **rem**

The “rem” command is a 3 letter word abbreviation for “remainder” of division. In some programming languages, the remainder is called the modulus. The following command can be used to get the remainder of division instead of the quotient.

```
60 7 rem ?
```

This will be 4 because the original number was 60 and the quotient is 56. The difference between these two numbers will always be 4.

Technically, you can also get the remainder doing something like this:

```
60 7 60 7 div mul sub ?
```

This works because the arithmetic always uses what is currently on the stack. By pushing 60 and 7 twice, we first divide for the quotient which is 8, then multiply that 8 by the 7 on the stack to get 56 which is then subtracted from the 60.

But don't worry, you don't need to do such madness because I put the rem command into the program already. But if you do understand why the longer version works, then you understand the full magic of why a postfix notation calculator works so well.

## **setradix**

The setradix command can change the radix or base that is being used for numbers. For example, the number 999 in decimal can be converted to hexadecimal.

```
999 16 setradix ?
```

It works because 999 pushed to the stack in the default decimal radix, but then 16 is pushed and then used to set the new radix to base 16 also called hexadecimal.

If you know about the binary and hexadecimal number bases, you will understand how important a conversion tool is when you are programming or just trying to hex edit a save file of your favorite retro video games.

## **exit**

Whenever you are done using the chasdin program, you can use the “exit” command. For example, the following will exit the program and then print “Hello World!” because once the program ends, the operating system will return control to the shell and then run literally any command on your system. Use this power wisely!

```
exit echo "Hello World!"
```

That is all you need to know about using the chasdin program. Understanding how it

actually works requires understanding the source code Chastity wrote.

However, because the program only reads standard input and then writes to standard output, it can be written in every programming language that exists without requiring external libraries.

## Epilogue

I admit the `chastdin` program is long and at first I considered whether it was too complicated to include in this book, but it is the perfect conclusion because it uses all of the core four functions of `chastelib` that I have been working on.

- `putstring` prints all information the user needs to read
- `intstr` converts all numbers into a string
- `putint` uses both `intstr` and `putstring` to display numbers
- `srint` converts strings to integers from user input entered and correctly fails if they are not a number

Besides those, it makes use of the following two functions:

- `getstring` reads keyboard input and stops at the first non-printable character
- `strcmp` can compare strings just like the C version from the C standard library.

Using these elements, useful programs can be constructed. The purpose of these functions and the example programs used in this book was to give new assembly programmers a foundation for how they can get started doing things that other languages like BASIC, Pascal, and C include in their standard libraries by default.

Assembly language is not about writing code fast nor about making money. Other languages allow faster development and easier debugging, but only Assembly really teaches you to understand the math and logic behind how your computer actually works.

I firmly believe that for humans to understand and enjoy things is the point of life. It might surprise you that as someone who loves computers that I am not a fan of Artificial Intelligence. I believe that real human intelligence and creativity is something that machines cannot replace. However, then a person writes a computer program, you can see their thought process and their specific choices in how they wrote each function, chose their variable names, and wrote comments on how it works so that future generations of humans can not only learn from it, but also improve on these designs and share them with others.

Sharing is what Free Software and Open Source is about. Linux is the best operating system for sharing code that is fast, efficient, low power consumption, and makes even autistic people like me smile because I know the secrets of how software works. I wrote this book to pass the torch on to you, dear reader!

# Assembly Magic

A programmer is not a magician  
They just know how to use addition  
Those with skill and high ambition  
Don't hesitate to break tradition

Reverse addition is subtraction  
There is no need for fancy abstraction  
Do not fall for hype and distraction  
Don't hesitate to learn, take action

Repeated addition is called multiplication  
Despite its badly taught reputation  
Teaching math is my obligation  
With my books I will teach the nation

Subtraction loops can form division  
Conditional jumps make each decision  
Divide by the radix for integer vision  
But a zero divisor can cause a collision

Programming languages are all the same  
When arithmetic is your favorite game  
It is fun to choose each variable name  
But when my code fails, I take the blame

But of every language I have used  
I love writing Assembly the most  
And I wrote the chaste hex program  
Of which I sometimes like to boast

I like Assembly language because  
It gives me the complete control  
And brings back the satisfaction  
That the evil tech companies stole

Anyone can learn to write code  
That is what some people say  
And I agree with this statement  
When they learn in the right way

People hear that something is hard  
And so they never even try to start  
But if they did they would soon see  
That building software is an art

I wrote a book to teach my favorite  
[Assembly Arithmetic Algorithms](#)  
I am a bit too obsessed with math  
And others suffer from my autism

I write my books and comment my functions  
So that other people have a chance to read  
And learn what makes computers work  
And do all the tasks that humans need

And to those who don't yet understand  
They think my math is some kind of magic  
I rarely meet those who take the time to learn  
And my lonely pursuit is kind of tragic

But the special way I write my programs  
Is yet another form of Creative Writing  
And because of evil tech companies  
People like me cannot stop fighting

They fire people and replace them with AI  
But that will only work for a short while  
Because the code is of no use at all  
Unless it can make a human soul smile